CS 6120

Natural Language Processing

Instructor: Amir Tahmasebi, PhD

Fall 2026



Introduction to Machine Learning

Part I



Announcements

- *HW4 will be posted today,*
  - *due by Oct. 26, 11:59pm ET*
- *Final Project Team selection*
  - *due by Nov. 3, 11:59pm ET*
- *Final Project topics will be posted this week*

Final Project

Total Mark: 40%

Submissions: in-class short presentation + weekly one-pager updates + Report + Code

Team size: 2-3 students

Topics:
- Proposed by Instructor (on Canvas)
- Thesis-related
- Student suggestion

Criteria: Depending on the team size, the project completion should take on average 2-3 days per person: e.g. one method implementation per student

Final Project

Submission Material:

IMPORTAT NOTE: (all material should be zipped as one file named as TeamNumber.zip; e.g. 101.zip)

Final Report (PDF ONLY**)**

Report format: Scientific publication (single or double columns)

- **Title**
- **Authors**
- **Abstract**
- **Introduction**: motivation and what problem are you solving, what are existing challenges, what is your contribution (bullet points)
- **Prior Work**: literature survey. Make sure to include references
- **Methods**: describe your methodology in detail. Diagram, flowcharts are recommended.
- **Dataset**: where the data is coming from, labels, statistics on size (table format), data split (table format)
- **Results**: include tables, include different metrics
- **Discussion**: describe your scientific achievements and findings, challenges you faces, strengths and shortcomings. Provide examples from your test set
- **References**
- **Appendix:** include a user manual on how to navigate through your code and material you submit. Provide a user manual on how to setup your environment and run your code successfully

Final Project

Remarks:

Coding Material:

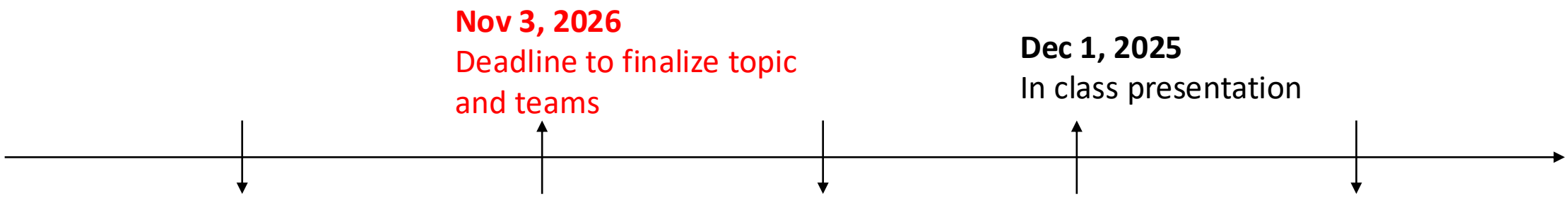
- Python code. Use classes and functions. .py or jupyter notebooks both acceptable.
- Provide comment for each class/function
  - What does it do, what is input and what is output
- Unit testing for all functions
  - Python: assert
- Provide a separate demo.py code (name EXACTLY as this). This code should provide two ways of running your demo: user provided input, pre-defined inputs.

Dataset:

- If data is available online, just provide link. No need to submit dataset as a part of your final submission.
- Processed data/Features: either provide code to re-generate / re-create or submit the processed data/features as a part of your submission.

Final Project

Timeline



Total of 4 Submissions: one every week

Final Project

Final Project Team and Topic Proposal

- *Scan QR or URL:*

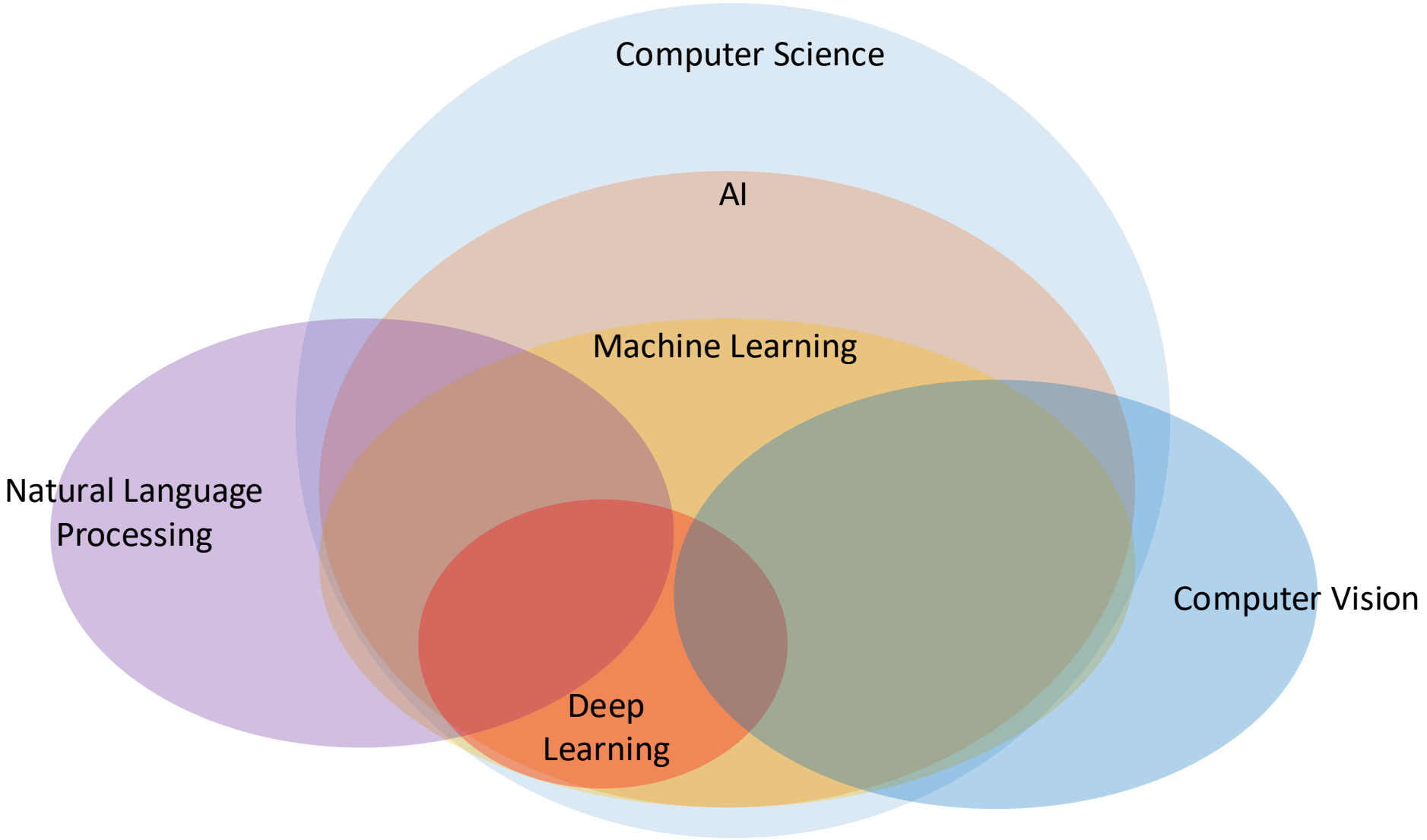
CS6120 - Fall 2026 - Final Project Submission Form – Fill out form



Syllabus

- What is Natural Language Processing?
  - History
  - Applications
- Python Primer
- Text Processing Fundamentals
  - Regular Expressions
  - Tokenization
  - Normalization
  - Libraries and Toolkits
- Parsing
  - Syntactic
  - Semantic
- Word Representation
  - TF-IDF
  - word2vec
  - Glove
- Intro to Machine Learning
  - Linear Regression
  - Logistic Regression (LR)
  - Clustering
  - Neural Networks
  - Recurrent Neural Networks (RNN)
  - Convolutional Neural Networks (CNN)
  - Transformers
- Language Models
- NLP Use Cases
  - Named Entity Recognition
  - Machine Translation
  - QA
- Large Language Models
  - chatGPT

Artificial Intelligence (AI)



Machine Learning: *Categories (by Purpose)*

- **Descriptive**
  - **Focus**: analyzing and summarizing historical data to reveal meaningful patterns, relationships, or trends. The goal is to answer “what happened” or “what is happening” by interpreting data and extracting insights.
  - **Use Cases**:
    - **Topic Modeling:** Automatically grouping text data (e.g., news articles, social media posts) by themes or topics.
    - **Summarization:** Generating concise summaries of long documents or articles to identify key points.
    - **Sentiment Analysis:** Assessing the overall emotional tone or attitude in tweets, reviews, or customer feedback.
  - **Examples**:
    - **Clustering** (e.g., K-Means, Hierarchical Clustering) for grouping similar text documents.
    - **Dimensionality Reduction** (e.g., PCA, t-SNE) for visualizing high-dimensional linguistic data.
    - **Association Rule Mining** to find common co-occurrences of words, phrases, or topics in large text corpora.

Machine Learning: *Categories (by Purpose)*

- Descriptive
- **Predictive**
  - **Focus**: forecast future outcomes or unknown values based on patterns learned from historical data. It addresses questions such as “what will happen” or “what might happen under certain conditions.”
  - **Applications**:
    - **Text Classification:** Predicting categories (spam vs. not spam, sentiment polarity) for new documents or messages.
    - **Intent Recognition:** Identifying the intent behind user queries in chatbots or virtual assistants.
    - **Next-Word Prediction / Autocomplete:** Suggesting likely next words while typing or interacting with a language model.
  - **Examples**:
    - **Classification Models** (e.g., Logistic Regression, Support Vector Machines, Transformers)
    - **Neural Networks** (e.g., RNNs, LSTMs, BERT, GPT-based models)
    - **Sequence-to-Sequence Models** (e.g., Machine Translation, text summarization)

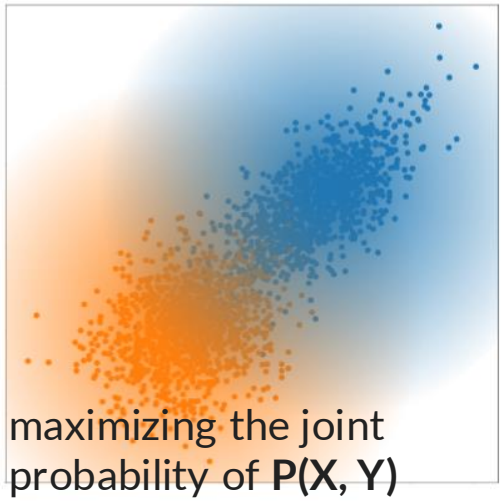
Machine Learning: *Categories (by Purpose)*

- Descriptive
- Predictive
- **Prescriptive**
  - **Focus**: goes beyond prediction to recommend actions or strategies that lead to desired outcomes. It addresses "what should be done" by leveraging the insights from predictive models and factoring in constraints and objectives.
  - **Applications**:
    - **Conversational AI Optimization:** Determining the best responses or dialogue paths in chatbots to improve user engagement and satisfaction.
    - **Content Recommendation:** Suggesting the optimal set of articles, books, or media to users based on their reading history and preferences.
    - **Policy and Decision Support:** Generating policy drafts or recommendation guidelines by analyzing large bodies of text and factoring in organizational objectives.
  - **Examples**:
    - **Reinforcement Learning** (training conversational agents that optimize for user satisfaction or task completion)
    - **Optimization Algorithms** to balance multiple objectives (e.g., personalization, diversity) in content recommendation.
    - **Advanced Language Generation** (e.g., fine-tuned GPT models) for automated report or policy draft creation, guided by goal-oriented constraints.

Machine Learning: *Categories (by Functionality)*

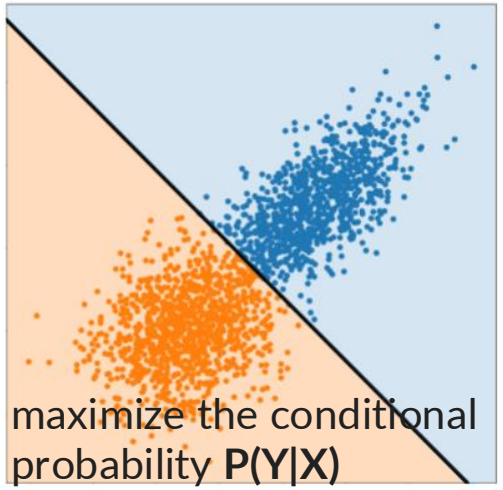
Generative

- **Definition**: aim to learn the joint probability distribution $p(x, y)$, where x represents the input features and y represents the target label (or some hidden variable). By modeling how data is generated, these models can generate new samples that resemble the original data distribution.



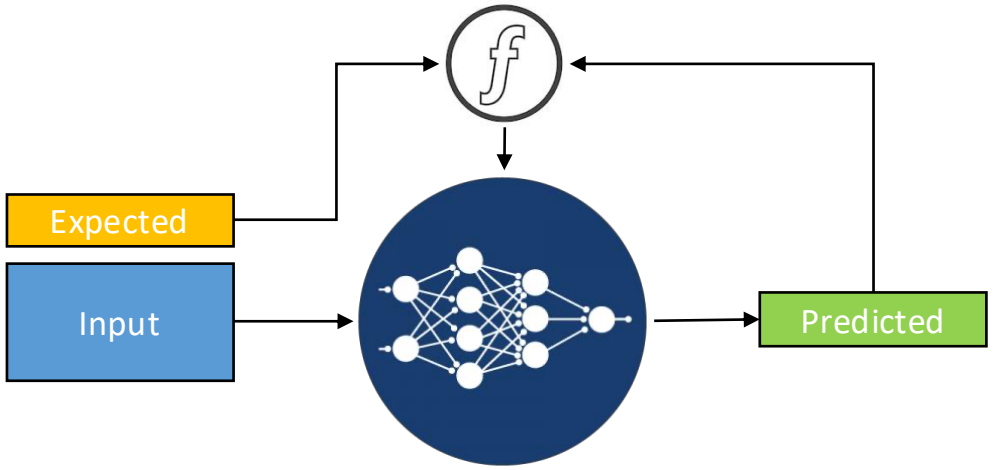
Discriminative

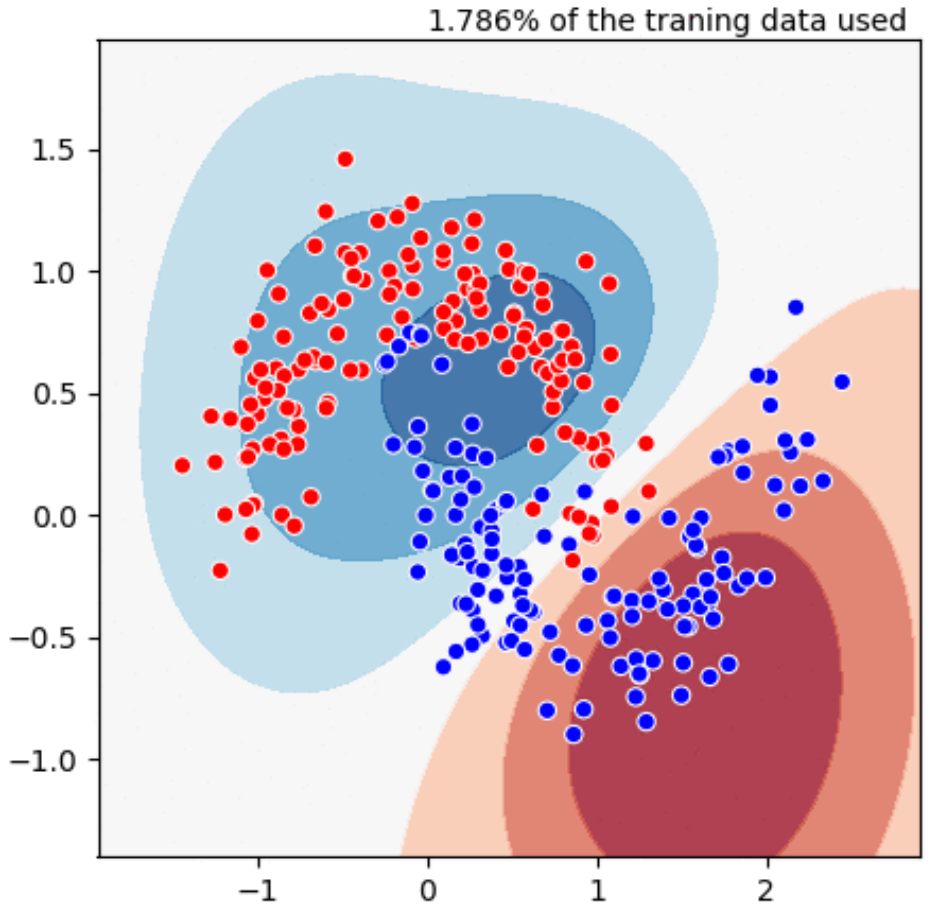
- **Definition**: focus on learning the conditional probability distribution $p(y \mid x)$ (or directly learning a decision boundary) that separates one class from another (in classification) or predicts a continuous value (in regression).



Machine Learning: *Categories (by Learning Paradigm)*

- Supervised Learning
- Unsupervised Learning
- Semi-supervised Learning
- Reinforcement Learning
- Self-supervised learning

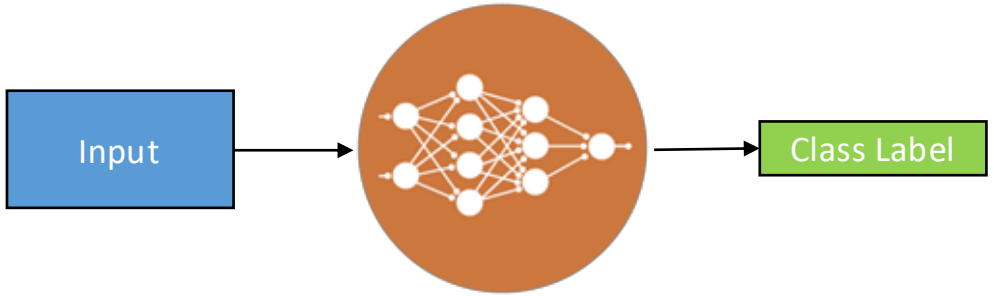


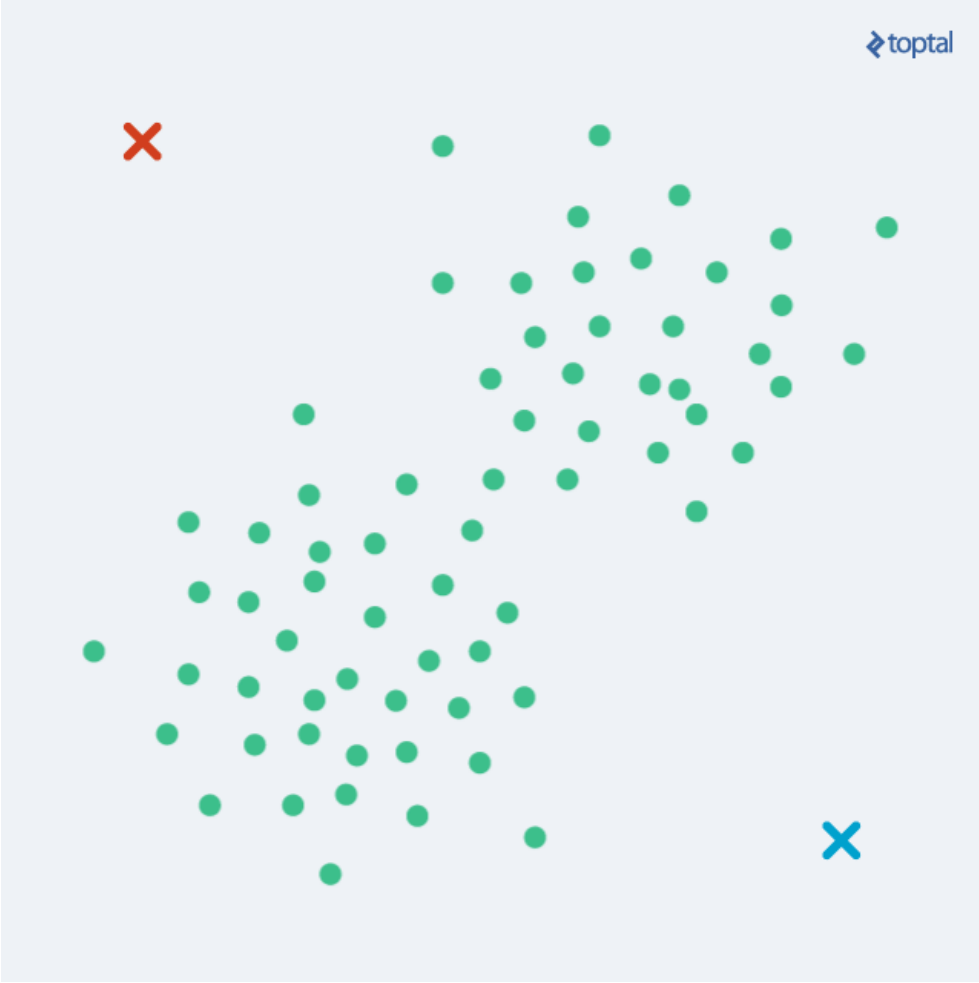


https://towardsdatascience.com/the-simplest-way-of-making-gifs-and-math-videos-with-python-aec41da74c6e

Machine Learning: *Categories (by Learning Paradigm)*

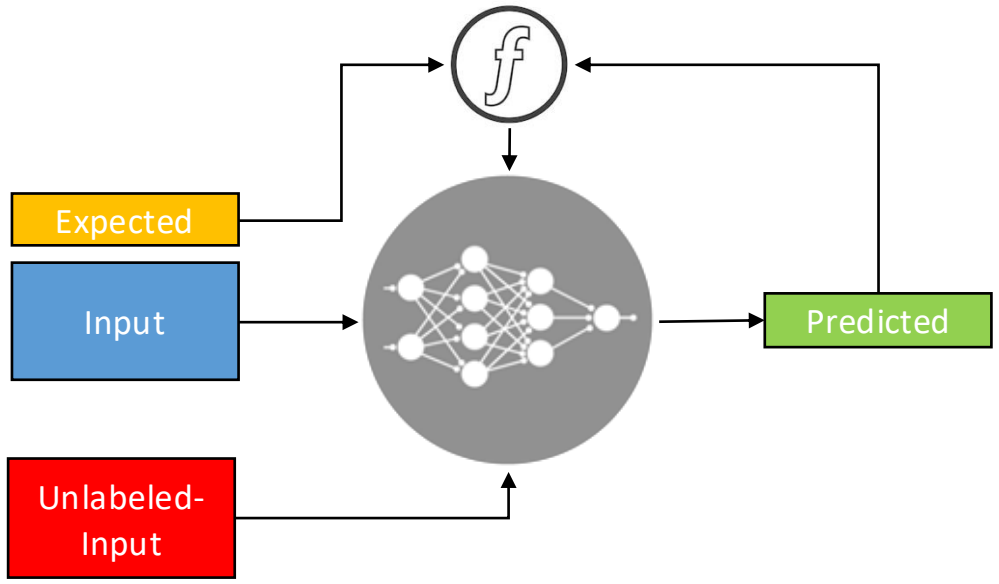
- Supervised Learning
- Unsupervised Learning
- Semi-supervised Learning
- Reinforcement Learning
- Self-supervised learning

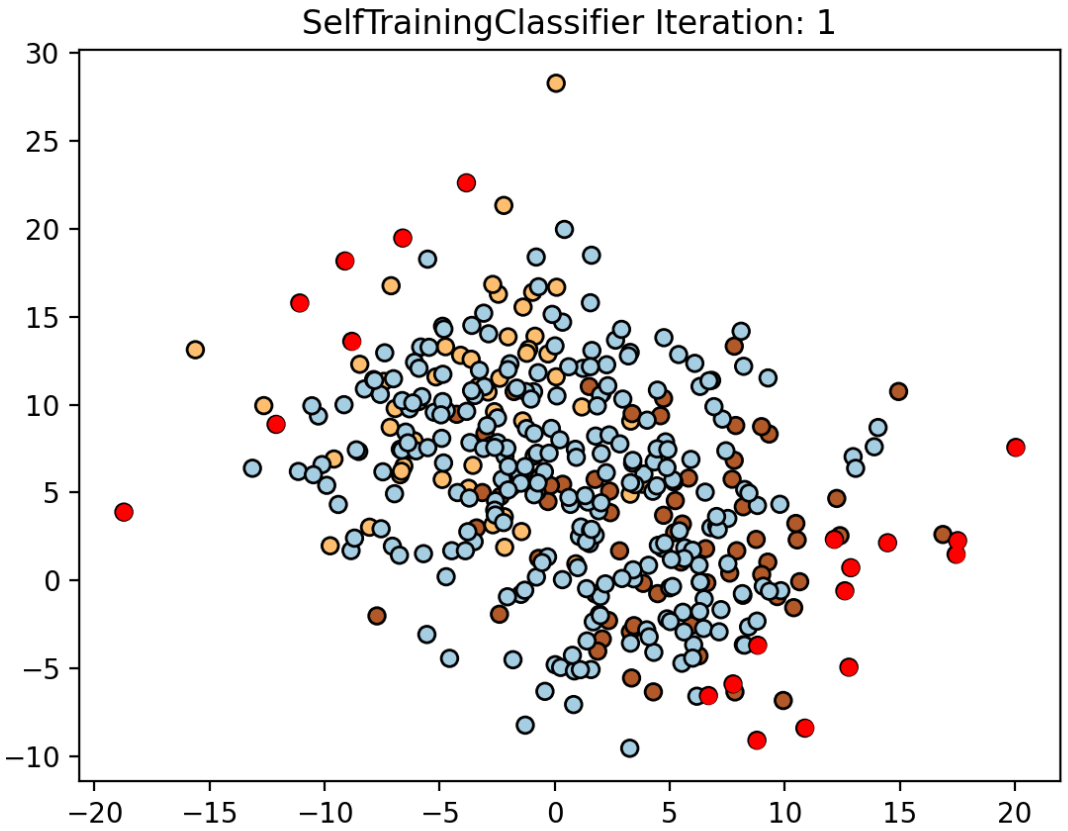




Machine Learning: *Categories (by Learning Paradigm)*

- Supervised Learning
- Unsupervised Learning
- Semi-supervised Learning
- Reinforcement Learning
- Self-supervised Learning

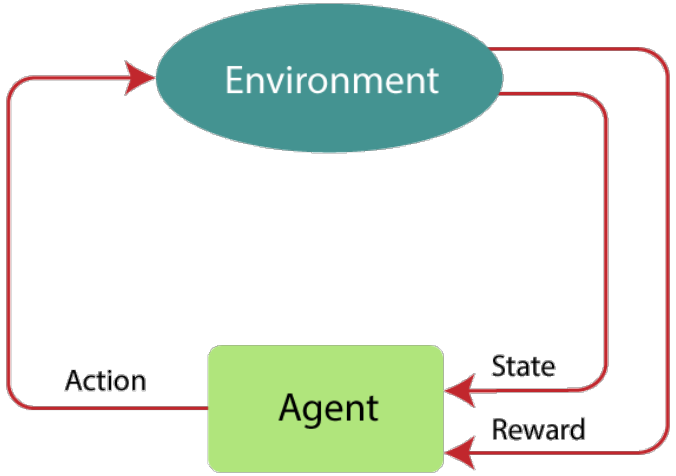


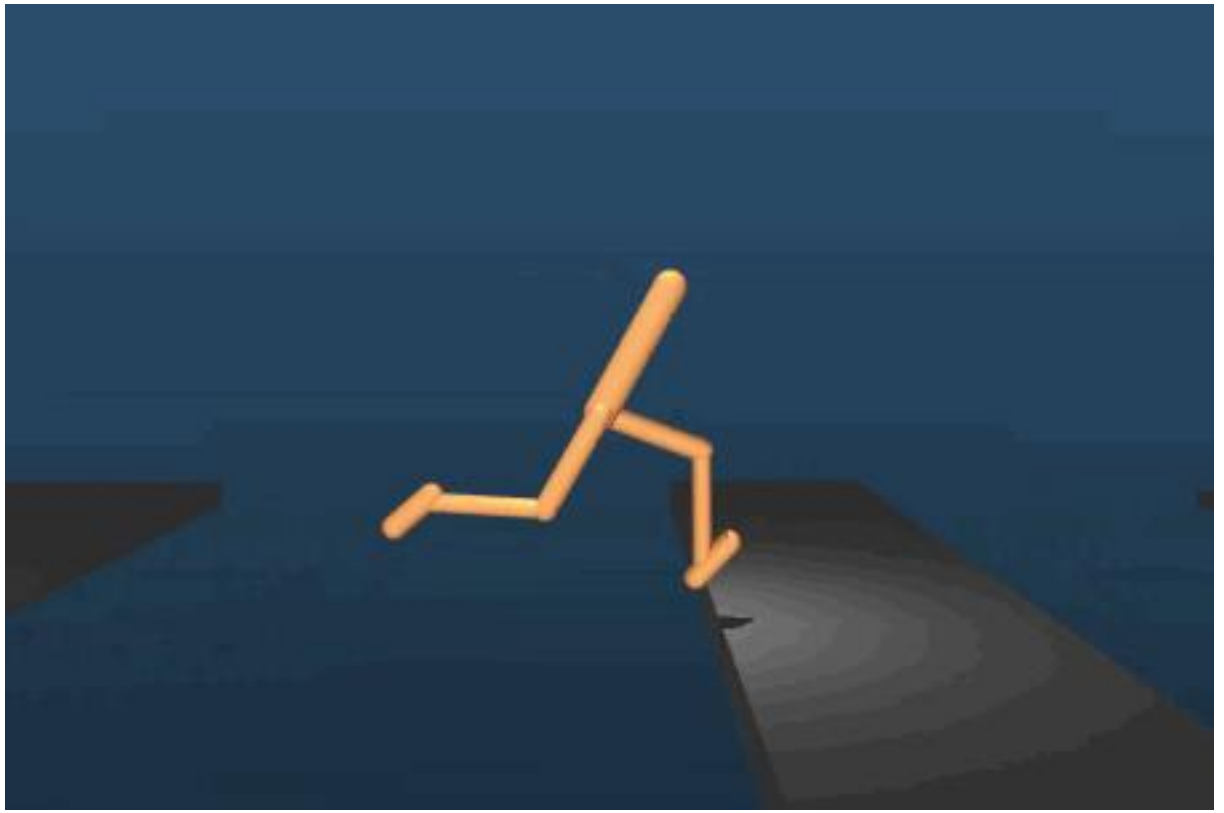


https://medium.com/@moonchangin/semi-supervised-learning-with-scikit-learns-selftrainingclassifier-a-visual-guide-e239857fc5c4

Machine Learning: *Categories (by Learning Paradigm)*

- Supervised Learning
- Unsupervised Learning
- Semi-supervised Learning
- Reinforcement Learning
- Self-supervised Learning

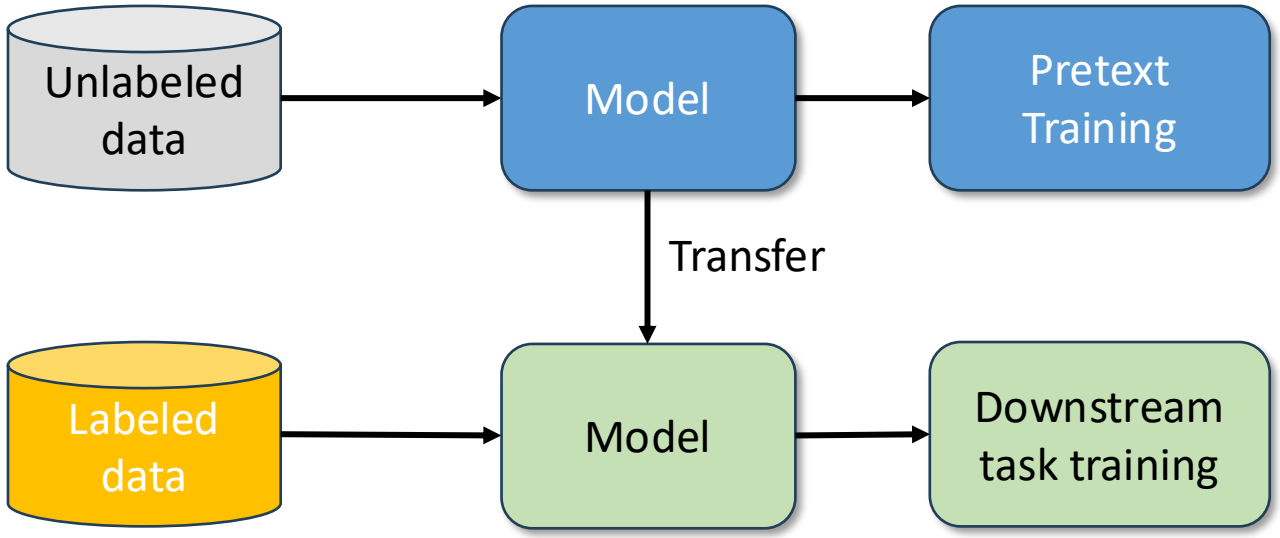


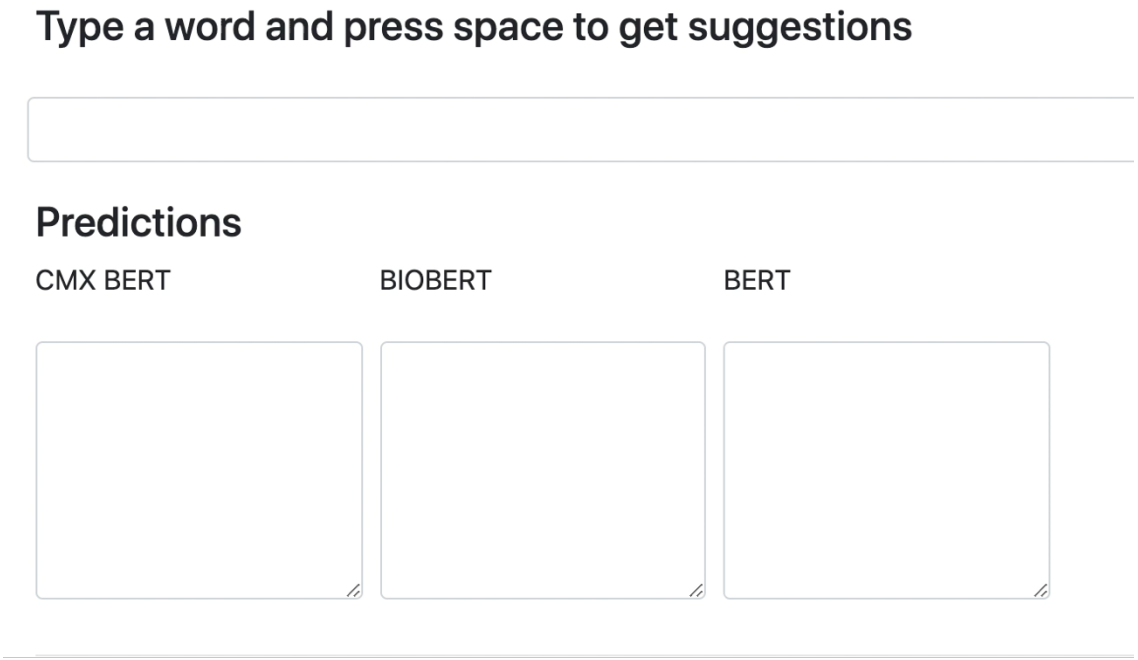


DeepMind's AI is teaching itself parkour

Machine Learning: *Categories (by Learning Paradigm)*

- Supervised Learning
- Unsupervised Learning
- Semi-supervised Learning
- Reinforcement Learning
- Self-supervised Learning





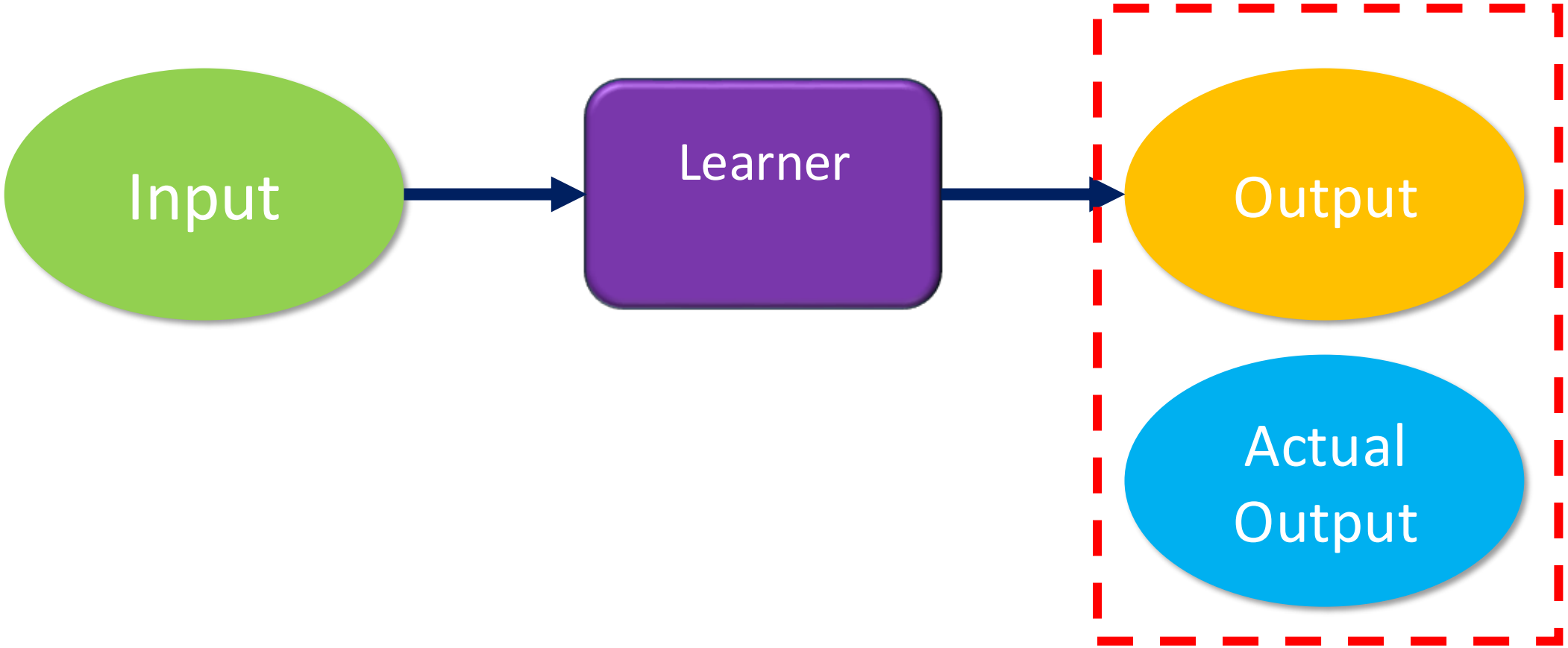
Machine Learning: *Categories (by Output)*

- **Regression**: The output variable is a real or continuous value
- **Classification**: The output variable is a category (discrete value)
- **Clustering**: Divide the dataset into groups based on similarity
- **Ranking**: Order data points based on a specific criteria
- **Generation**: Generate new samples

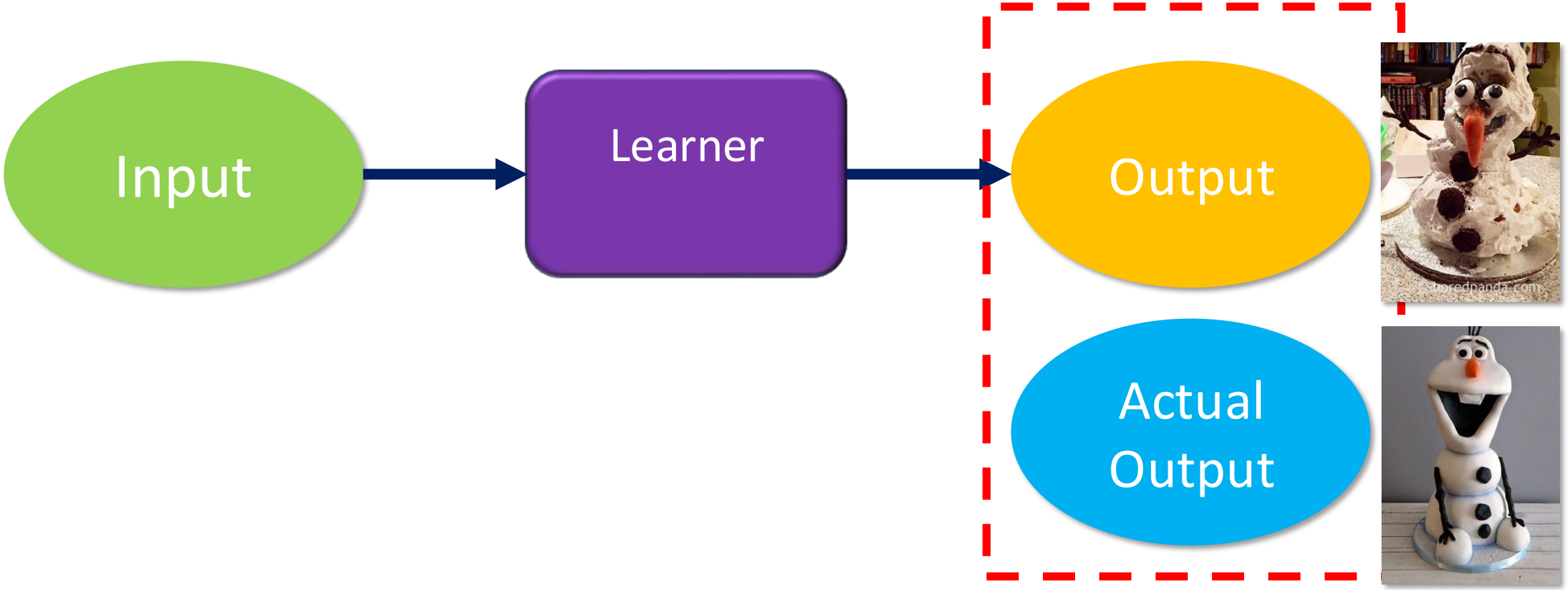
Machine Learning: *Categories (by Learning Paradigm)*

- Supervised Learning
- Unsupervised Learning
- Semi-supervised Learning
- Reinforcement Learning
- Self-supervised learning

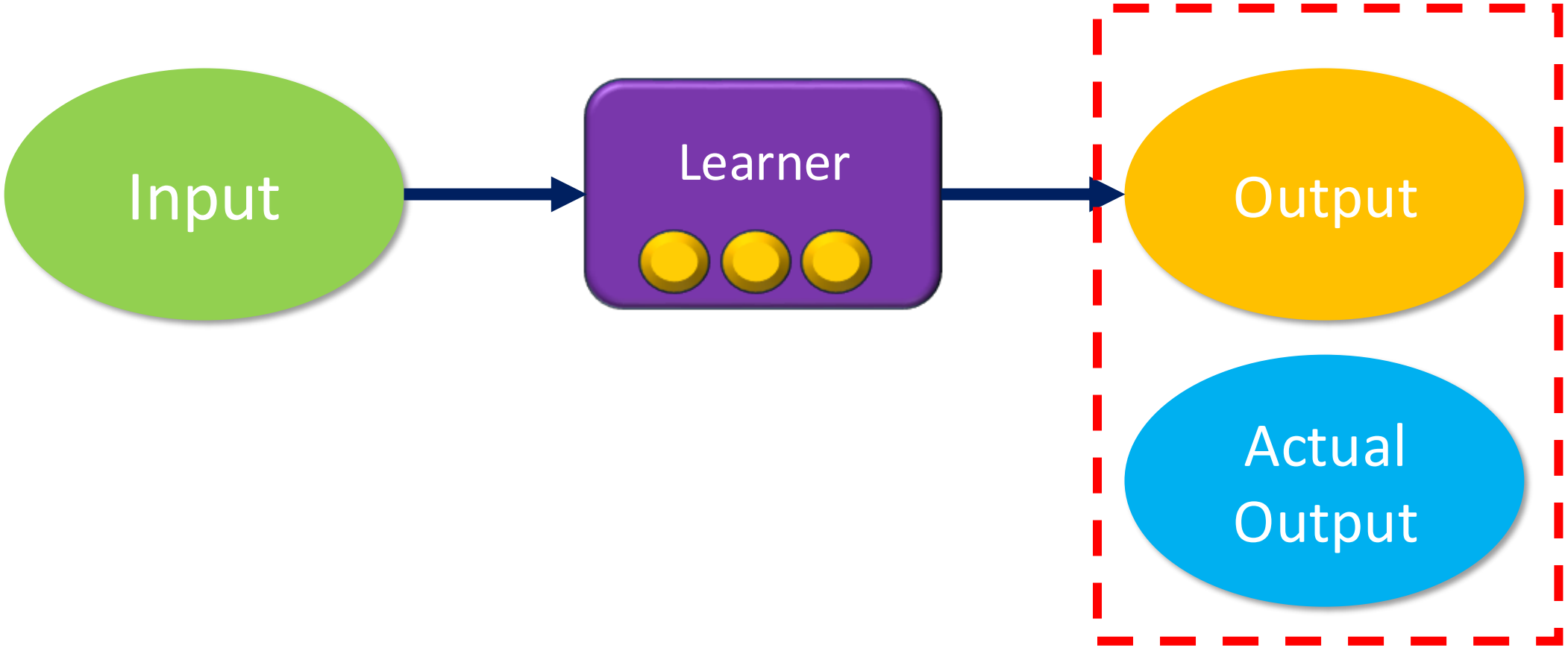
Machine Learning: *Model*



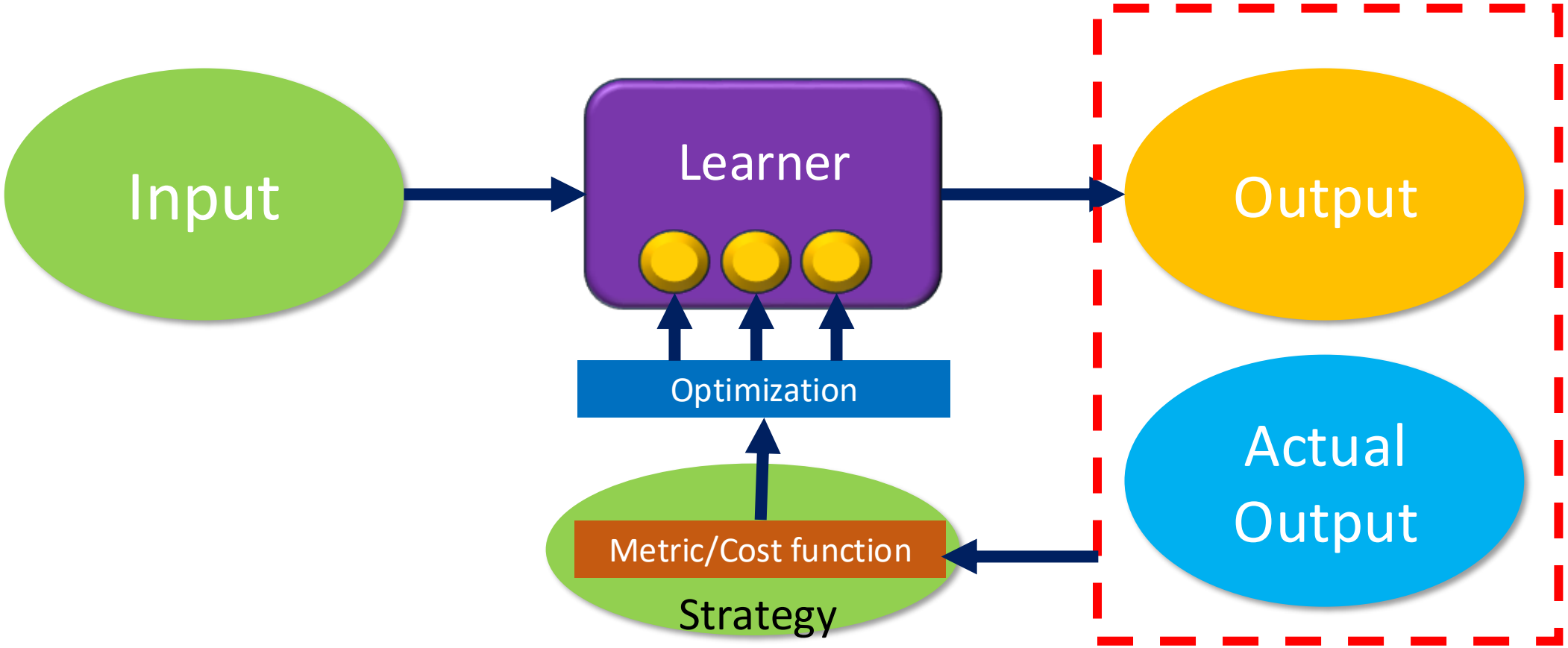
Machine Learning: *Model*



Machine Learning: *Model*



Machine Learning: *Model*



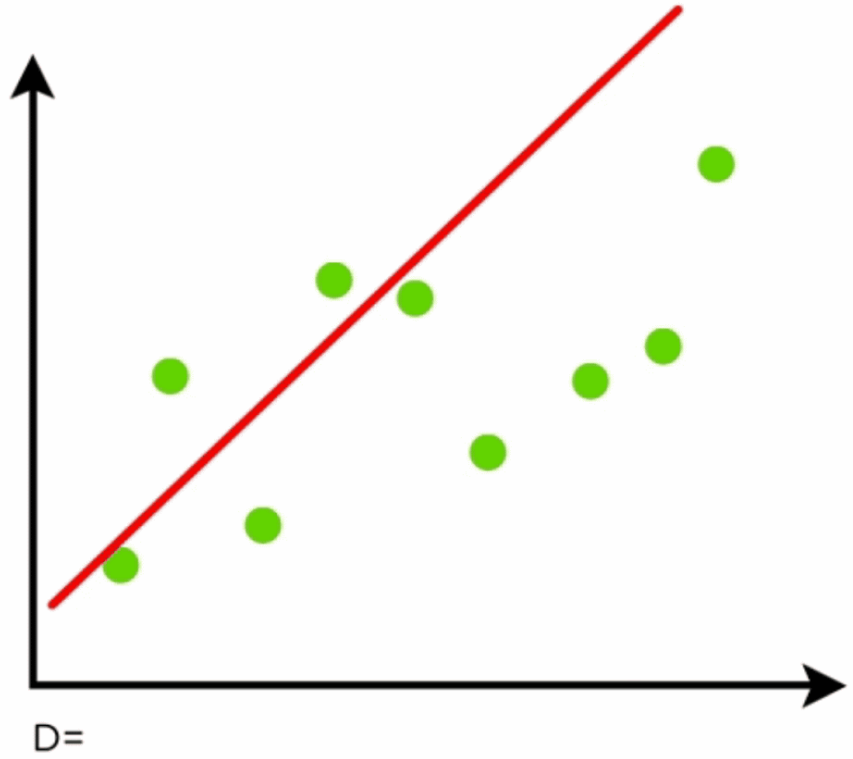
Machine Learning: *Supervised Learning*

Regression

Objective: Estimate the value of the target variable for a new set of input features.

Approach: learn the relationship between independent variables (features) and a dependent variable (target variable) based on a set of training data.

Output: Continuous values
(e.g., temperature, price, weight, etc.)



https://medium.com/@novus_afk/linear-regression-an-overview-13d37a6bc4dd

Machine Learning: *Supervised Learning*

Linear Regression

Linear: Fitting a straight line to the data
Regression: predicting a number

Nutrients. 2018 Dec; 10(12): 1976. PMCID: PMC6316192
Published online 2018 Dec 13. doi: 10.3390/nu10121976 PMID: 30551656

Body Mass Index is Strongly Associated with Hypertension: Results from the Longevity Check-Up 7+ Study

Francesco Landi,[1,*] Riccardo Calvani,[1] Anna Picca,[1] Matteo Tosato,[1] Anna Maria Martone,[1] Elena Ortolani,[1] Alex Sisto,[1] Emanuela D'Angelo,[1] Elisabetta Serafini,[1] Giovambattista Desideri,[2] Maria Tecla Fuga,[2] and Emanuele Marzetti[1]

Author information ▸ Article notes ▸ Copyright and License information PMC Disclaimer

Journal Article

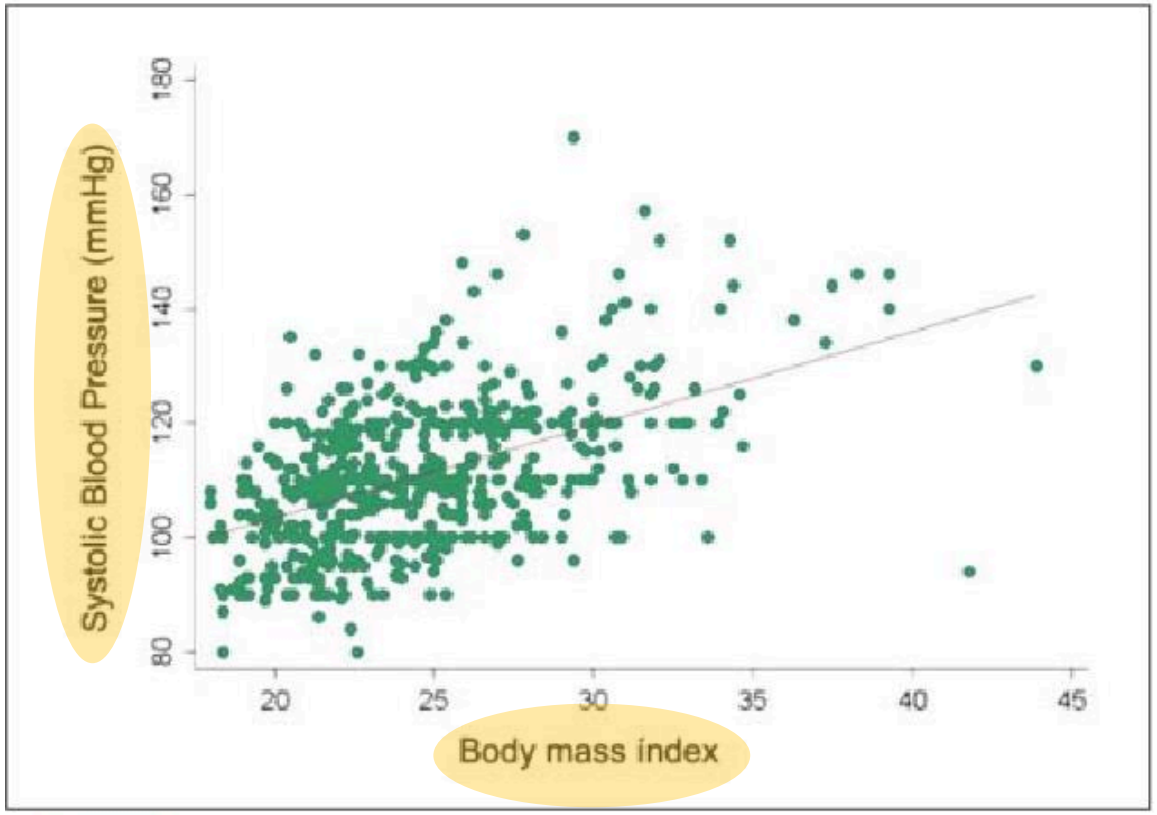


Figure. 1 - Simple linear regression of systolic blood pressure (SBP) in relation to body mass index (BMI) in a sample of 536 adolescents (r = 0.436; p=0.000; Beta coef. = 1.198).

Machine Learning: *Supervised Learning*

Linear Regression

Linear: Fitting a straight line to the data
Regression: predicting a number

Nutrients. 2018 Dec; 10(12): 1976. PMCID: PMC6316192
Published online 2018 Dec 13. doi: 10.3390/nu10121976 PMID: 30551656

Body Mass Index is Strongly Associated with Hypertension: Results from the Longevity Check-Up 7+ Study

Francesco Landi,[1,*] Riccardo Calvani,[1] Anna Picca,[1] Matteo Tosato,[1] Anna Maria Martone,[1] Elena Ortolani,[1] Alex Sisto,[1] Emanuela D'Angelo,[1] Elisabetta Serafini,[1] Giovambattista Desideri,[2] Maria Tecla Fuga,[2] and Emanuele Marzetti[1]

Author information ▸ Article notes ▸ Copyright and License information PMC Disclaimer

Journal Article

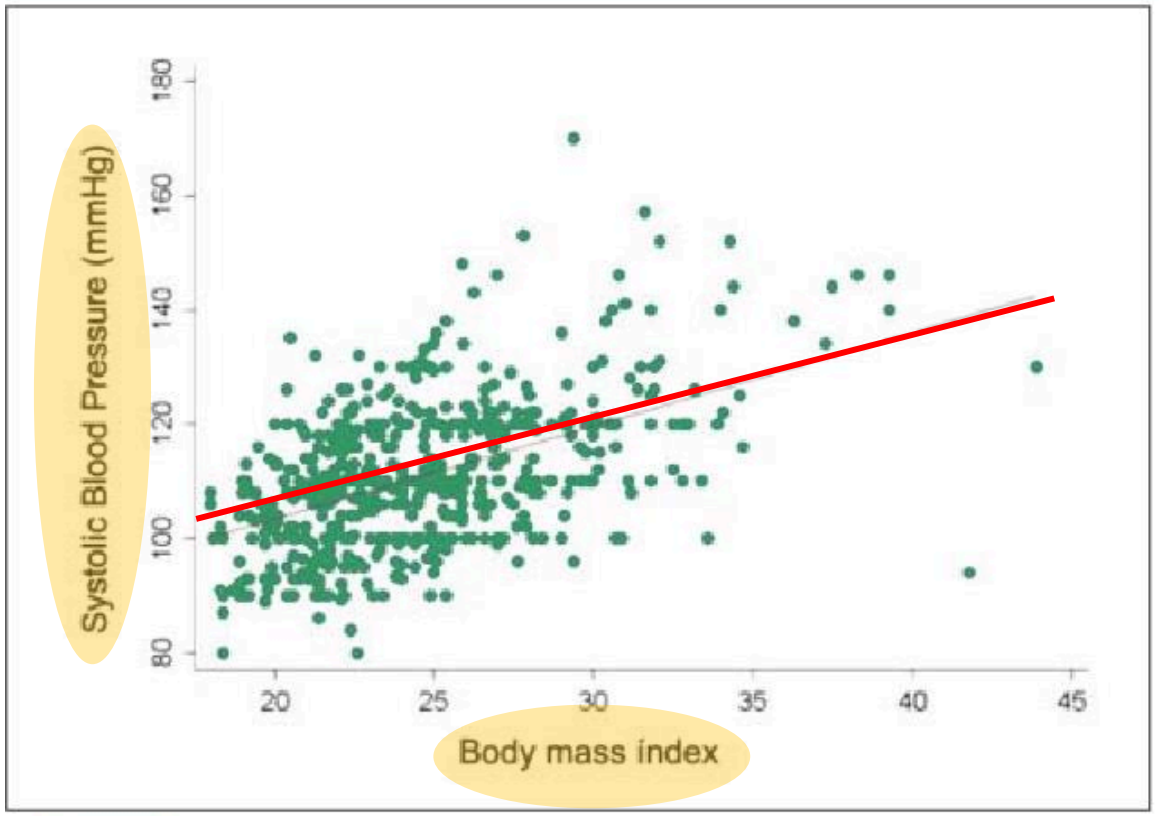


Figure. 1 - Simple linear regression of systolic blood pressure (SBP) in relation to body mass index (BMI) in a sample of 536 adolescents (r = 0.436; p=0.000; Beta coef. = 1.198).

Machine Learning: *Supervised Learning*

Linear Regression

Linear: Fitting a straight line to the data
Regression: predicting a number

Nutrients. 2018 Dec; 10(12): 1976. PMCID: PMC6316192
Published online 2018 Dec 13. doi: 10.3390/nu10121976 PMID: 30551656

Body Mass Index is Strongly Associated with Hypertension: Results from the Longevity Check-Up 7+ Study

Francesco Landi,[1,*] Riccardo Calvani,[1] Anna Picca,[1] Matteo Tosato,[1] Anna Maria Martone,[1] Elena Ortolani,[1] Alex Sisto,[1] Emanuela D'Angelo,[1] Elisabetta Serafini,[1] Giovambattista Desideri,[2] Maria Tecla Fuga,[2] and Emanuele Marzetti[1]

Author information ▸ Article notes ▸ Copyright and License information PMC Disclaimer

Journal Article

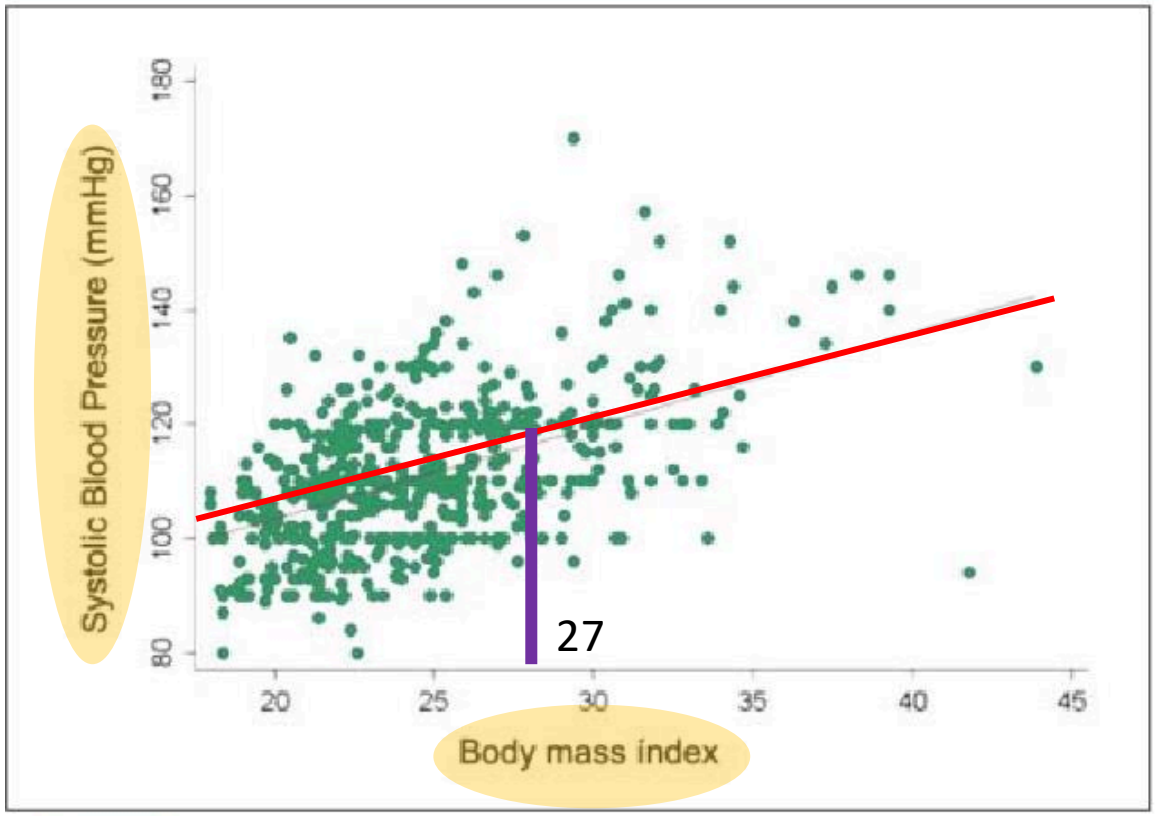


Figure. 1 - Simple linear regression of systolic blood pressure (SBP) in relation to body mass index (BMI) in a sample of 536 adolescents (r = 0.436; p=0.000; Beta coef. = 1.198).

Machine Learning: *Supervised Learning*

Linear Regression

Linear: Fitting a straight line to the data
Regression: predicting a number

Nutrients. 2018 Dec; 10(12): 1976. PMCID: PMC6316192
Published online 2018 Dec 13. doi: 10.3390/nu10121976 PMID: 30551656

Body Mass Index is Strongly Associated with Hypertension: Results from the Longevity Check-Up 7+ Study

Francesco Landi,[1,*] Riccardo Calvani,[1] Anna Picca,[1] Matteo Tosato,[1] Anna Maria Martone,[1] Elena Ortolani,[1] Alex Sisto,[1] Emanuela D'Angelo,[1] Elisabetta Serafini,[1] Giovambattista Desideri,[2] Maria Tecla Fuga,[2] and Emanuele Marzetti[1]

Author information ▸ Article notes ▸ Copyright and License information PMC Disclaimer

Journal Article

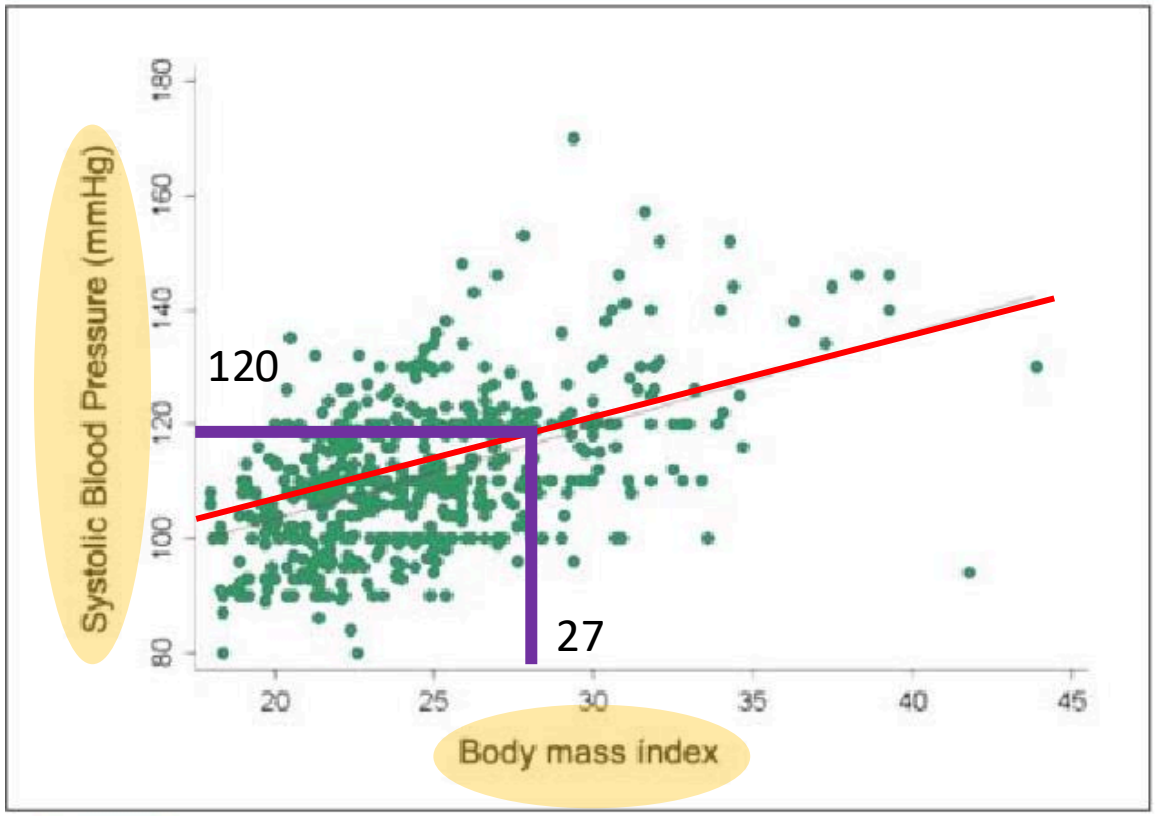


Figure. 1 - Simple linear regression of systolic blood pressure (SBP) in relation to body mass index (BMI) in a sample of 536 adolescents (r = 0.436; p=0.000; Beta coef. = 1.198).

Machine Learning: *Supervised Learning*

Linear Regression

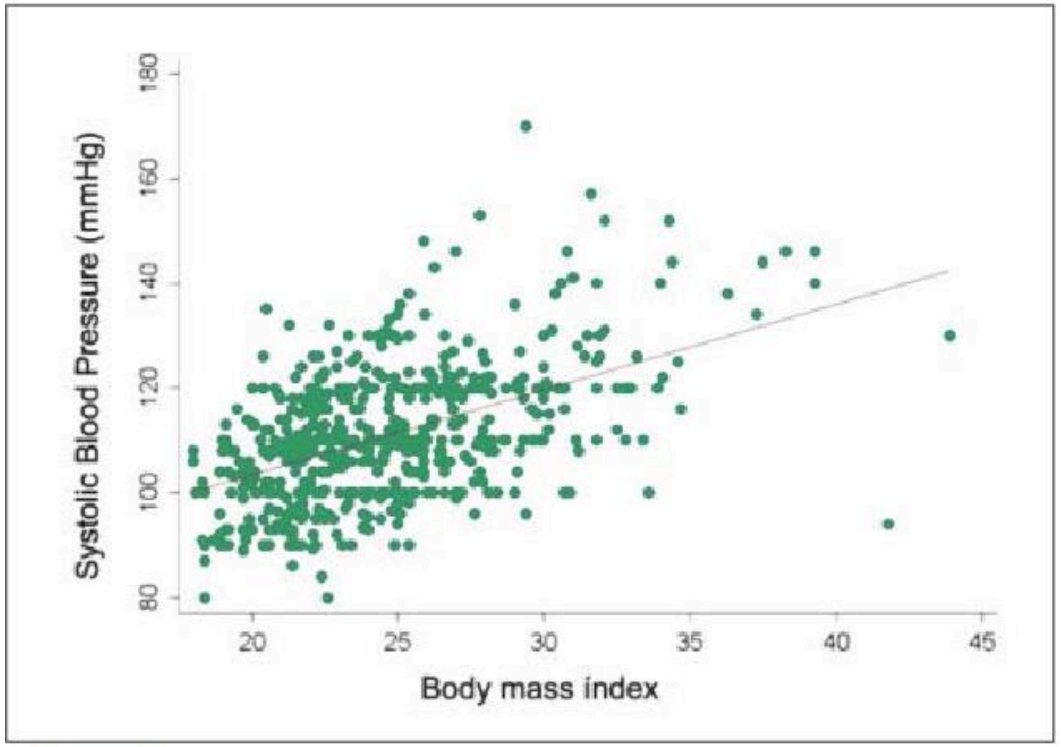


Figure. 1 - Simple linear regression of systolic blood pressure (SBP) in relation to body mass index (BMI) in a sample of 536 adolescents (r = 0.436; p=0.000; Beta coef. = 1.198).

BMI (kg/m2)	Blood Pressure (mmHg)
19	120
22	126
26	125
29	131
…	…

Machine Learning: *Supervised Learning*

Linear Regression

x: Input/feature

y: output/target

BMI (kg/m2)	Blood Pressure (mmHg)	
19	120	1
22	126	2
26	125	⋮
29	131	
…	…	m

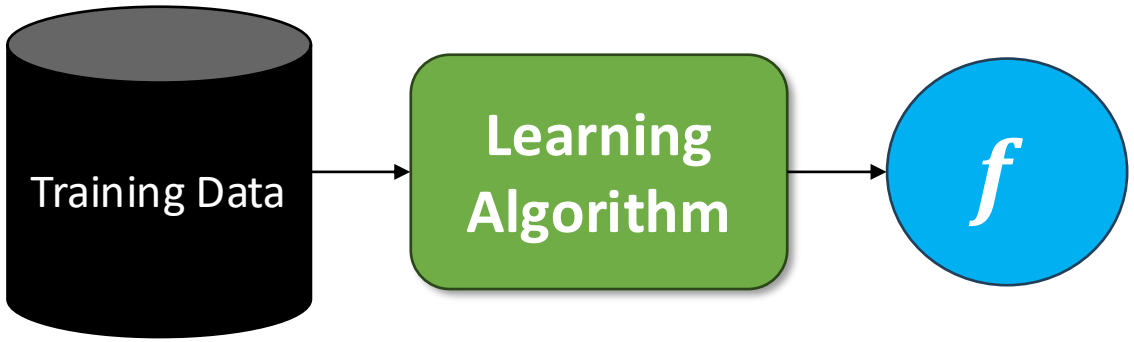
Terminology:

$(x^{(i)}, y^{(i)})$ $(i)^{th}$ training example

Machine Learning: *Supervised Learning*

Linear Regression

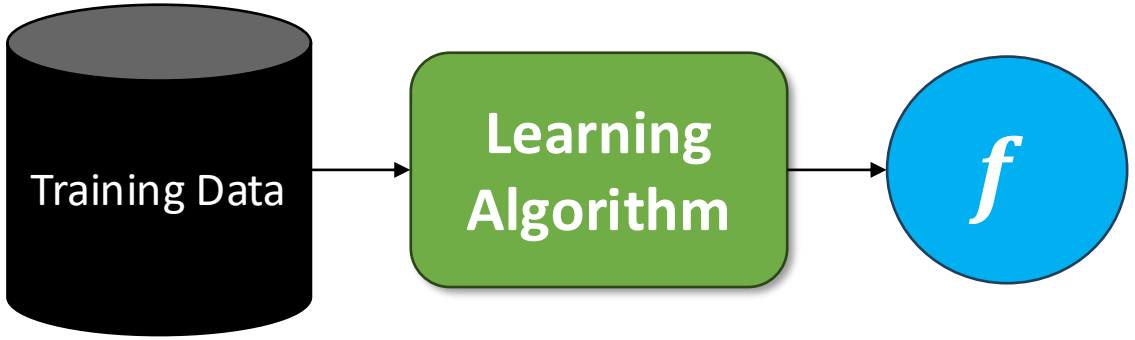
x: Input/feature
y: output/target

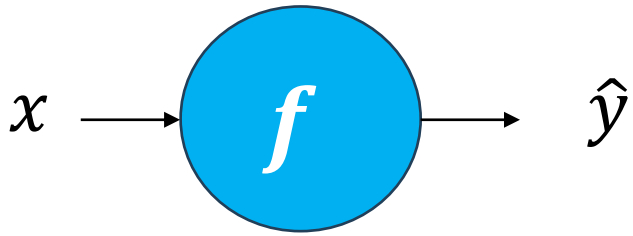


Machine Learning: *Supervised Learning*

Linear Regression

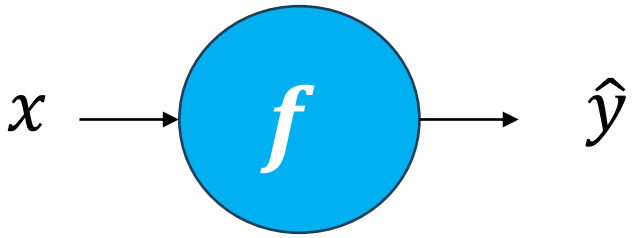
x: Input/feature
y: output/target



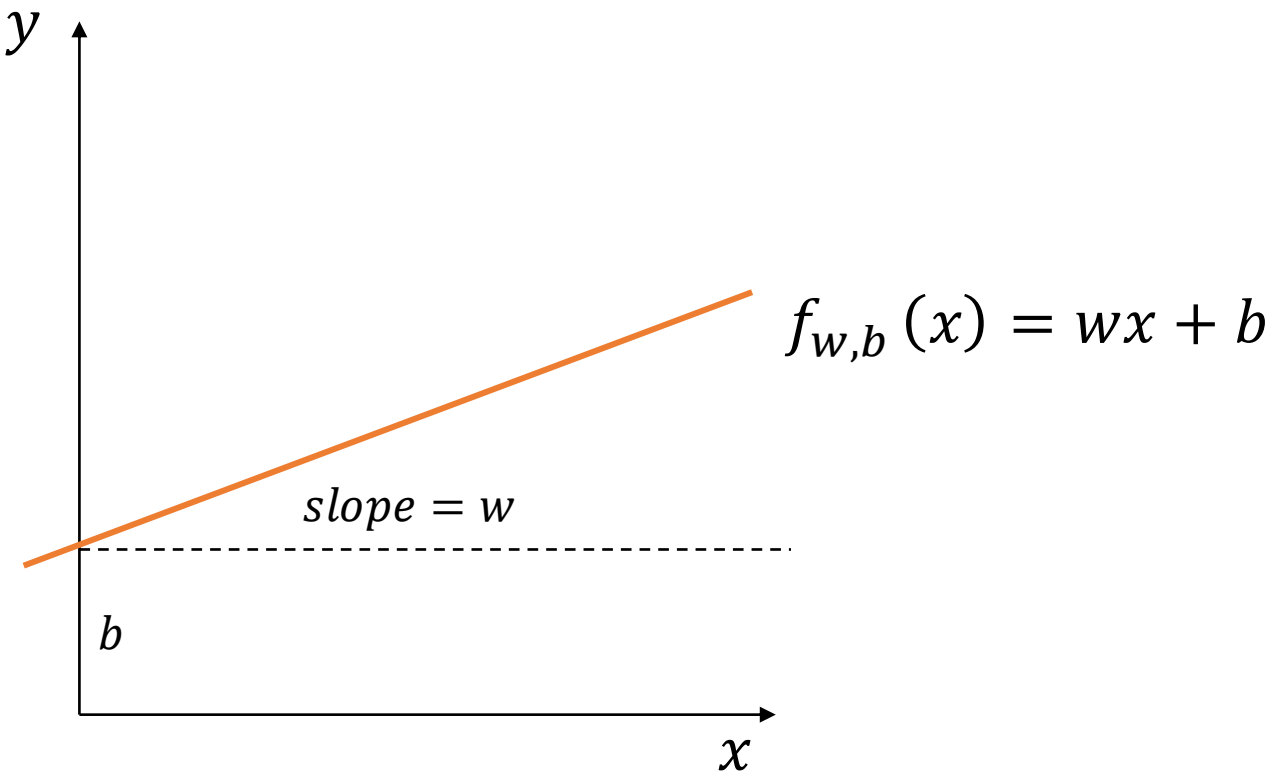


Machine Learning: *Supervised Learning*

Linear Regression



$$f_{w,b}(x) = wx + b$$



Machine Learning: *Supervised Learning*

Linear Regression

Cost Function

Definition: a mathematical function that quantifies the error or difference between the predicted values by a model and the actual values of the data being modeled

Purpose: to measure how well a given model performs

Machine Learning: *Supervised Learning*

Linear Regression

Model Parameters

Definition: configuration variables that are internal to the model and whose values can be estimated from the data.

Example: w, b in Linear Regression

$$f_{w,b}(x) = wx + b$$

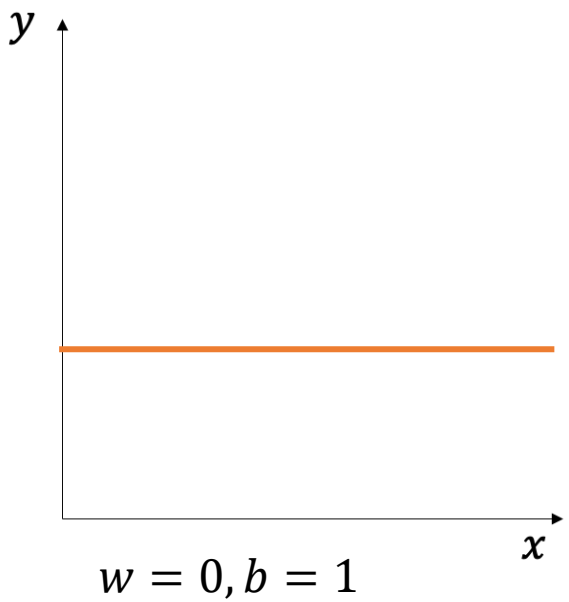
Machine Learning: *Supervised Learning*

Linear Regression

Model Parameters

$$f_{w,b}(x) = wx + b$$

$$f_{w,b}(x) = wx + b$$
$$= 0.x + 1 = 1$$
$$\hat{y} = 1$$



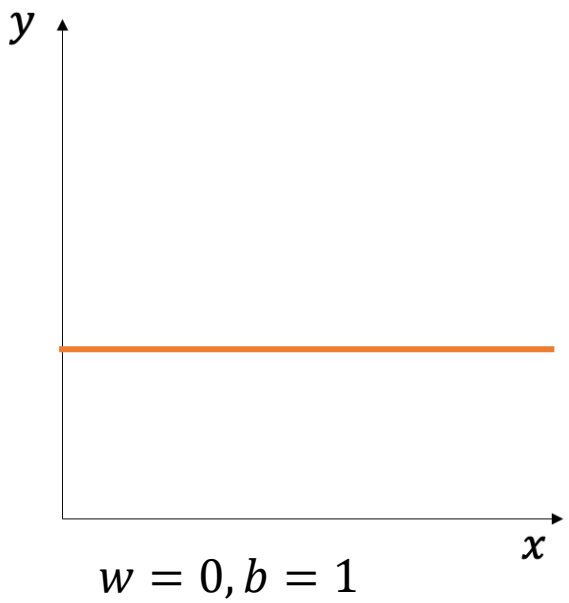
Machine Learning: *Supervised Learning*

Linear Regression

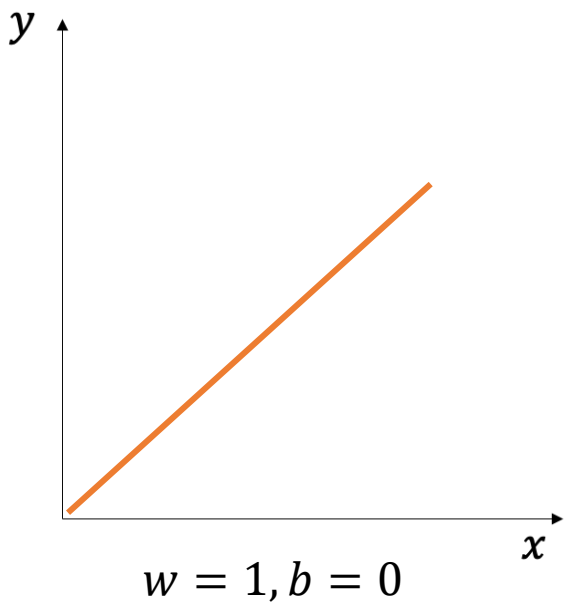
Model Parameters

$$f_{w,b}(x) = wx + b$$

$$f_{w,b}(x) = wx + b$$
$$= 1.x + 0 = x$$
$$\hat{y} = x$$



$w = 0, b = 1$



$w = 1, b = 0$

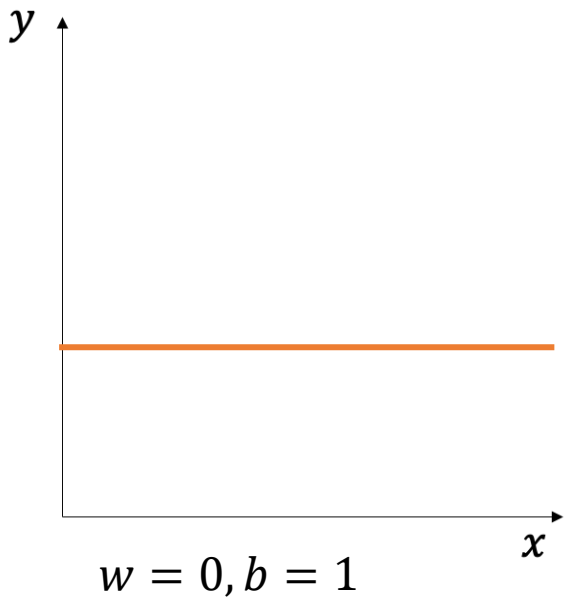
Machine Learning: *Supervised Learning*

Linear Regression

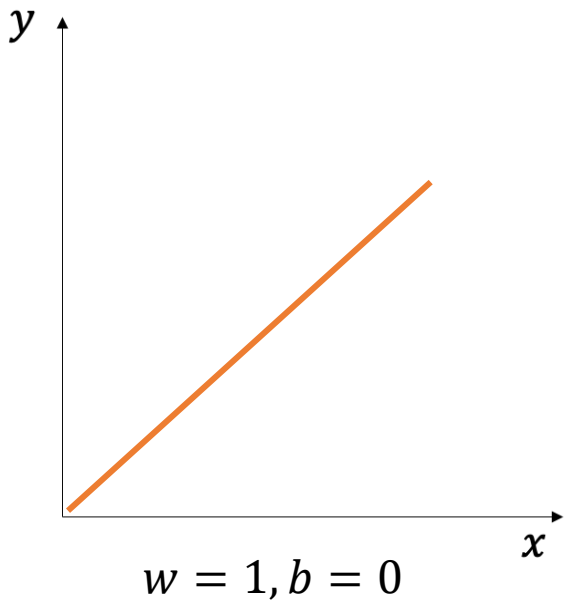
Model Parameters

$$f_{w,b}(x) = wx + b$$

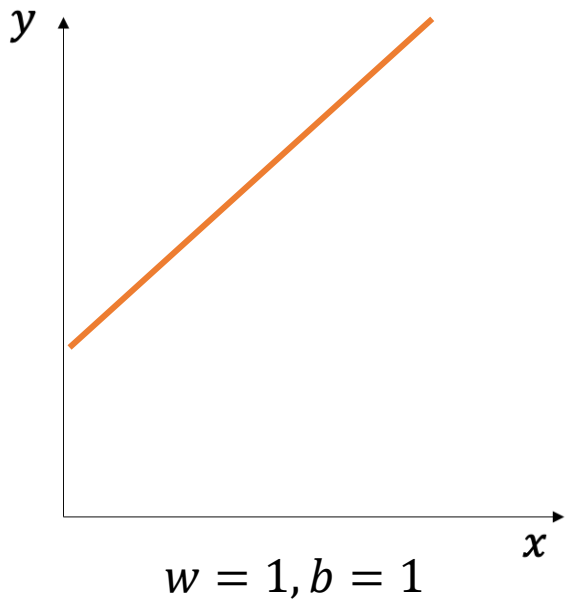
$$f_{w,b}(x) = wx + b$$
$$= 1.x + 1 = x + 1$$
$$\hat{y} = x + 1$$



$w = 0, b = 1$



$w = 1, b = 0$



$w = 1, b = 1$

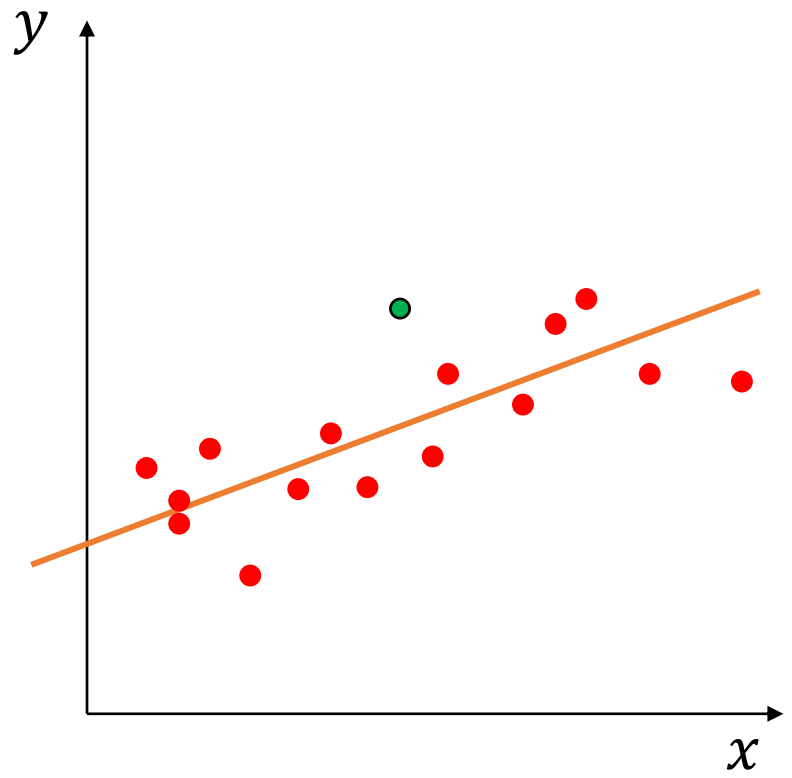
Machine Learning: *Supervised Learning*

Linear Regression

$$f_{w,b}(x) = wx + b$$

$$\hat{y}^{(i)} = f_{w,b}\left(x^{(i)}\right)$$

$$\hat{y}^{(i)} = wx^{(i)} + b$$



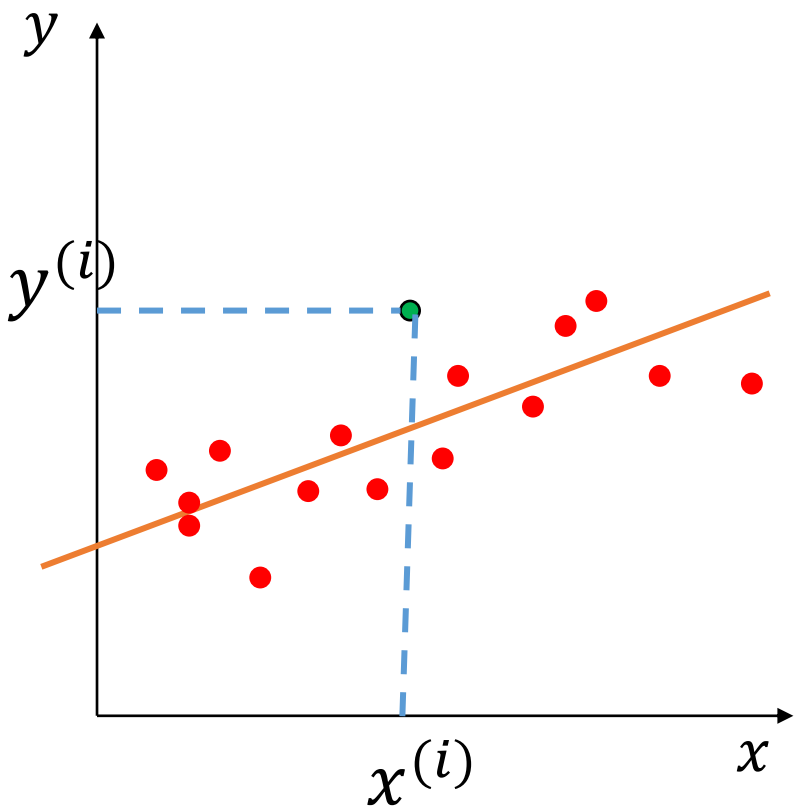
Machine Learning: *Supervised Learning*

Linear Regression

$$f_{w,b}(x) = wx + b$$

$$\hat{y}^{(i)} = f_{w,b}\left(x^{(i)}\right)$$

$$\hat{y}^{(i)} = wx^{(i)} + b$$



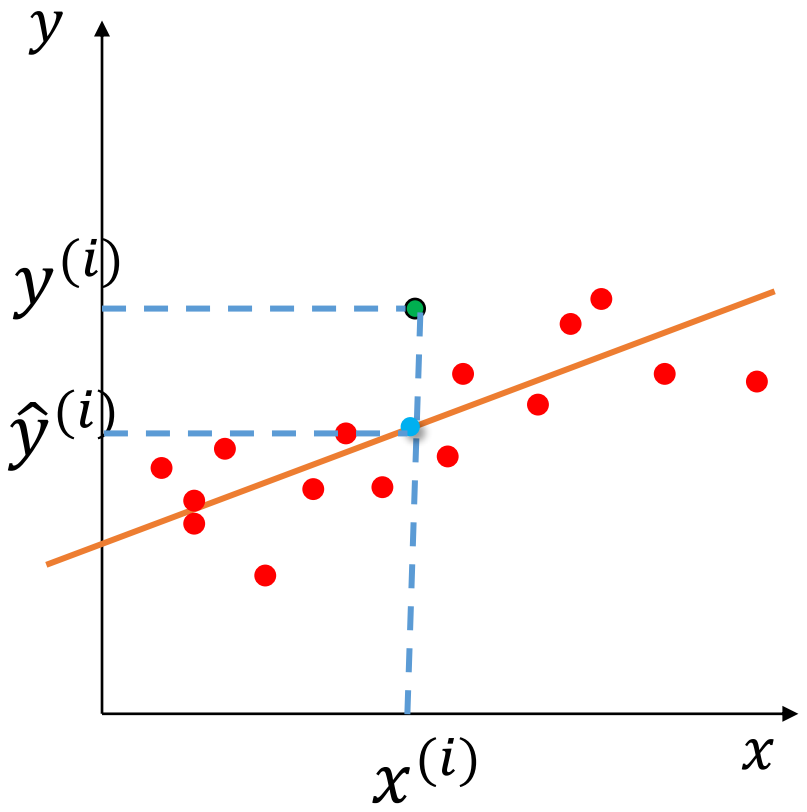
Machine Learning: *Supervised Learning*

Linear Regression

$$f_{w,b}(x) = wx + b$$

$$\hat{y}^{(i)} = f_{w,b}\left(x^{(i)}\right)$$

$$\hat{y}^{(i)} = wx^{(i)} + b$$

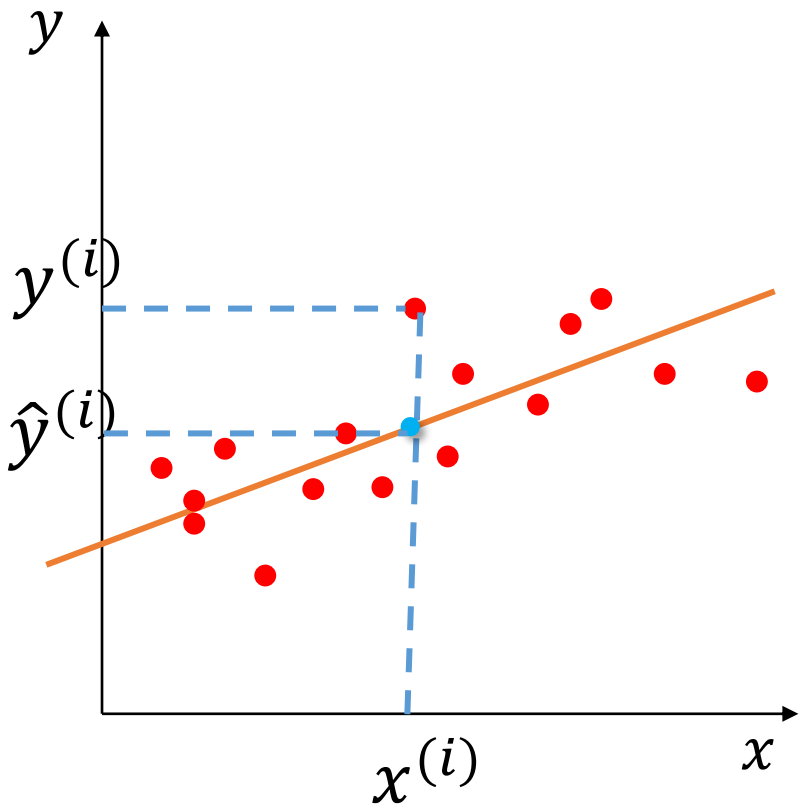


Machine Learning: *Supervised Learning*

Linear Regression

$$f_{w,b}(x) = wx + b$$

Find w, b so that $\hat{y}^{(i)}$ (predictions) are as close as possible to $y^{(i)}$ (actual values) for all samples



Machine Learning: *Supervised Learning*

Linear Regression

$$f_{w,b}(x) = wx + b$$

Find w, b so that $\hat{y}^{(i)}$ (predictions) are as close as possible to $y^{(i)}$ (actual values) for all samples

$$\hat{y}^{(i)} - y^{(i)}$$

Error

Machine Learning: *Supervised Learning*

Linear Regression

$$f_{w,b}(x) = wx + b$$

Find w, b so that $\hat{y}^{(i)}$ (predictions) are as close as possible to $y^{(i)}$ (actual values) for all samples

$$(\hat{y}^{(i)} - y^{(i)})^2$$

Error

Machine Learning: *Supervised Learning*

Fun Fact

Why square the error?

$$(\hat{y}^{(i)} - y^{(i)})^2$$

Error

Machine Learning: *Supervised Learning*

Fun Fact

Why square the error?

1. Emphasizing Larger Errors
2. Differentiability
3. Convexity
4. Analytical Simplicity
5. Statistical Justification

$$(\hat{y}^{(i)} - y^{(i)})^2$$

Error

Machine Learning: *Supervised Learning*

However…

<u>Disadvantage</u>:
While squaring the error in cost functions has many advantages, it also makes the model more sensitive to outliers.

$$(\hat{y}^{(i)} - y^{(i)})^2$$

Error

Machine Learning: *Supervised Learning*

Linear Regression

$$f_{w,b}(x) = wx + b$$

Find w, b so that $\hat{y}^{(i)}$ (predictions) are as close as possible to $y^{(i)}$ (actual values) for all samples

$$\frac{1}{2m}\sum_{i=1}^{m}\left(\hat{y}^{(i)} - y^{(i)}\right)^2$$

m: number of samples

Machine Learning: *Supervised Learning*

Linear Regression

$$f_{w,b}(x) = wx + b$$

Find w, b so that $\hat{y}^{(i)}$ (predictions) are as close as possible to $y^{(i)}$ (actual values) for all samples

Squared Error Cost Function

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{w,b}\left(x^{(i)}\right) - y^{(i)}\right)^2$$

m: number of samples

Machine Learning: *Supervised Learning*

Jacobian

Definition: represents the first-order partial derivatives of a vector-valued function of several variables, offering a systematic way to capture how changes in input variables lead to changes in output variables.

$$\mathbf{f} : \mathbb{R}^u \rightarrow \mathbb{R}^v$$

$$\mathbf{J} = \frac{d\mathbf{f}(\mathbf{x})}{d\mathbf{x}} = \left[\frac{\partial \mathbf{f}(\mathbf{x})}{\partial x_1} \cdots \frac{\partial \mathbf{f}(\mathbf{x})}{\partial x_u}\right] = \begin{bmatrix} \frac{\partial f_1(\mathbf{x})}{\partial x_1} & \cdots & \frac{\partial f_1(\mathbf{x})}{\partial x_u} \\ \vdots & & \vdots \\ \frac{\partial f_v(\mathbf{x})}{\partial x_1} & \cdots & \frac{\partial f_v(\mathbf{x})}{\partial x_u} \end{bmatrix}$$

Machine Learning: *Linear Regression*

Model: $f_{w,b}(x) = wx + b$

Parameters: w, b

Cost Function: $J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{w,b}\left(x^{(i)}\right) - y^{(i)}\right)^2$

Goal: $\min_{w,b} J(w,b)$

Machine Learning: *Supervised Learning*

Linear Regression

Cost Function: Intuition

Simplified version:

$$f_w(x) = wx$$

Model: $f_w(x) = wx$

Parameters: w

Cost Function: $J(w) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_w\left(x^{(i)}\right) - y^{(i)}\right)^2$

Goal: $\min_w J(w)$

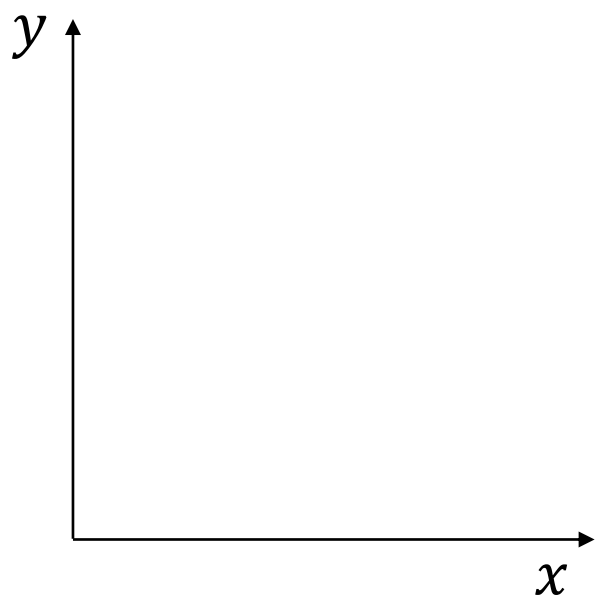
Machine Learning: *Supervised Learning*

Linear Regression

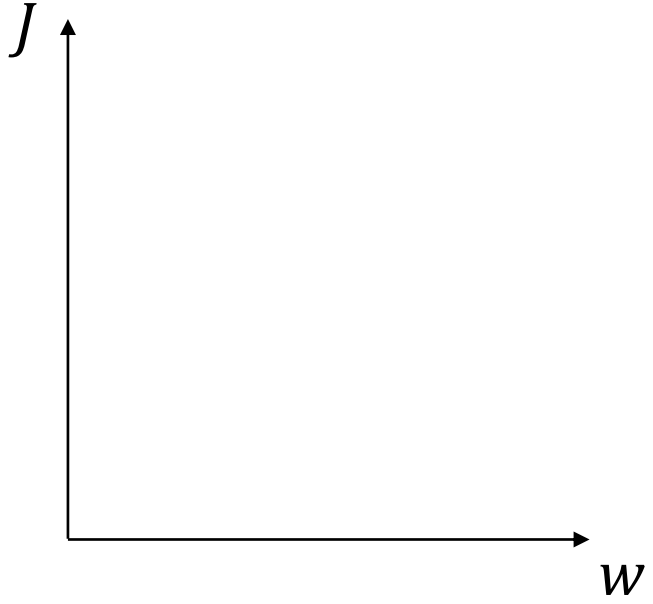
Cost Function: Intuition

$$f_w(x) = wx$$

$y = f_w(x)$



$J(w)$



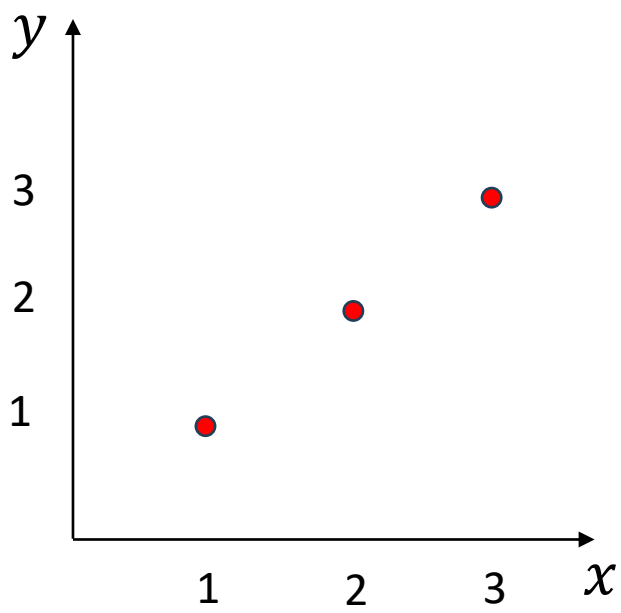
Machine Learning: *Supervised Learning*

Linear Regression

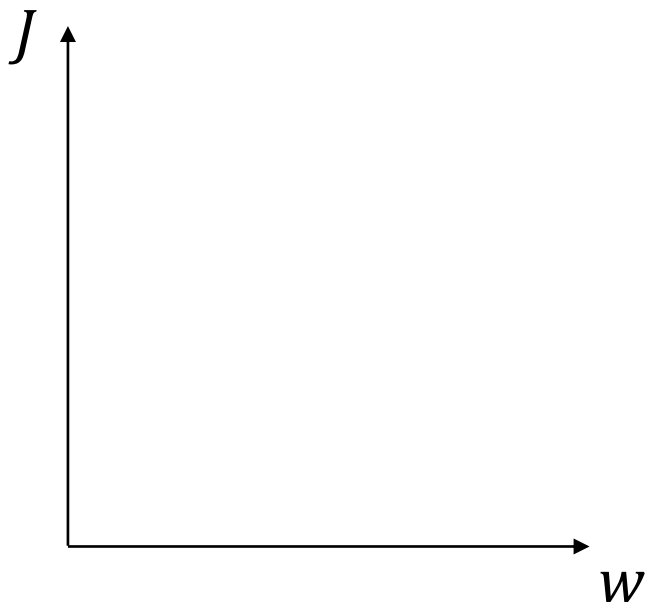
Cost Function: Intuition

$$f_w(x) = wx$$

$y = f_w(x)$



$J(w)$



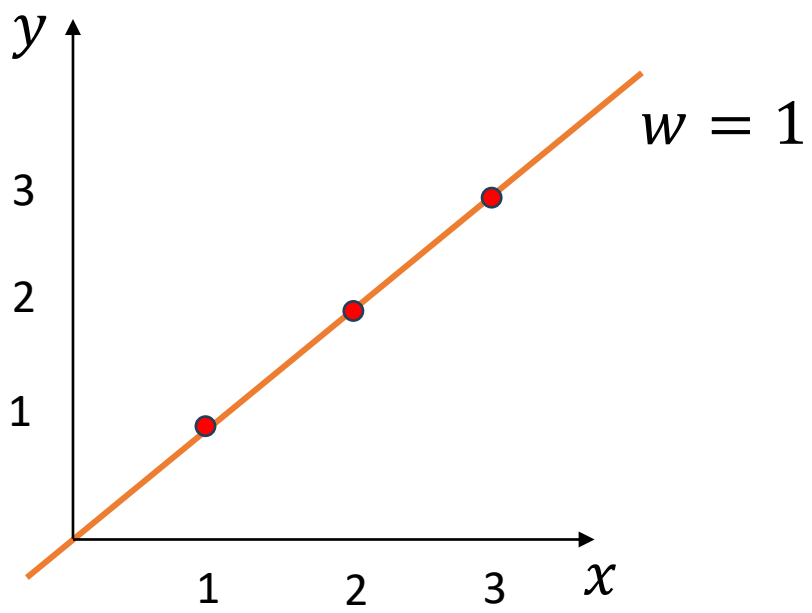
Machine Learning: *Supervised Learning*

Linear Regression

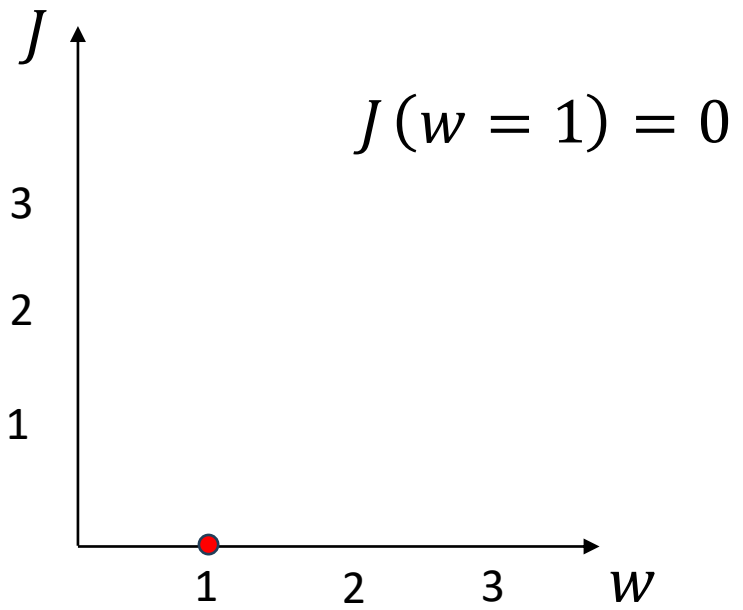
Cost Function: Intuition

$$f_w(x) = wx$$

$y = f_w(x)$



$J(w)$



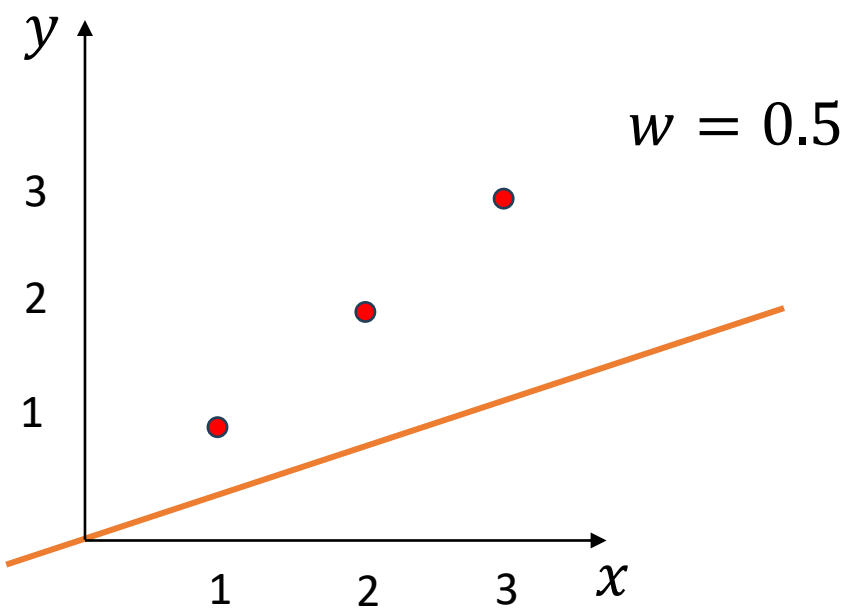
Machine Learning: *Supervised Learning*

Linear Regression

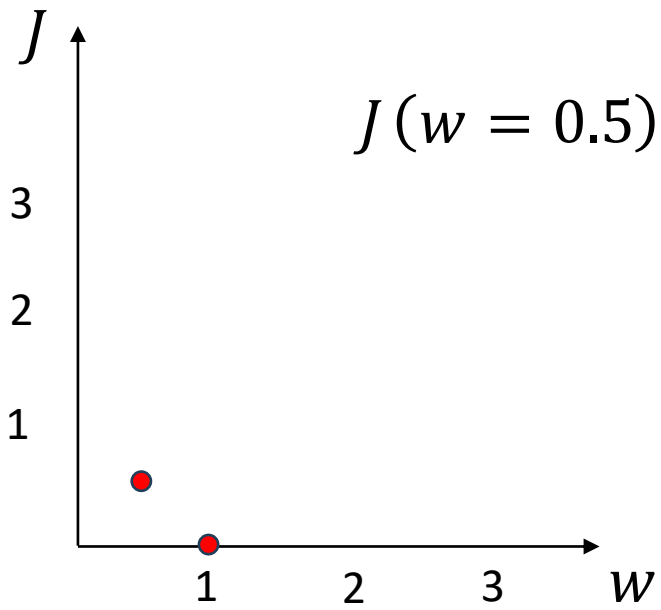
Cost Function: Intuition

$$f_w(x) = wx$$

$y = f_w(x)$



$J(w)$



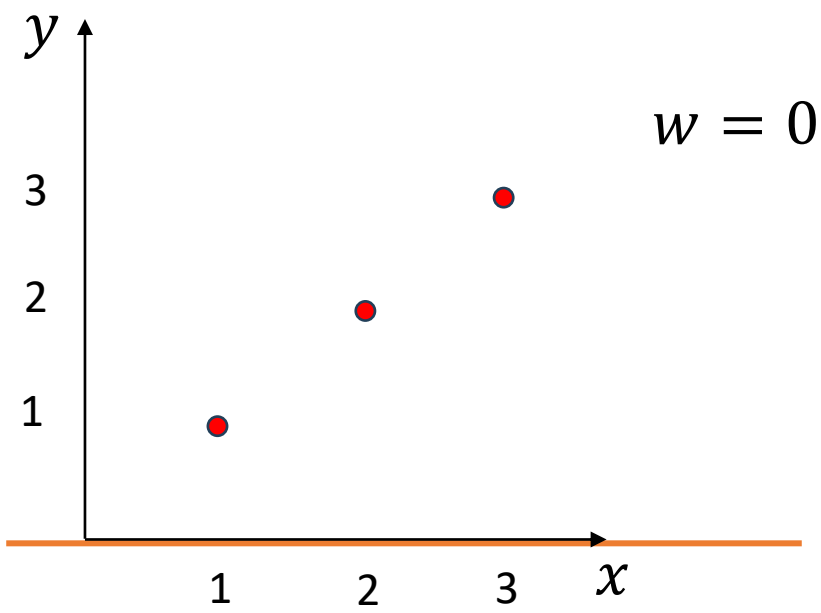
Machine Learning: *Supervised Learning*

Linear Regression

Cost Function: Intuition

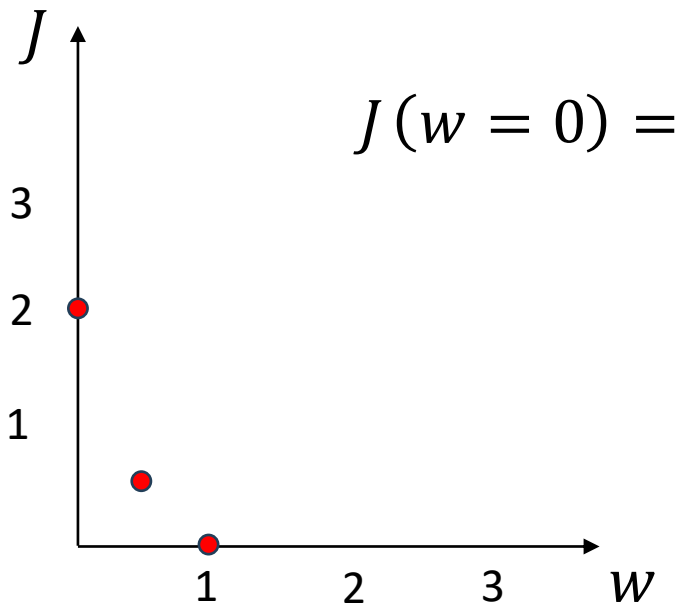
$$f_w(x) = wx$$

$y = f_w(x)$



$w = 0$

$J(w)$



$J(w = 0) = 2.3$

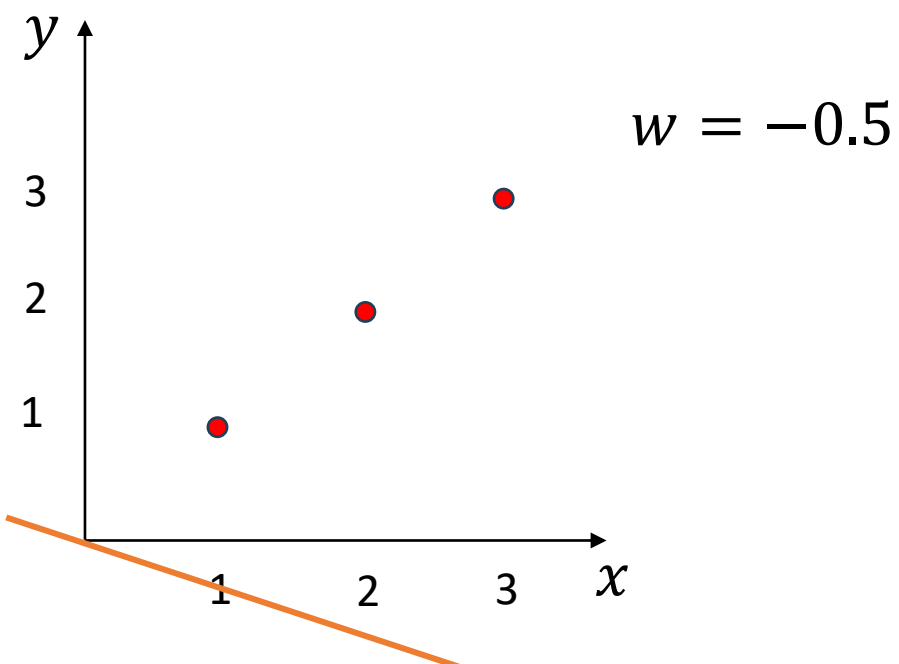
Machine Learning: *Supervised Learning*

Linear Regression

Cost Function: Intuition

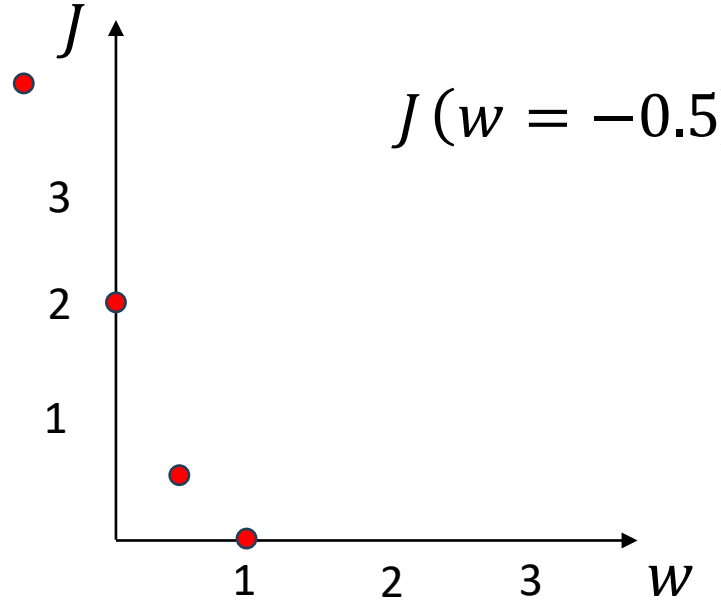
$$f_w(x) = wx$$

$y = f_w(x)$



$w = -0.5$

$J(w)$



$J(w = -0.5) = 5.25$

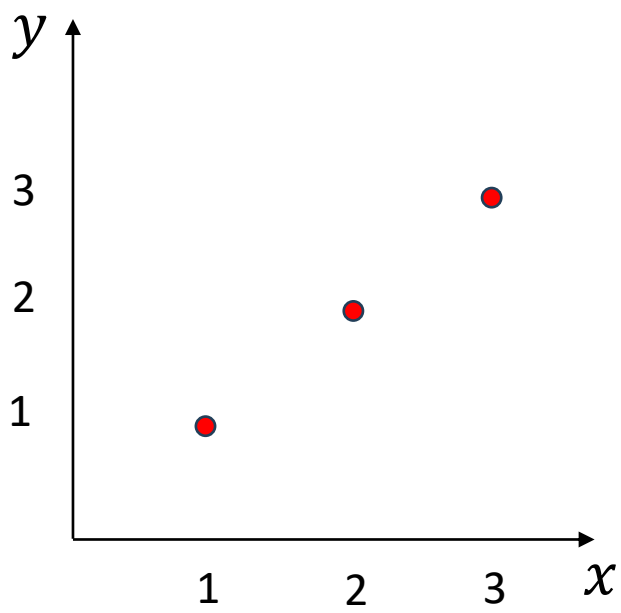
Machine Learning: *Supervised Learning*

Linear Regression

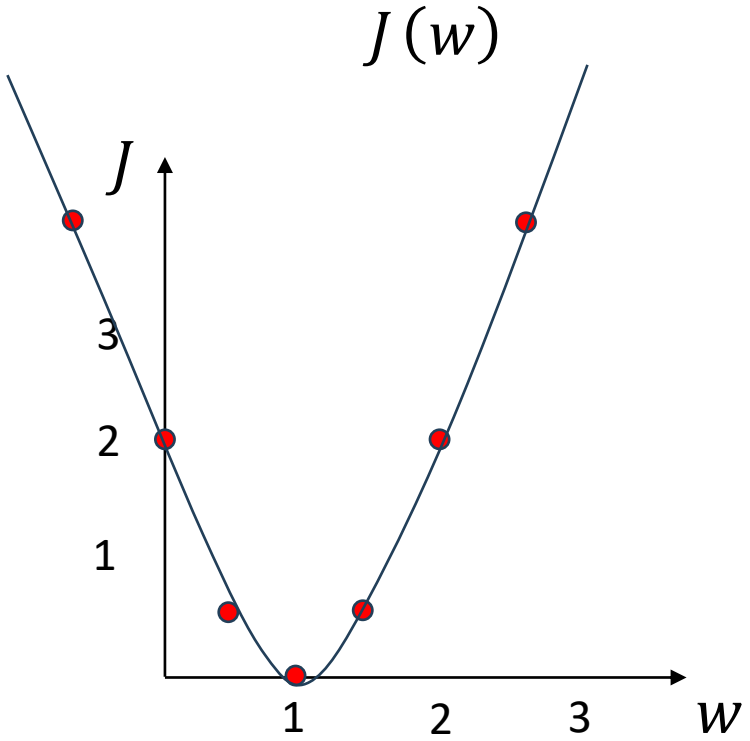
Cost Function: Intuition

$$f_w(x) = wx$$

$y = f_w(x)$



$J(w)$



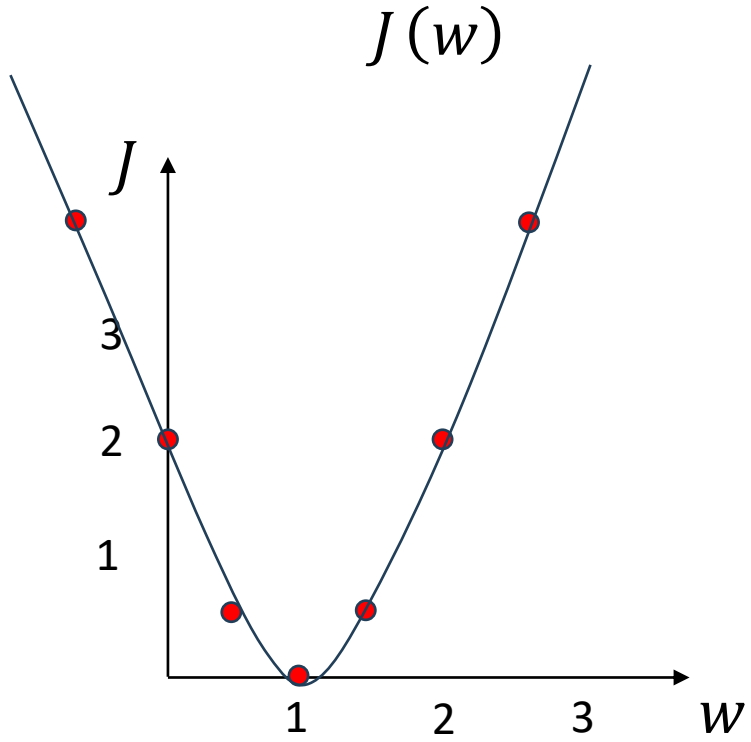
Machine Learning: *Supervised Learning*

Linear Regression

Cost Function: Intuition

$$f_w(x) = wx$$

Goal: $\min_w J(w)$



Machine Learning: *Supervised Learning*

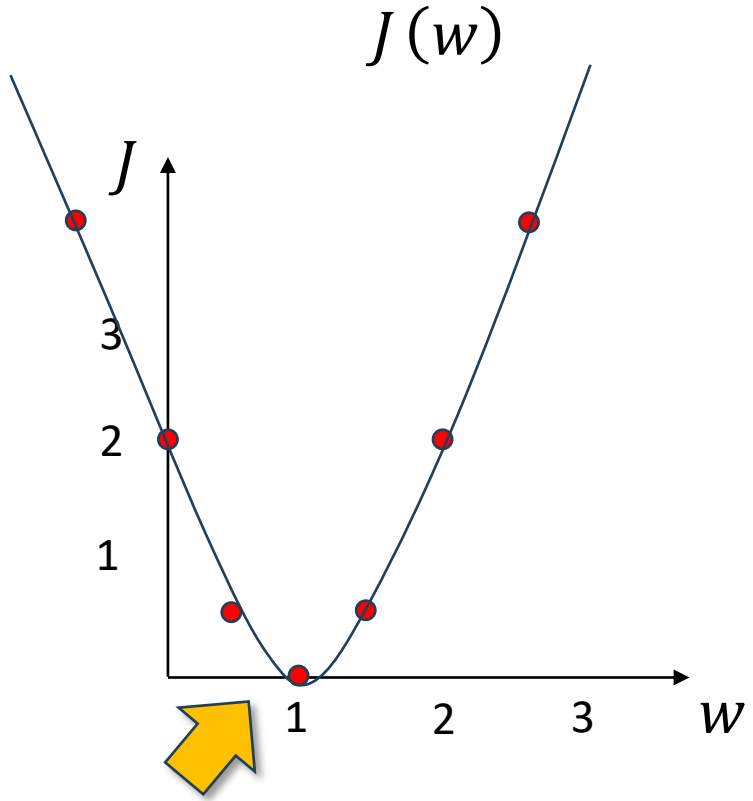
Linear Regression

Cost Function: Intuition

$$f_w(x) = wx$$

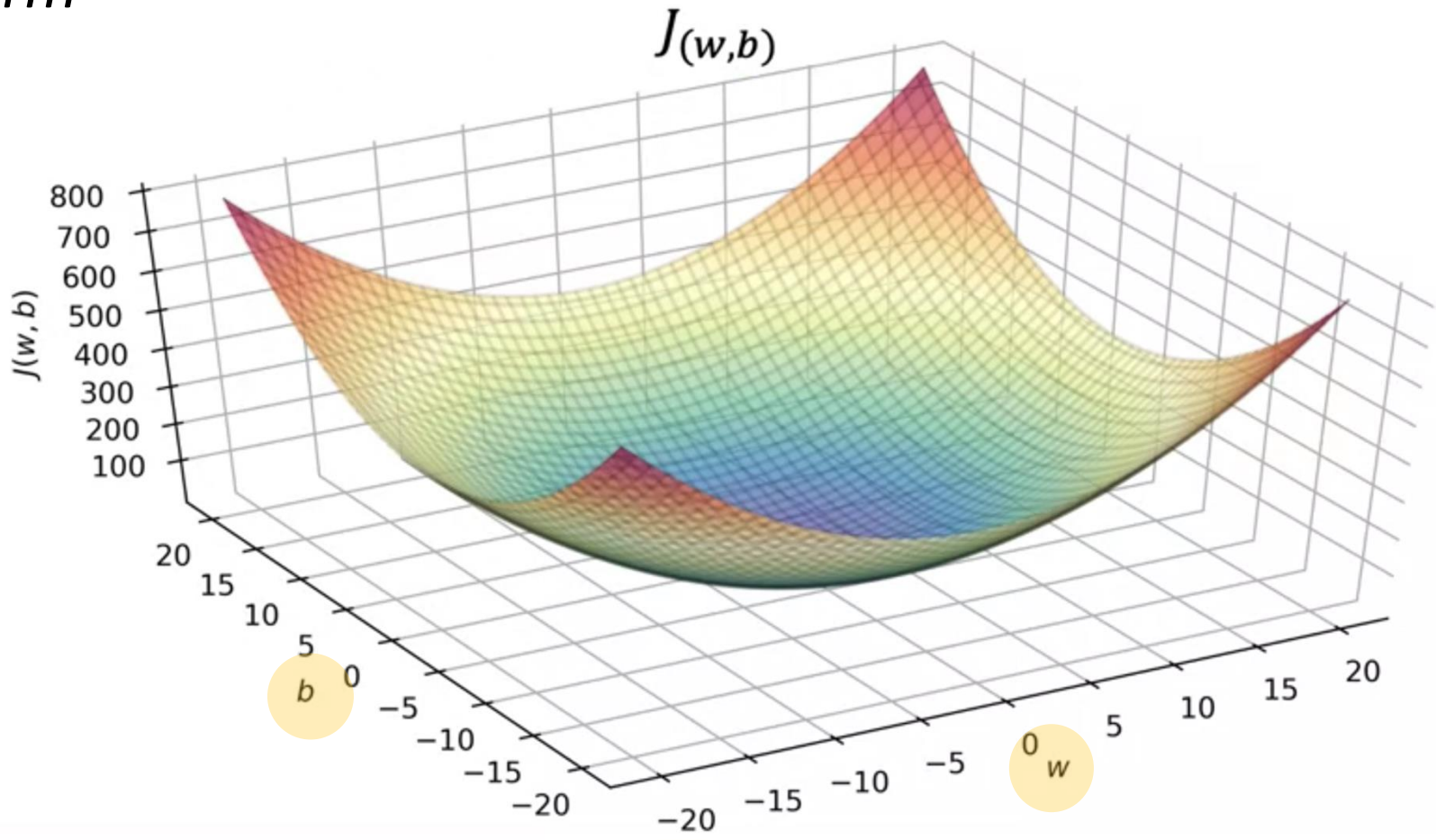
Goal: $\min_w J(w)$

$w = 1 \rightarrow J(w) = 0$



Machine Learning: *Linear Regression*

Cost Function: *General form*



Machine Learning: *Gradient Descent*

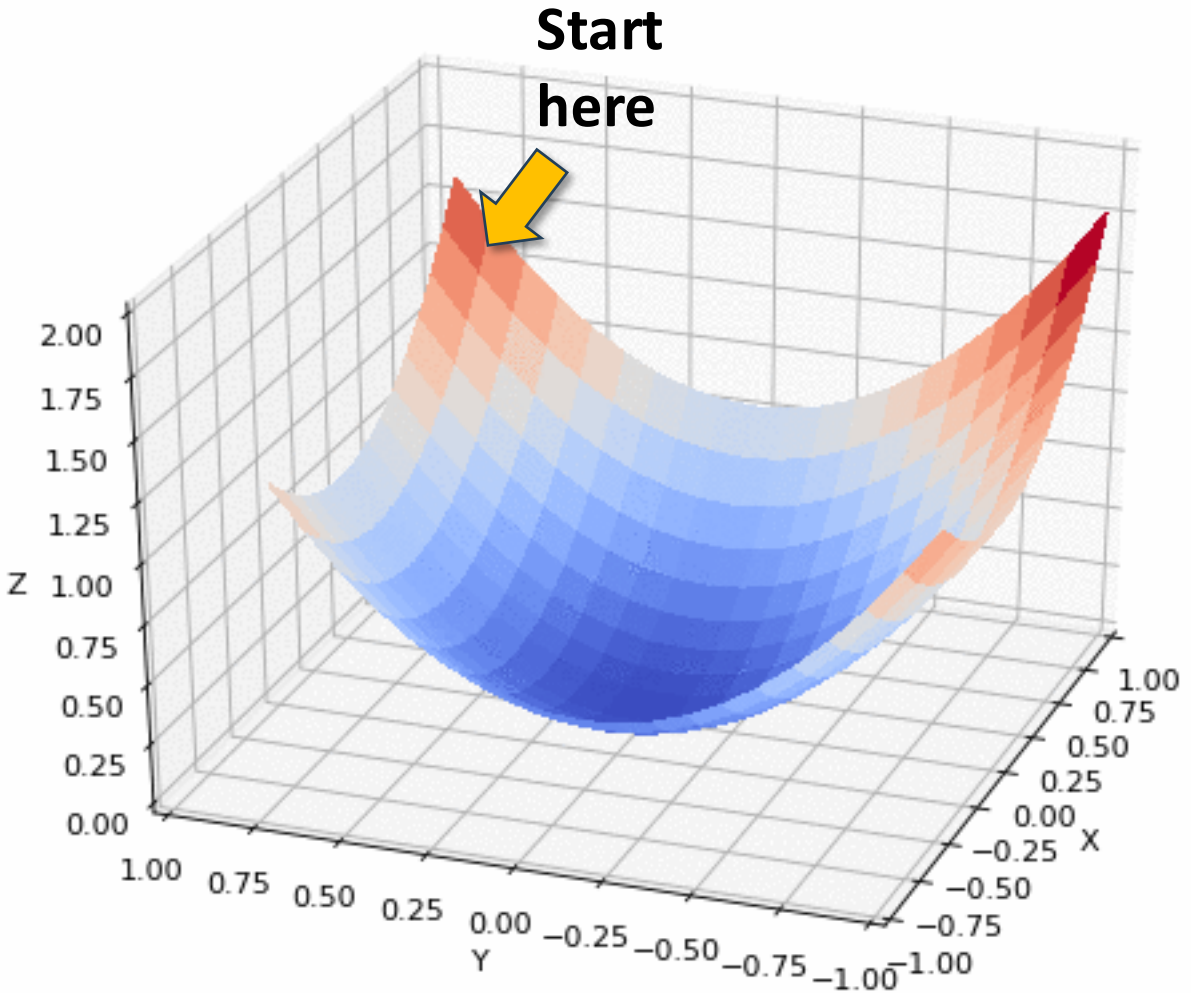
Algorithm

Select a function $J(w, b)$

Goal: $\min_{w,b} J(w, b)$

Strategy:

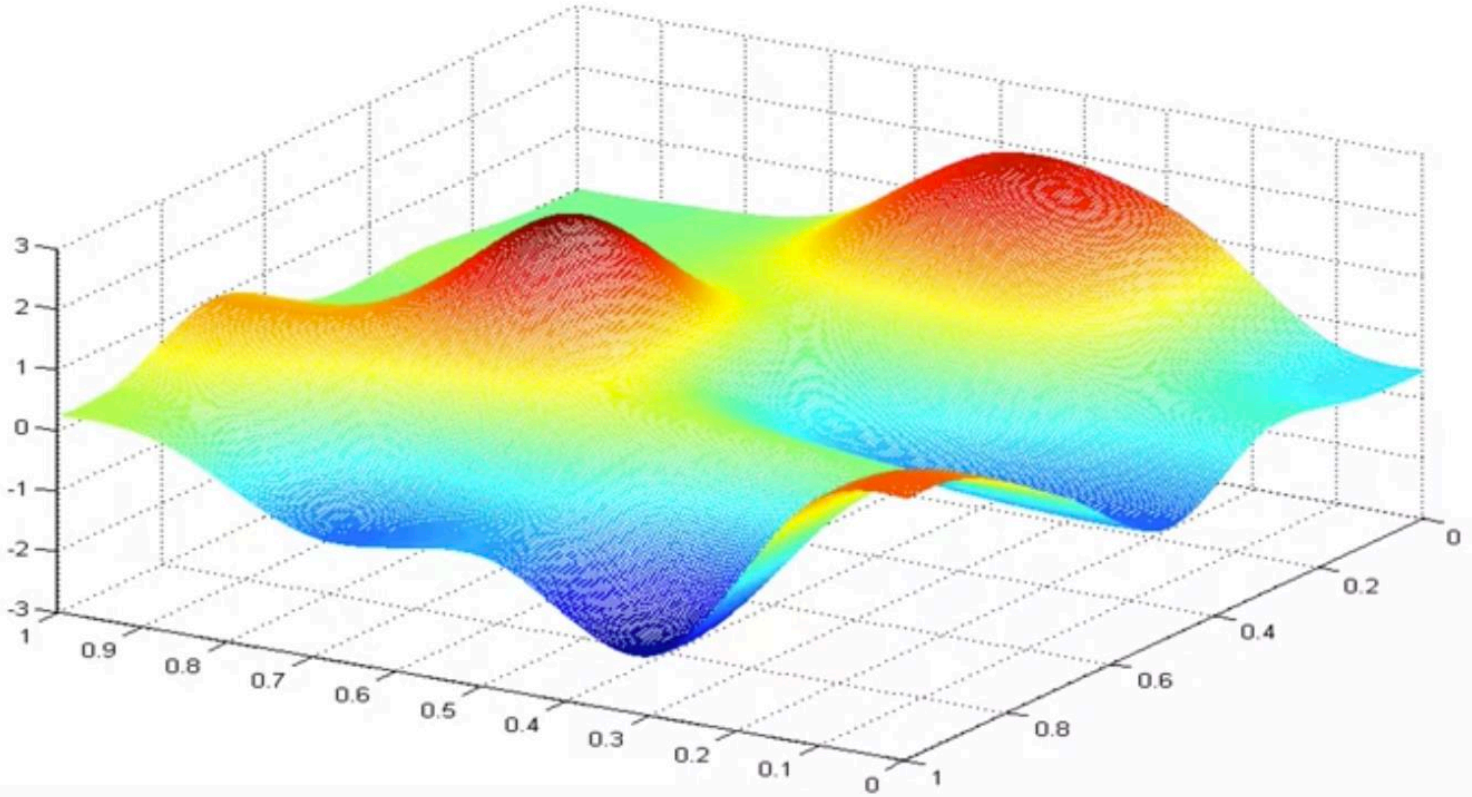
1. Start with some w, b
2. Change w, b to reduce $J(w, b)$
3. Repeat (2) until reach a minimum



DEMO: https://blog.skz.dev/gradient-descent

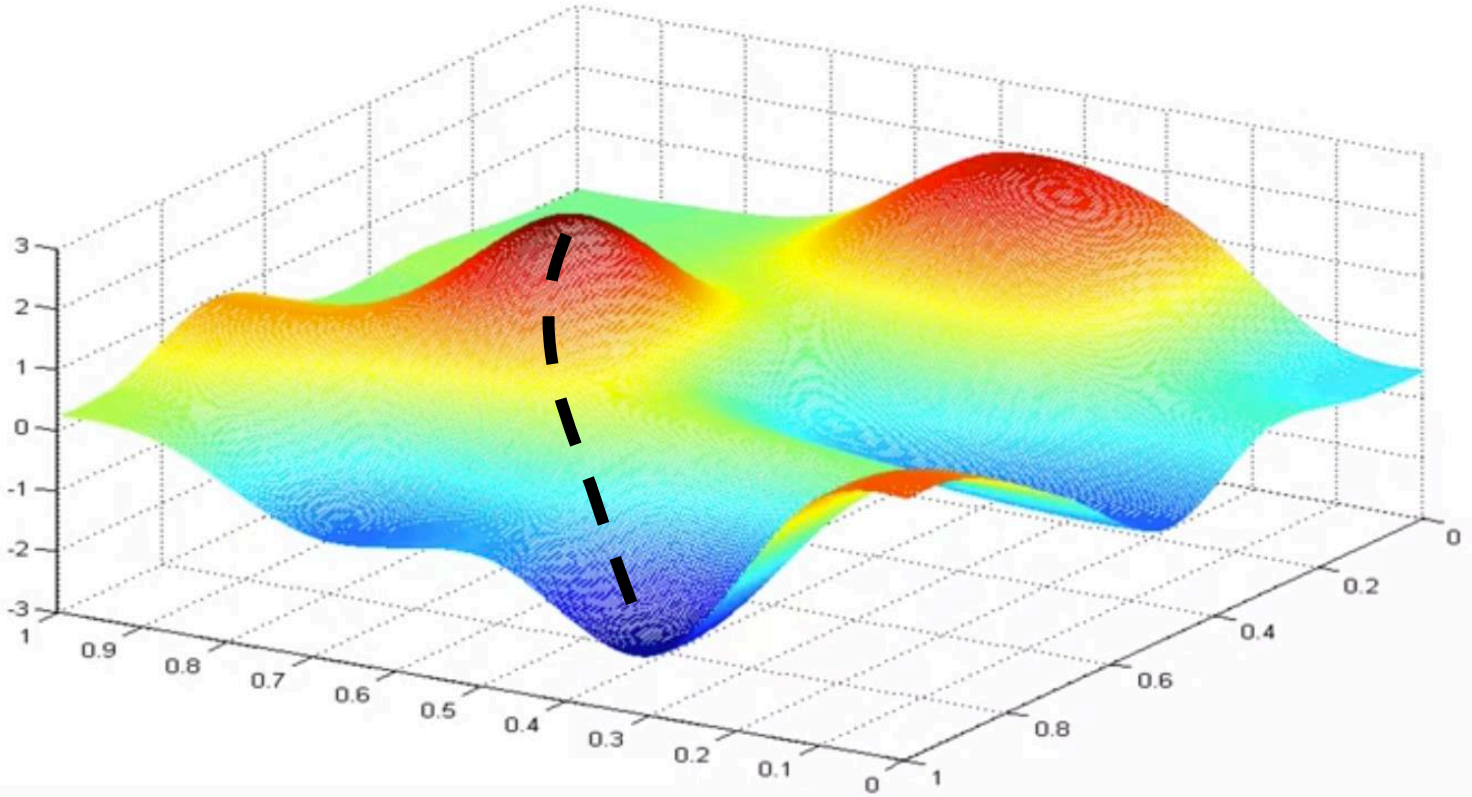
Machine Learning: *Gradient Descent*

If J is not a convex shape, then there might be multiple minimum values possible



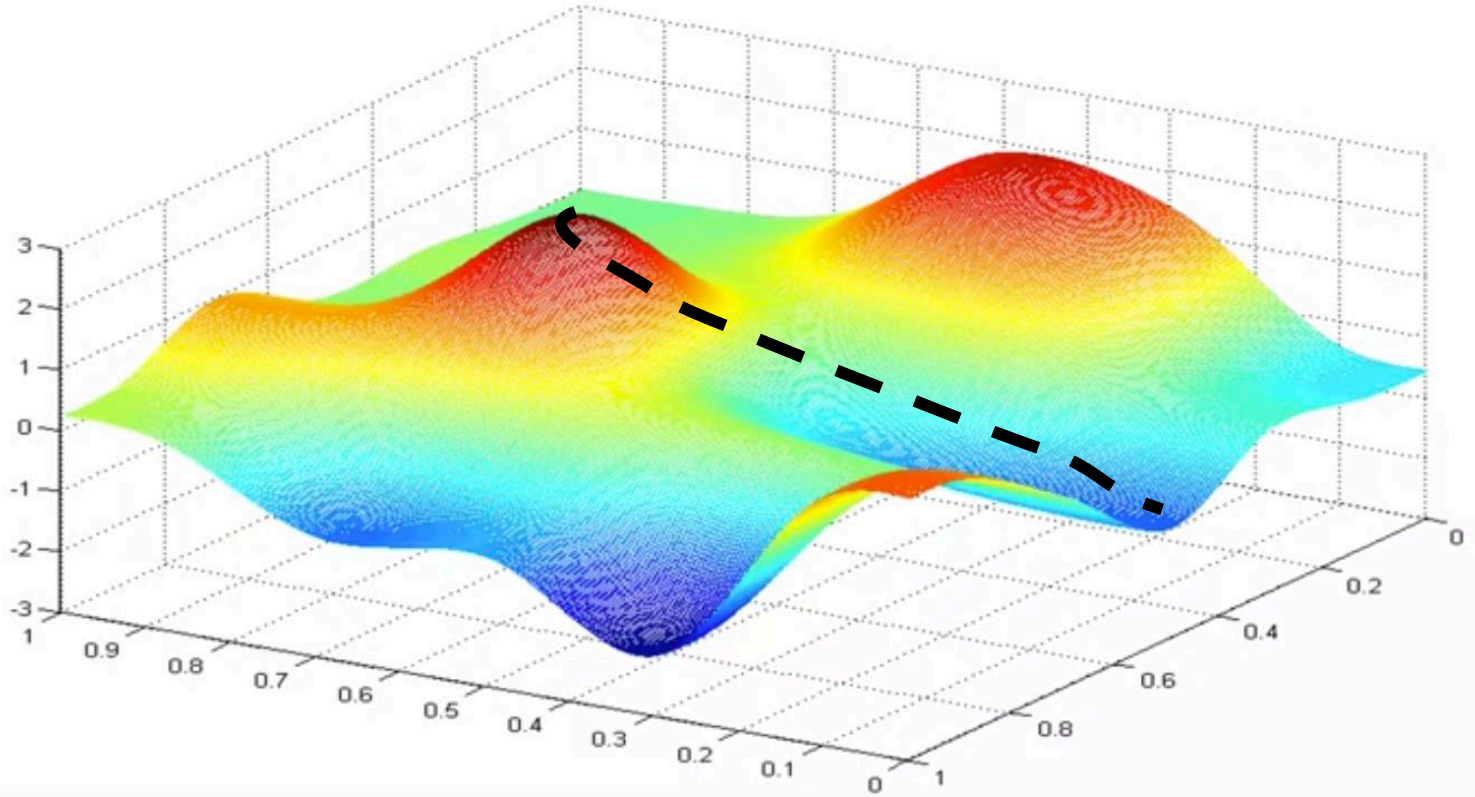
Machine Learning: *Gradient Descent*

If J is not a convex shape, then there might be multiple minimum values possible



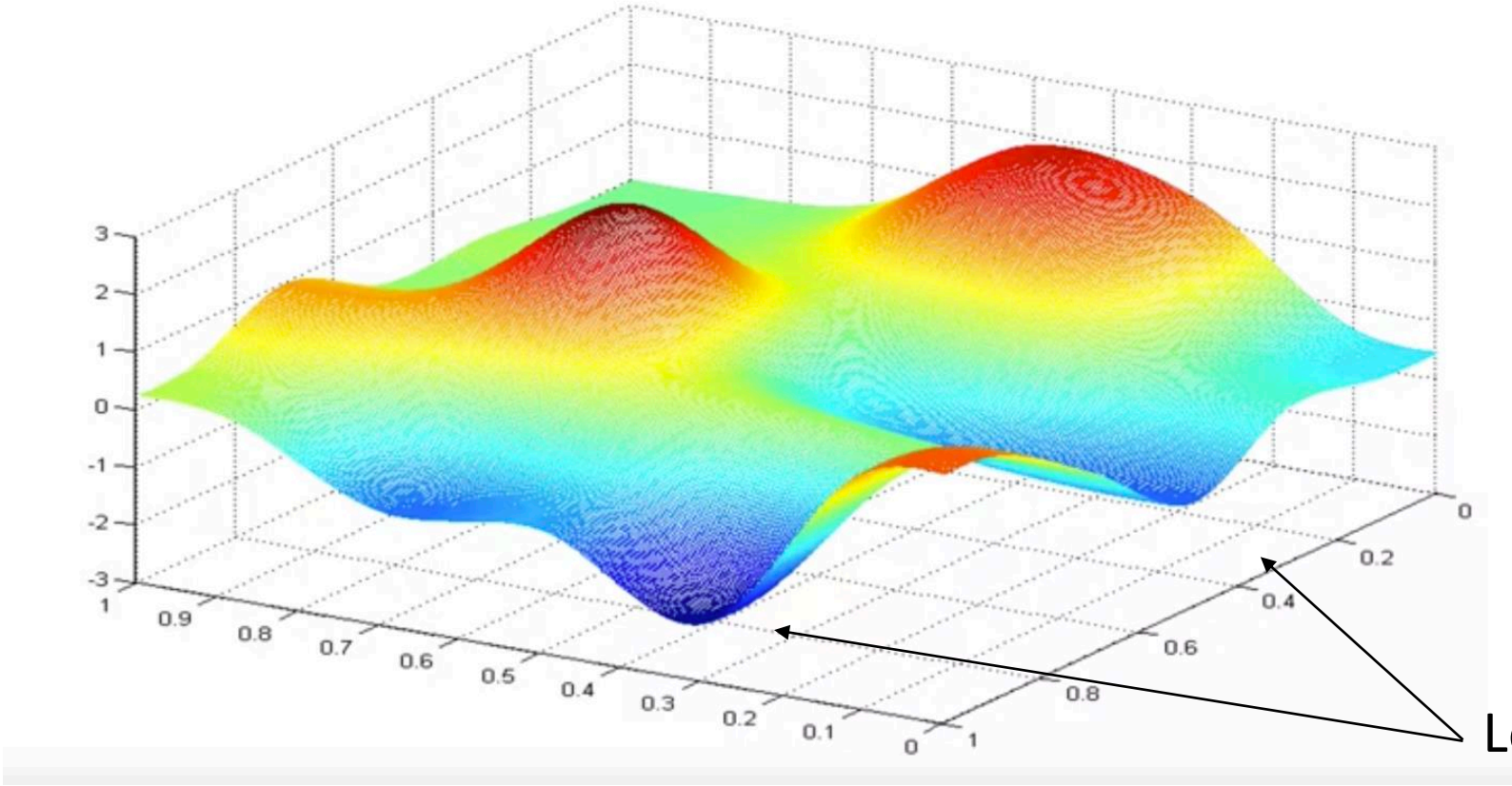
Machine Learning: *Gradient Descent*

If J is not a convex shape, then there might be multiple minimum values possible



Machine Learning: *Gradient Descent*

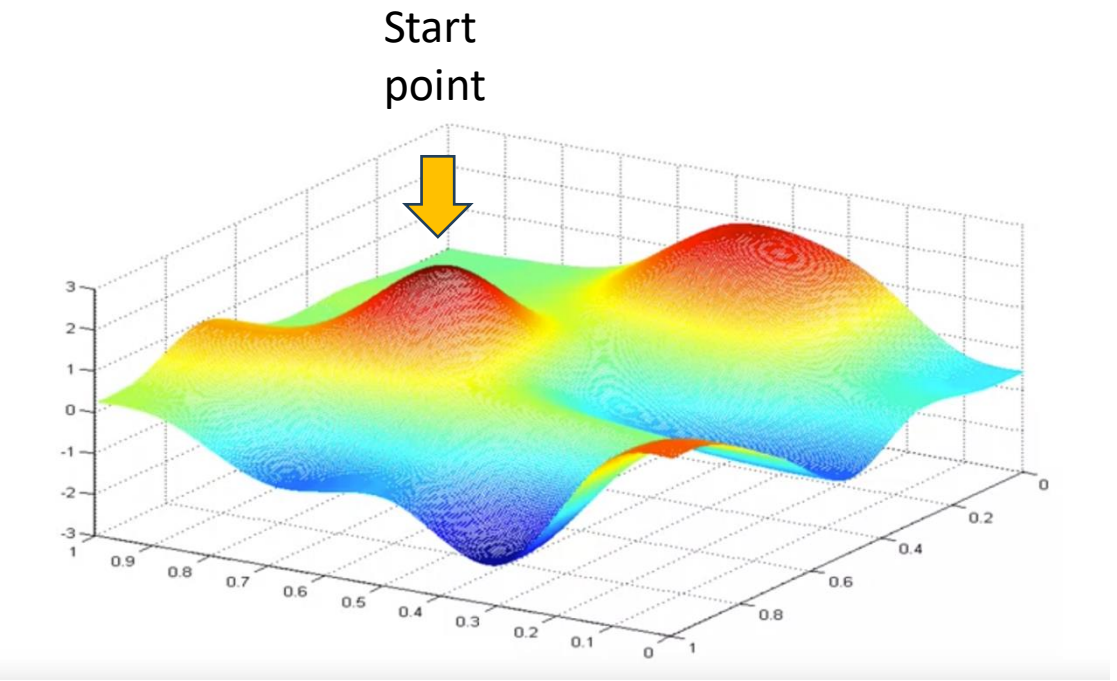
If J is not a convex shape, then there might be multiple minimum values possible



Machine Learning: *Gradient Descent*

Gradient Descent: Intuition

$$w_{next} = w_{current} - tiny\ change$$

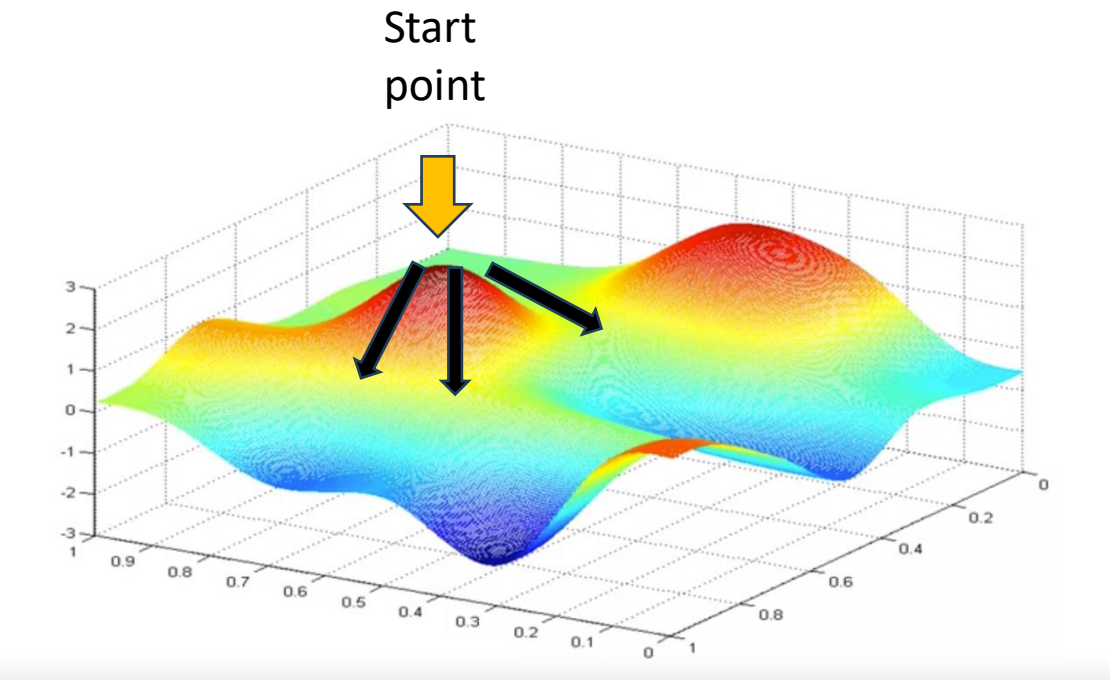


Machine Learning: *Gradient Descent*

Gradient Descent: Intuition

$$w_{next} = w_{current} - tiny\ change$$

Direction?

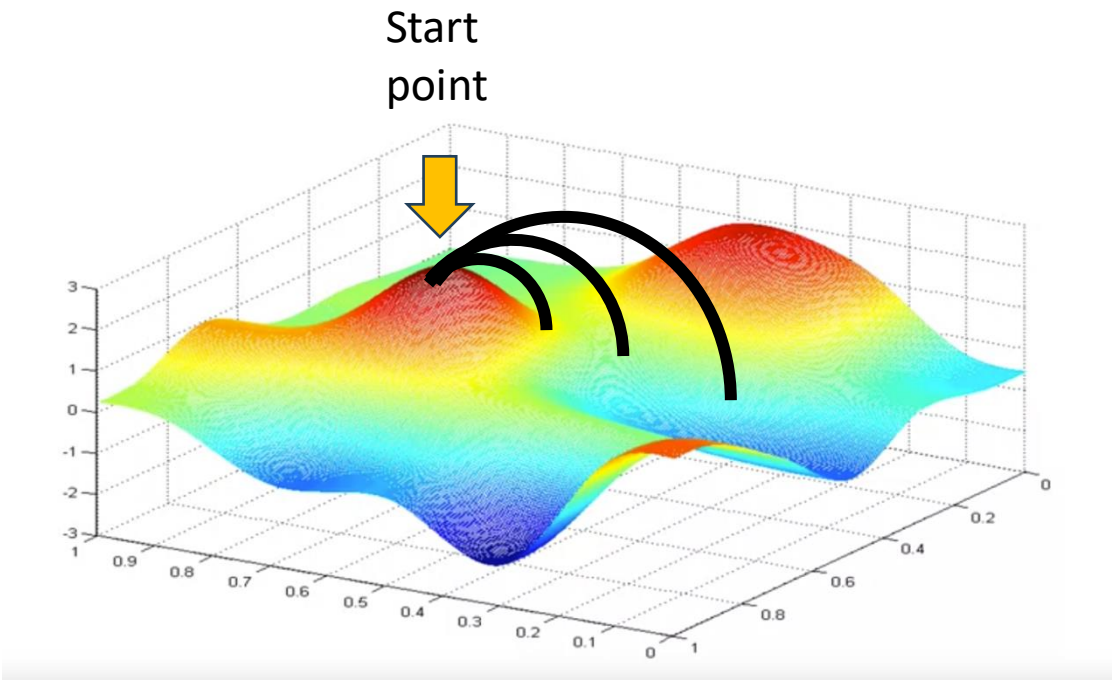


Machine Learning: *Gradient Descent*

Gradient Descent: Intuition

$$w_{next} = w_{current} - tiny\ change$$

Step size?



Machine Learning: *Gradient Descent*

Gradient Descent: Intuition

$$w_{next} = w_{current} - \frac{\partial}{\partial w} J(w, b)$$

DIRECTION

Machine Learning: *Gradient Descent*

Gradient Descent: Intuition

$$w_{next} = w_{current} - \alpha \frac{\partial}{\partial w} J(w, b)$$

STEP SIZE

Machine Learning: *Gradient Descent*

Gradient Descent: Formula

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

Learning Rate

$$b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

Machine Learning: *Gradient Descent*

Updating w , b

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

= : Assignment

Machine Learning: *Gradient Descent*

Updating w , b

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

Simultaneously update w, b

Machine Learning: *Gradient Descent*

Updating w , b

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

CORRECT

$$w_{tmp} = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

$$b_{tmp} = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

$$w = \mathrm{w_{tmp}}$$

$$\mathrm{b} = b_{tmp}$$

Machine Learning: *Gradient Descent*

Updating w , b

$$w = w - \alpha \frac{\partial}{\partial w} J(w,b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w,b)$$

CORRECT

$$w_{tmp} = w - \alpha \frac{\partial}{\partial w} J(w,b)$$

$$b_{tmp} = b - \alpha \frac{\partial}{\partial b} J(w,b)$$

$$w = \mathrm{w_{tmp}}$$

$$\mathrm{b} = b_{tmp}$$

INCORRECT

$$w_{tmp} = w - \alpha \frac{\partial}{\partial w} J(w,b)$$

$$w = \mathrm{w_{tmp}}$$

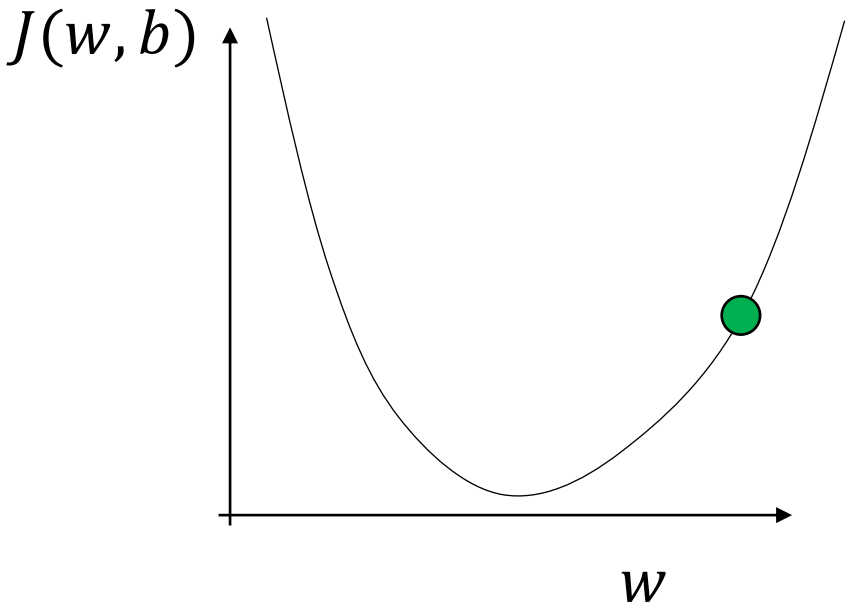
$$b_{tmp} = b - \alpha \frac{\partial}{\partial b} J(w,b)$$

$$\mathrm{b} = b_{tmp}$$

Machine Learning: *Gradient Descent*

Intuition

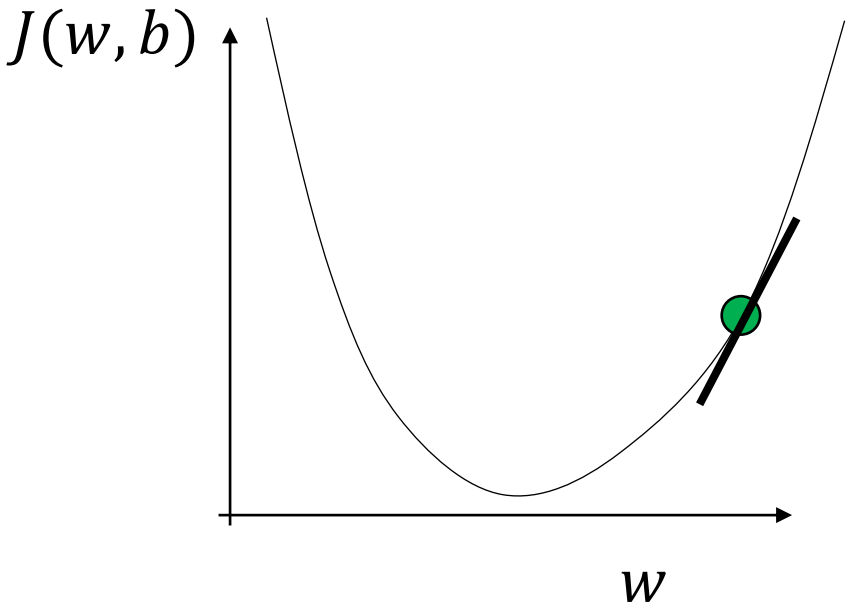
$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$



Machine Learning: *Gradient Descent*

Intuition

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

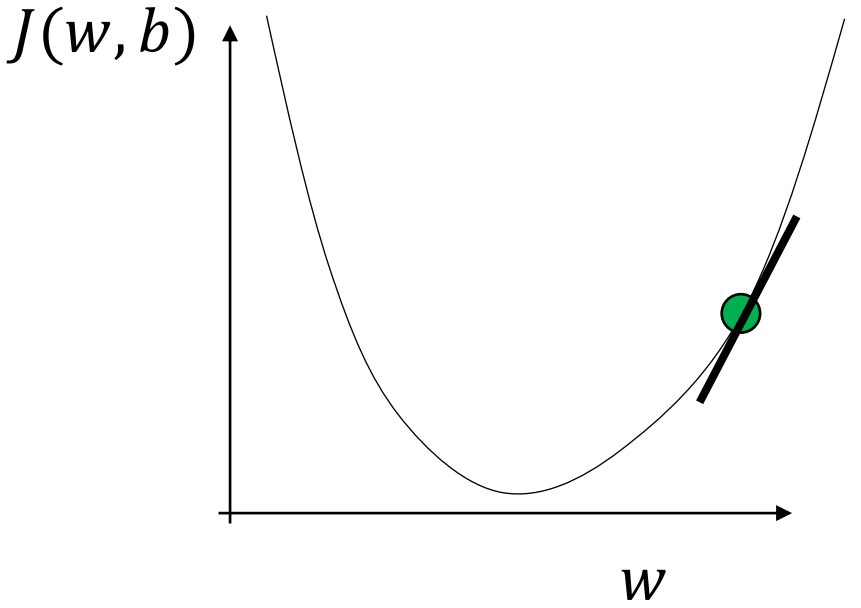


$$\frac{\partial}{\partial w} J(w, b) > 0$$

Machine Learning: *Gradient Descent*

Intuition

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$



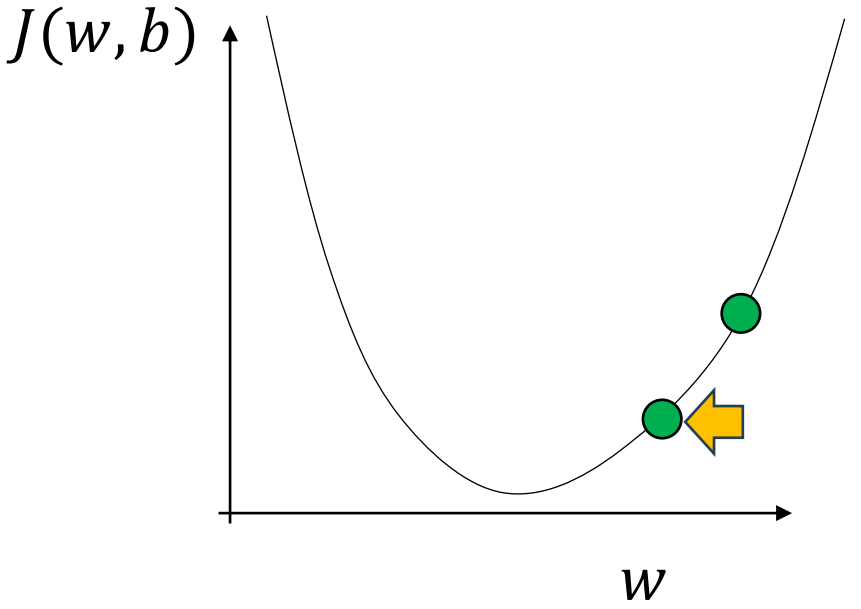
$$\frac{\partial}{\partial w} J(w, b) > 0$$

$$w - \alpha \frac{\partial}{\partial w} J(w, b) = w - \alpha.(positive)$$

Machine Learning: *Gradient Descent*

Intuition

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$



$$\frac{\partial}{\partial w} J(w, b) > 0$$

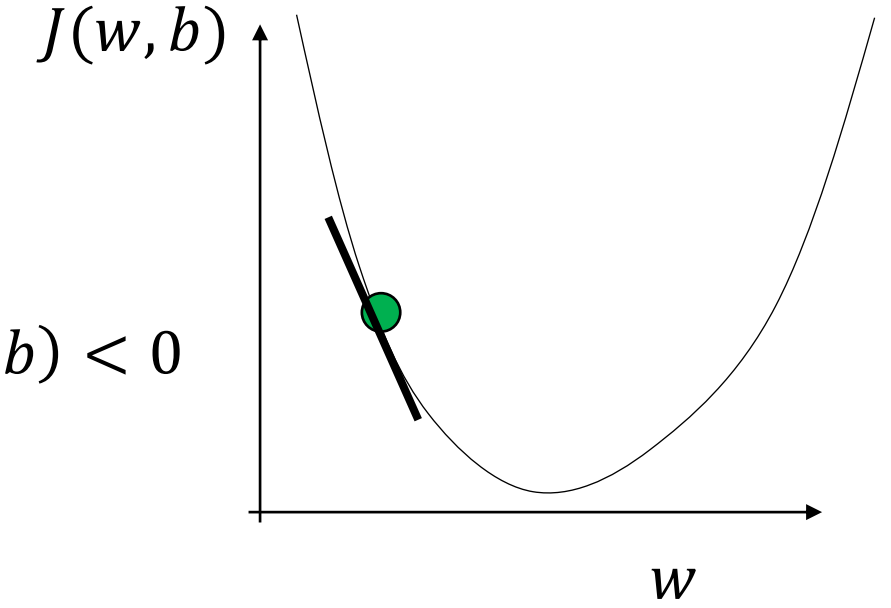
$$w - \alpha \frac{\partial}{\partial w} J(w, b) = w - \alpha . (positive)$$

Machine Learning: *Gradient Descent*

Intuition

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

$$\frac{\partial}{\partial w} J(w, b) < 0$$



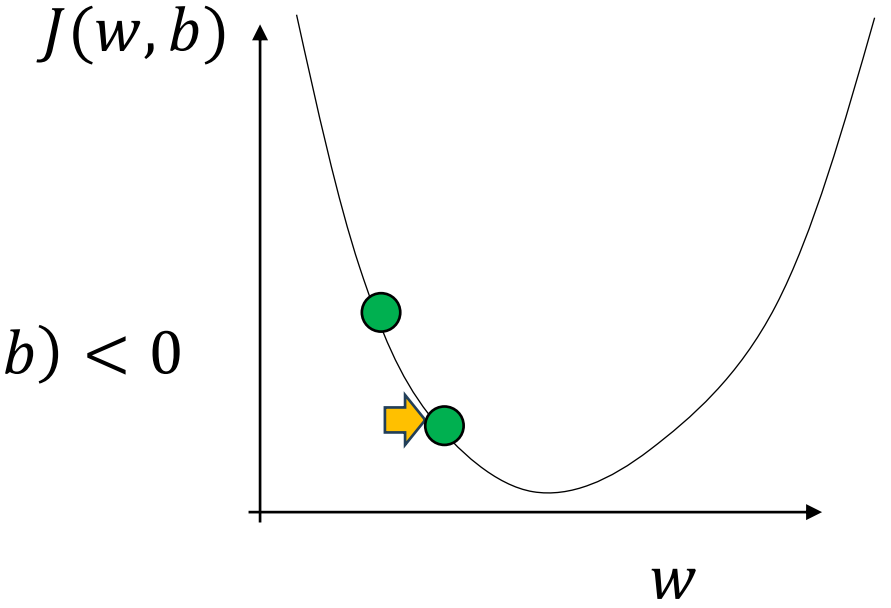
$$w - \alpha \frac{\partial}{\partial w} J(w, b) = w - \alpha . (negative)$$

Machine Learning: *Gradient Descent*

Intuition

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

$$\frac{\partial}{\partial w} J(w, b) < 0$$



$$w - \alpha \frac{\partial}{\partial w} J(w, b) = w - \alpha . (negative)$$

Machine Learning: *Gradient Descent*

Gradient Descent: Naming Convention

$$w_{next} = w_{current} - \alpha \frac{\partial}{\partial w} J(w, b)$$

Gradient

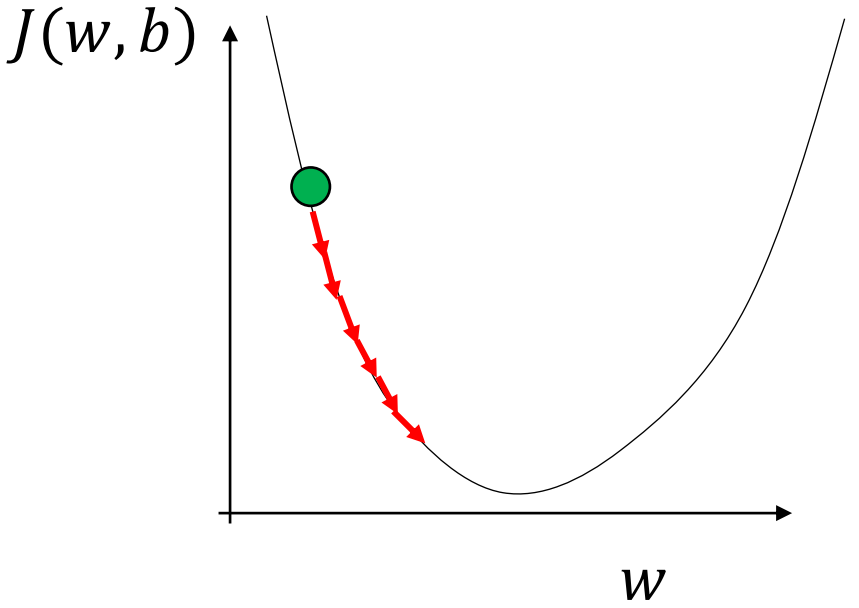
Descent

Machine Learning: *Gradient Descent*

Learning Rate

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

α: very small

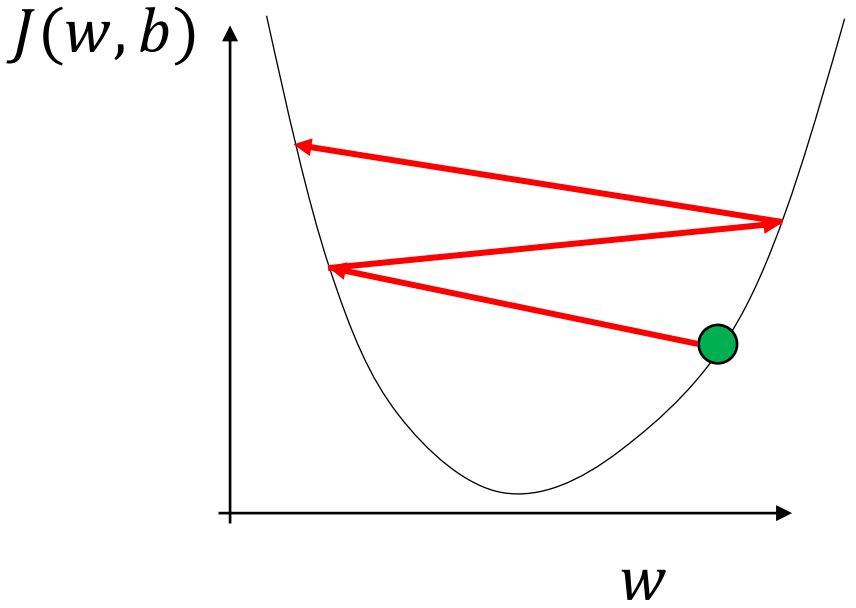


Machine Learning: *Gradient Descent*

Learning Rate

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

α: too large



Machine Learning: *Gradient Descent*

Algorithm

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{w,b}\left(x^{(i)}\right) - y^{(i)}\right)^2$$

$$w = w - \alpha\frac{\partial}{\partial w}J(w,b)$$

$$b = b - \alpha\frac{\partial}{\partial b}J(w,b)$$

Simultaneously update w, b

Machine Learning: *Gradient Descent for Linear Regression*

Repeat Until Convergence:

{

$$w = w - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{w,b}(x^{(i)}) - y^{(i)})$$

}

Simultaneously update w, b

$$f_{w,b}(x^{(i)}) = wx^{(i)} + b$$

Machine Learning: *Gradient Descent for Linear Regression*

Repeat Until Convergence:

{

$$w = w - \alpha \frac{1}{m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})x^{(i)}$$

$$b = b - \alpha \frac{1}{m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})$$

}

Simultaneously update w, b

$\frac{\partial}{\partial w}J(w,b)$

$\frac{\partial}{\partial b}J(w,b)$

$$f_{w,b}(x^{(i)}) = wx^{(i)} + b$$

Machine Learning: *General Form Linear Regression*

Multiple Linear Regression

x_j: j[th] feature
n: number of features
x_j^i: value of feature *j* in *i*[th] training sample

Model: $$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

Parameters: $$\vec{w} = [w_1 \quad w_2 \quad \ldots \quad w_n], b$$

Input $$\vec{x} = [x_1 \quad x_2 \quad \ldots \quad x_n]$$

Machine Learning: *General Form Linear Regression*

Multiple Linear Regression

x_j: j[th] feature
n: number of features
x_j^i: value of feature *j* in *i*[th] training sample

Model: $$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

Parameters: $$\vec{w} = [w_1 \quad w_2 \quad \ldots \quad w_n], b$$

Vector form: $$f_{\vec{w},b}(\vec{x}) = \vec{w}.\vec{x} + b$$

Machine Learning: *General Form Linear Regression*

Gradient Descent

$n = 1$

Repeat Until Convergence:

{

$$w = w - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{w,b}(x^{(i)}) - y^{(i)})$$

}

Simultaneously update w, b

$n \geq 2$

Repeat Until Convergence:

{

$$w_1 = w_1 - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_1^{(i)}$$

$\vdots$

$$w_n = w_n - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_n^{(i)}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

}

Simultaneously update $w_1, , w_2, \dots, w_n, b$

Machine Learning: *General Form Linear Regression*

Gradient Descent

$n = 1$

Repeat Until Convergence:

{

$$w = w - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{w,b}(x^{(i)}) - y^{(i)})$$

}

Simultaneously update w, b

$n \geq 2$

Repeat Until Convergence:

{

$$w_j = w_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(x^{(i)}) - y^{(i)})$$

}

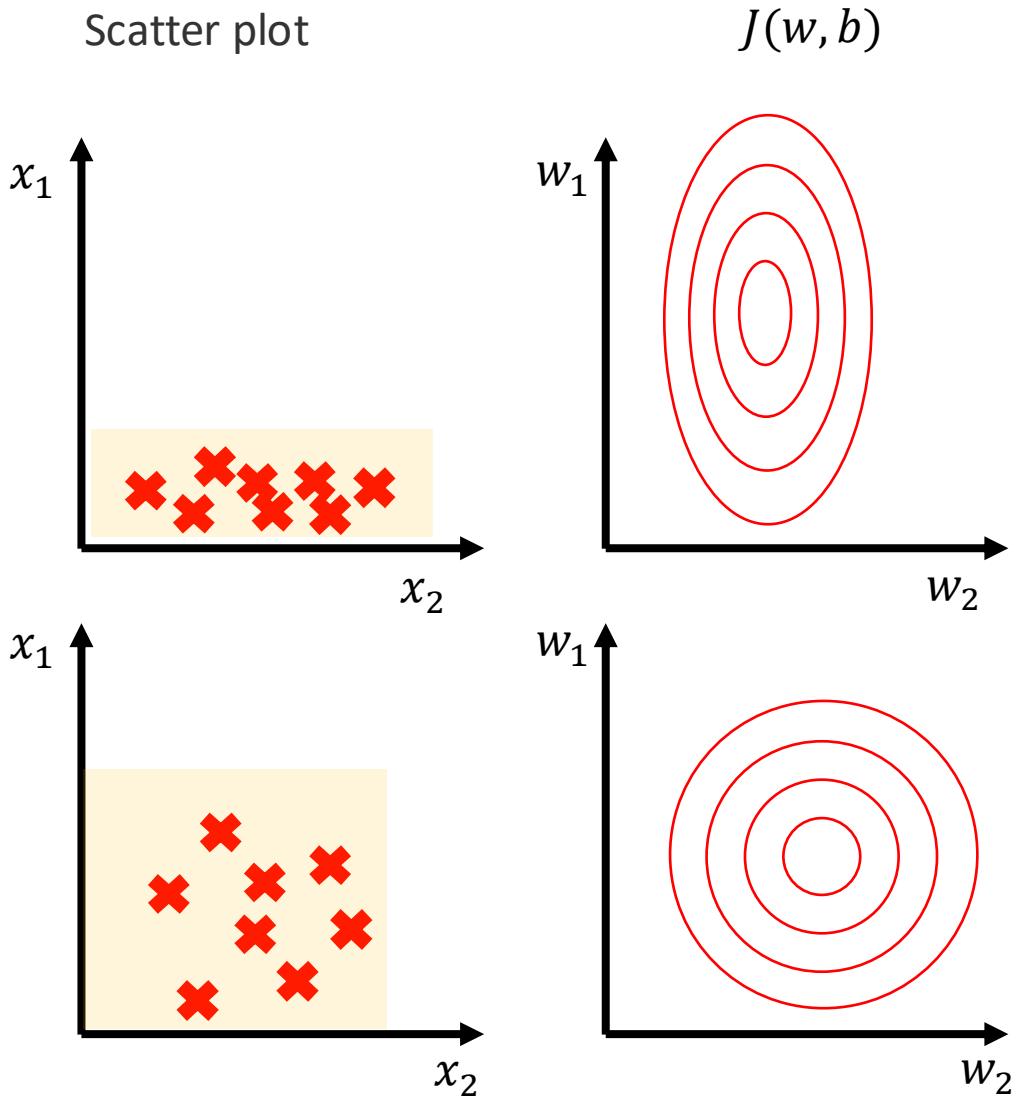
Simultaneously update $\vec{w}\ (w_1, , w_2, \dots, w_n), b$

Machine Learning: *Feature Scale*

Feature Scale

Feature scaling normalizes the ranges of input features so that no feature dominates the gradient updates.

This makes the cost function surface more symmetric, allowing gradient descent to take larger, more efficient steps toward the minimum instead of oscillating or taking a zig-zag path.

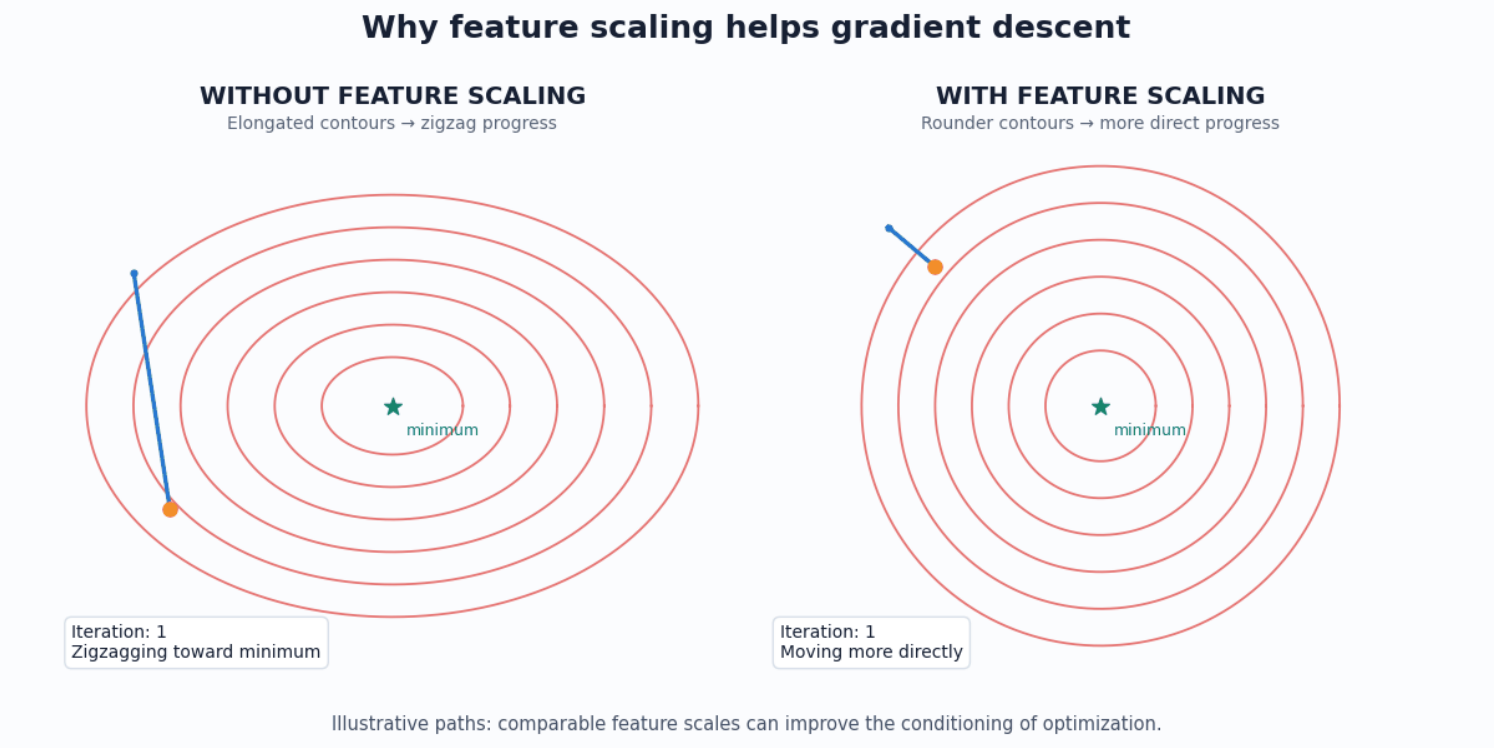


Machine Learning: *Feature Scale*

Feature Scale

Feature scaling normalizes the ranges of input features so that no feature dominates the gradient updates.

This makes the cost function surface more symmetric, allowing gradient descent to take larger, more efficient steps toward the minimum instead of oscillating or taking a zig-zag path.



Machine Learning: *Feature Scaling*

Methods

- Divide by max

$$0 \leq \frac{x}{x_{max}} \leq 1$$

$$500 < x_1 < 1000 \qquad 0.01 < x_2 < 10$$

$$\frac{500}{1000} < x_1 < \frac{1000}{1000} \qquad \frac{0.01}{10} < x_2 < \frac{10}{10}$$

$$0.5 < x_1 < 1 \qquad 0.001 < x_2 < 1$$

Machine Learning: *Feature Scaling*

Methods

- Divide by max
- Mean normalization

$$-1 \leq \frac{x - \mu_x}{x_{max} - x_{min}} \leq 1$$

μ_x: mean

Machine Learning: *Feature Scaling*

Methods

- Divide by max
- Mean normalization
- Z-score normalization

$$-1 \leq \frac{x - \mu_x}{\sigma_x} \leq 1$$

μ_x: mean

σ_x: standard Deviation

Machine Learning: *Feature Engineering*

Definition

Using Intuition to come up with new features derived from original features by:

- Selecting
- Combining
- Transforming

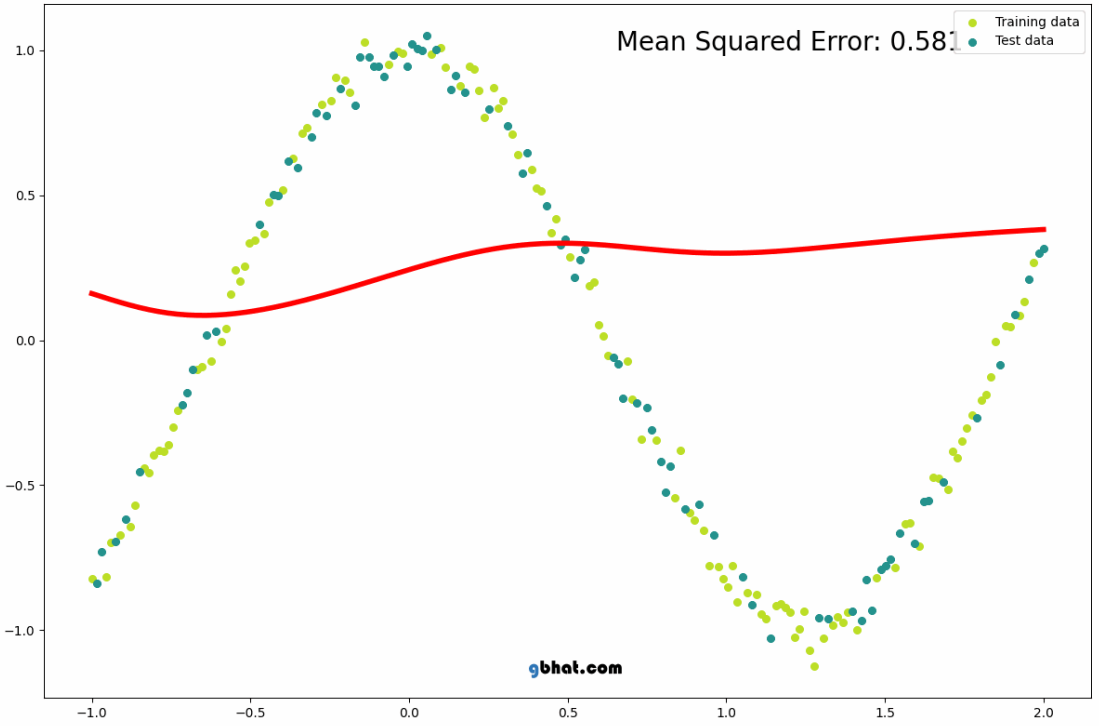
Goal

- **Improve model accuracy**: better capture the relationships within your data.
- **Reduce complexity**: reduce the dimensionality of data, making it easier for for models to learn from and avoid overfitting.
- **Handle data issues**: address common data problems like missing values, outliers, and categorical data, leading to more robust models.
- **Domain knowledge integration**: incorporate domain expertise into the process by creating features that reflect specific insights about the problem trying to solve.

Machine Learning: *Polynomial Regression*

Definition

- A method for capturing non-linear relationships between the independent variables (features) and the dependent variable (target)
- Modeled as an nth degree polynomial.



Advantages

- Can capture complex non-linear relationships
- Easy to interpret the coefficients of the polynomial
- Flexible and can be adapted to different data types

Disadvantages

- Prone to overfitting, especially with higher degrees
- Can be computationally expensive for large datasets
- Susceptible to noise in the data
- Less interpretable than simpler models like linear regression

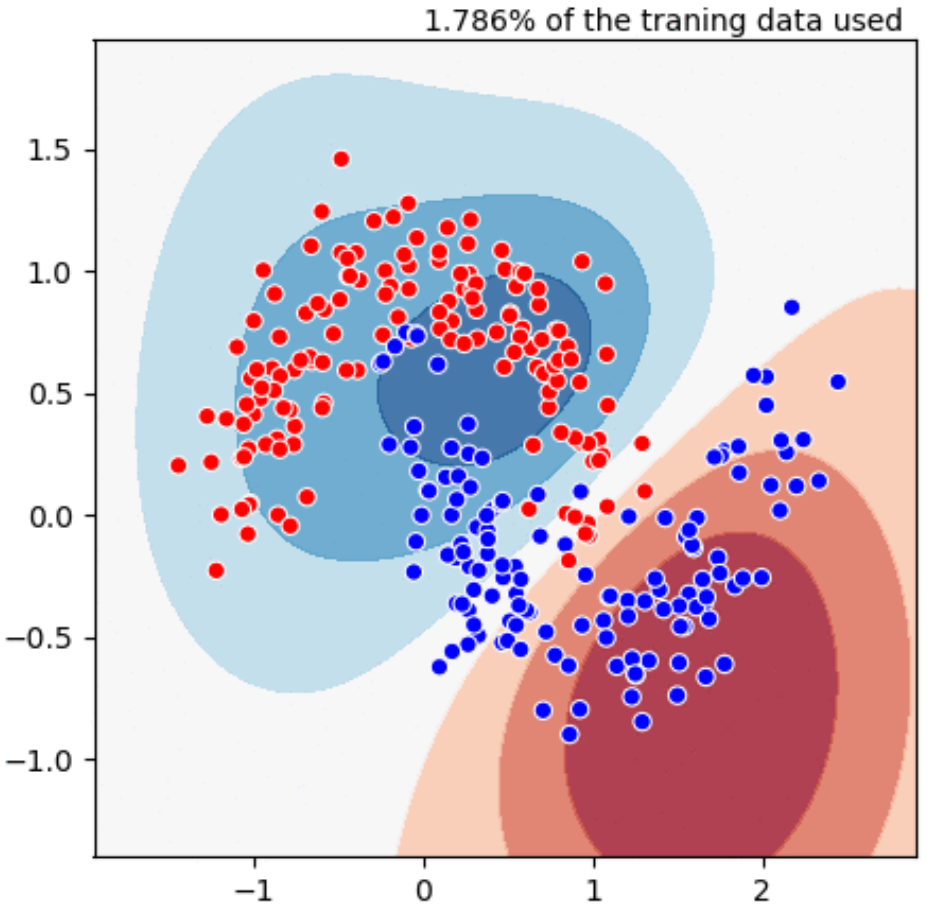
Machine Learning: *Categories (by Output)*

Classification

- Categorize data points into discrete classes or groups.
- Creating a model that learns the characteristics of each class based on a set of labeled training data.

Output: Discrete categories (labels)
(e.g., spam/not spam, dog/cat, healthy/sick, etc.)

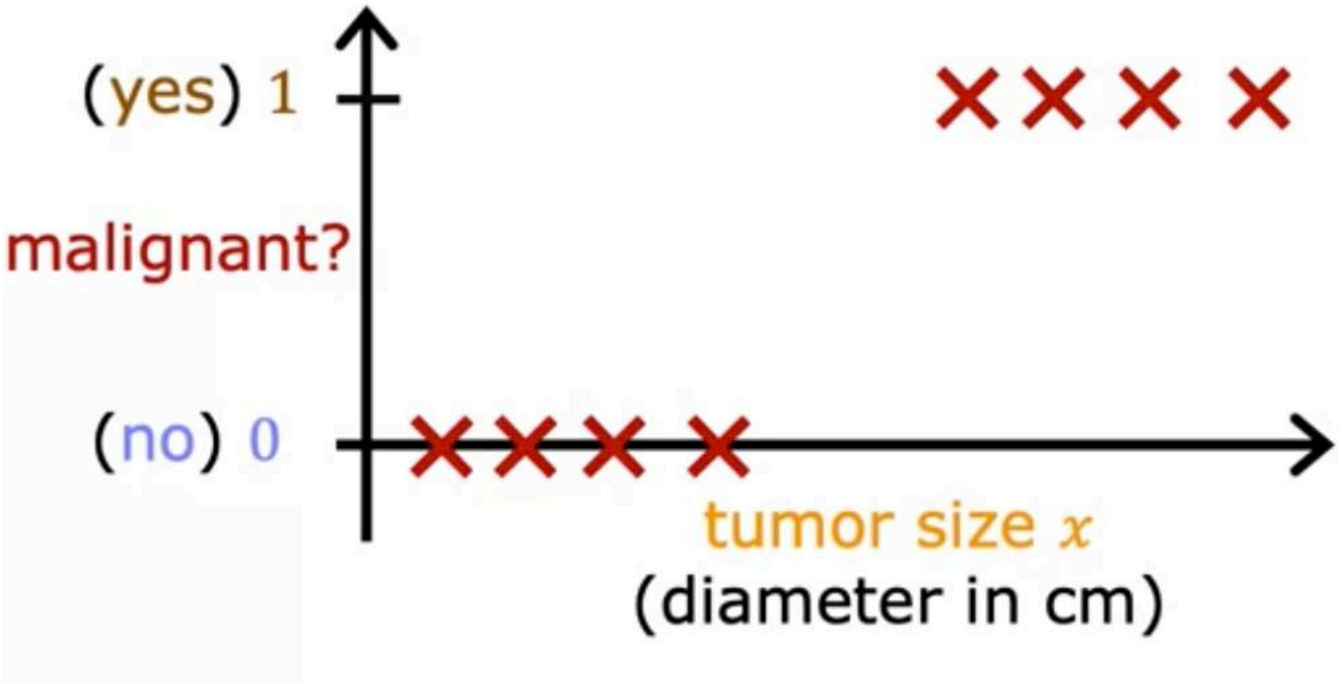
Objective: Assign a data point to one of the predefined classes based on its features.



https://towardsdatascience.com/the-simplest-way-of-making-gifs-and-math-videos-with-python-aec41da74c6e

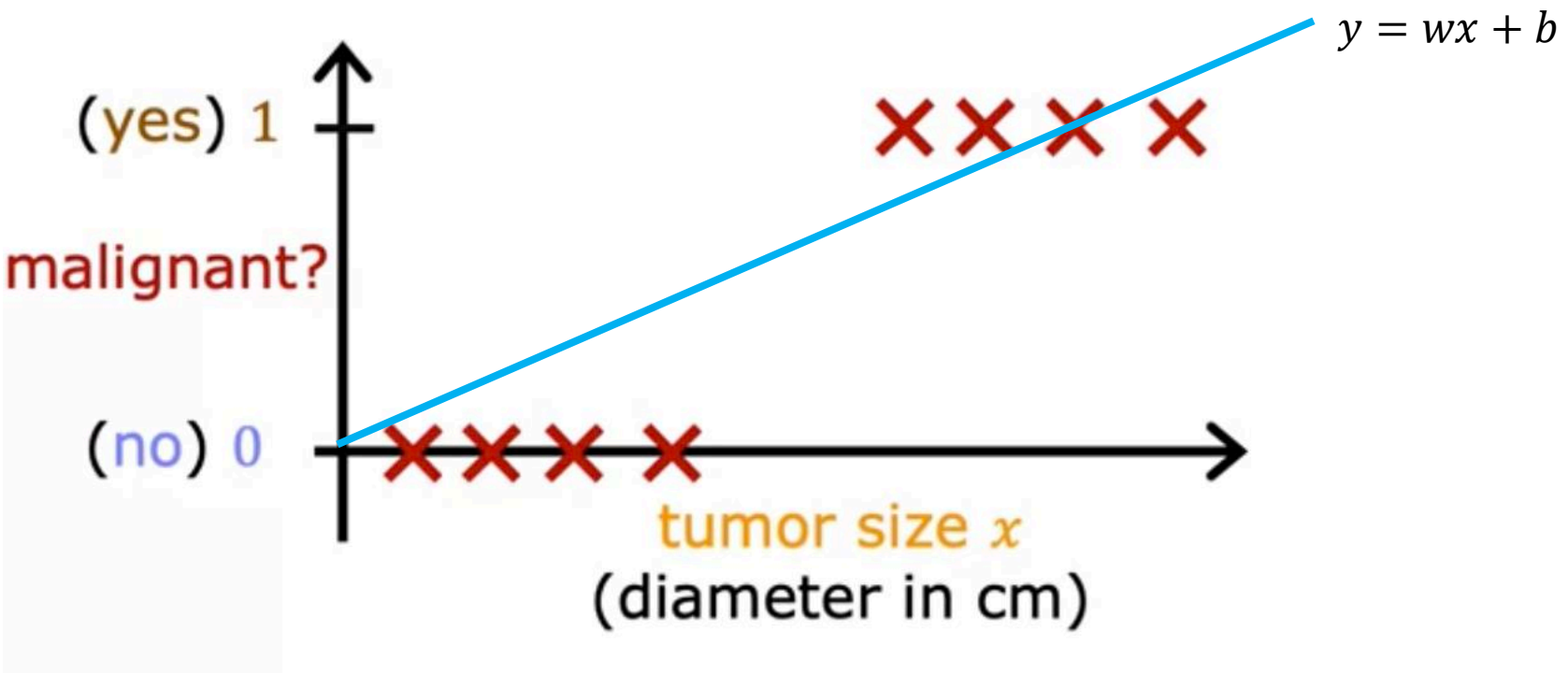
Machine Learning: *Classification*

Example



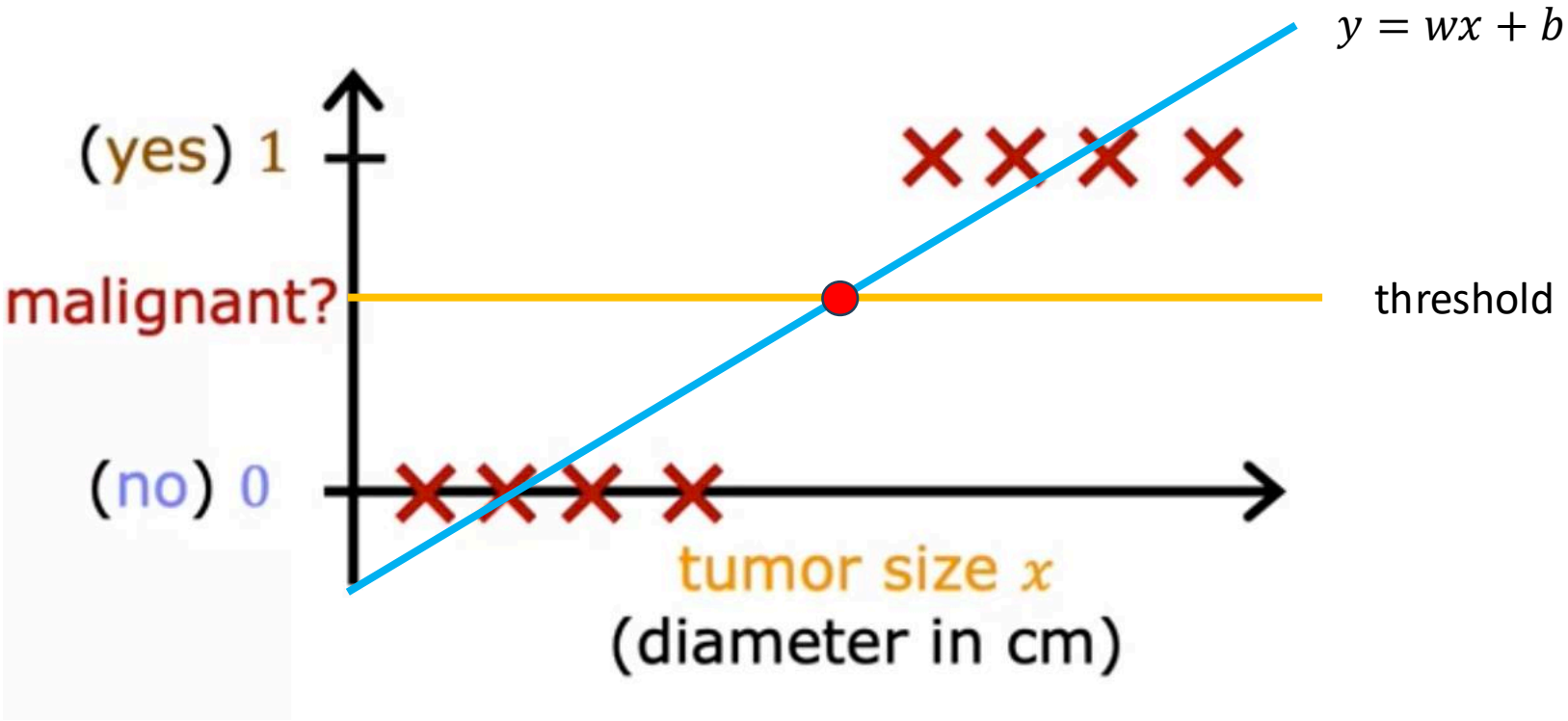
Machine Learning: *Classification*

Classification with Linear Regression



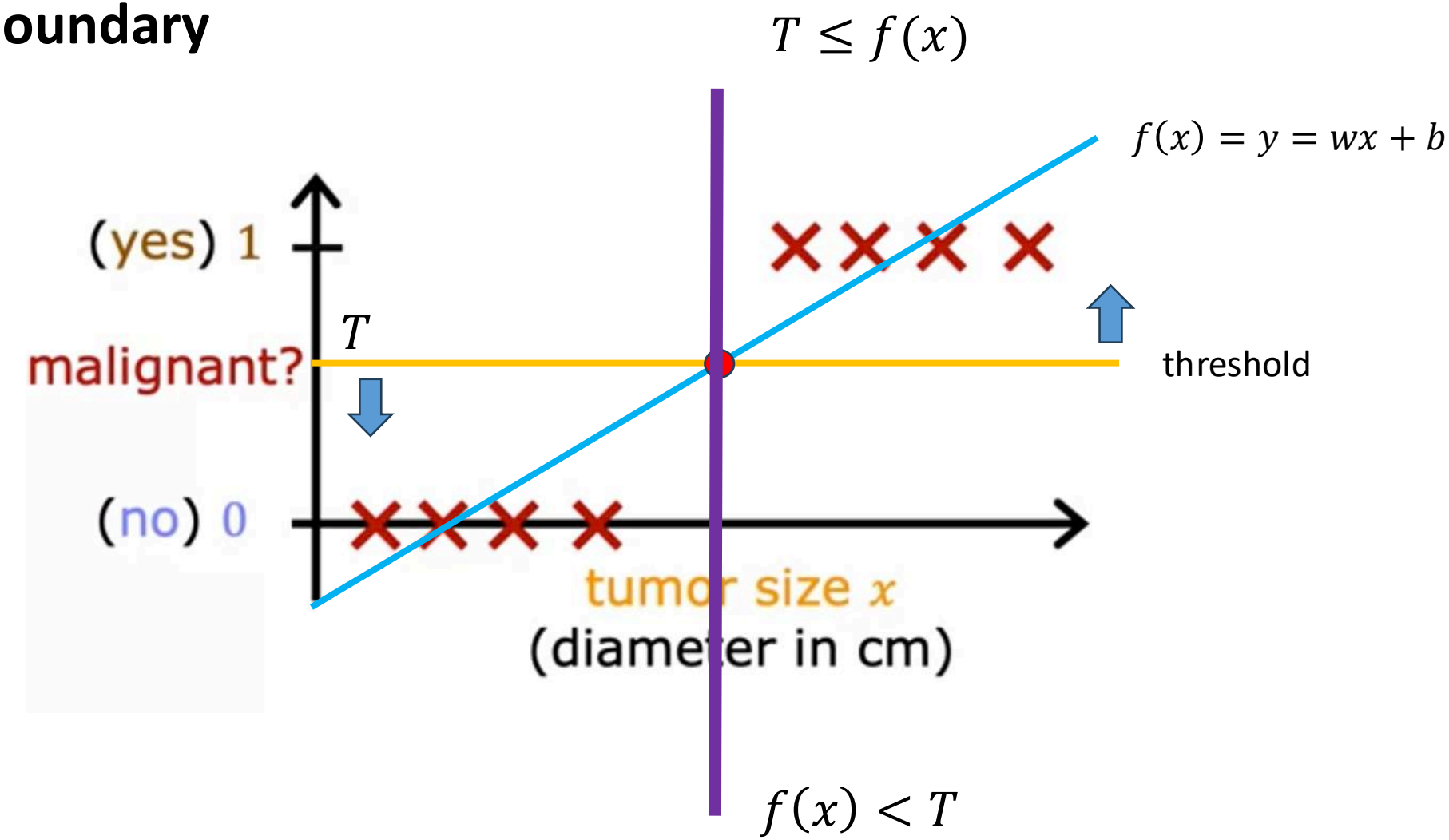
Machine Learning: *Classification*

Classification with Linear Regression



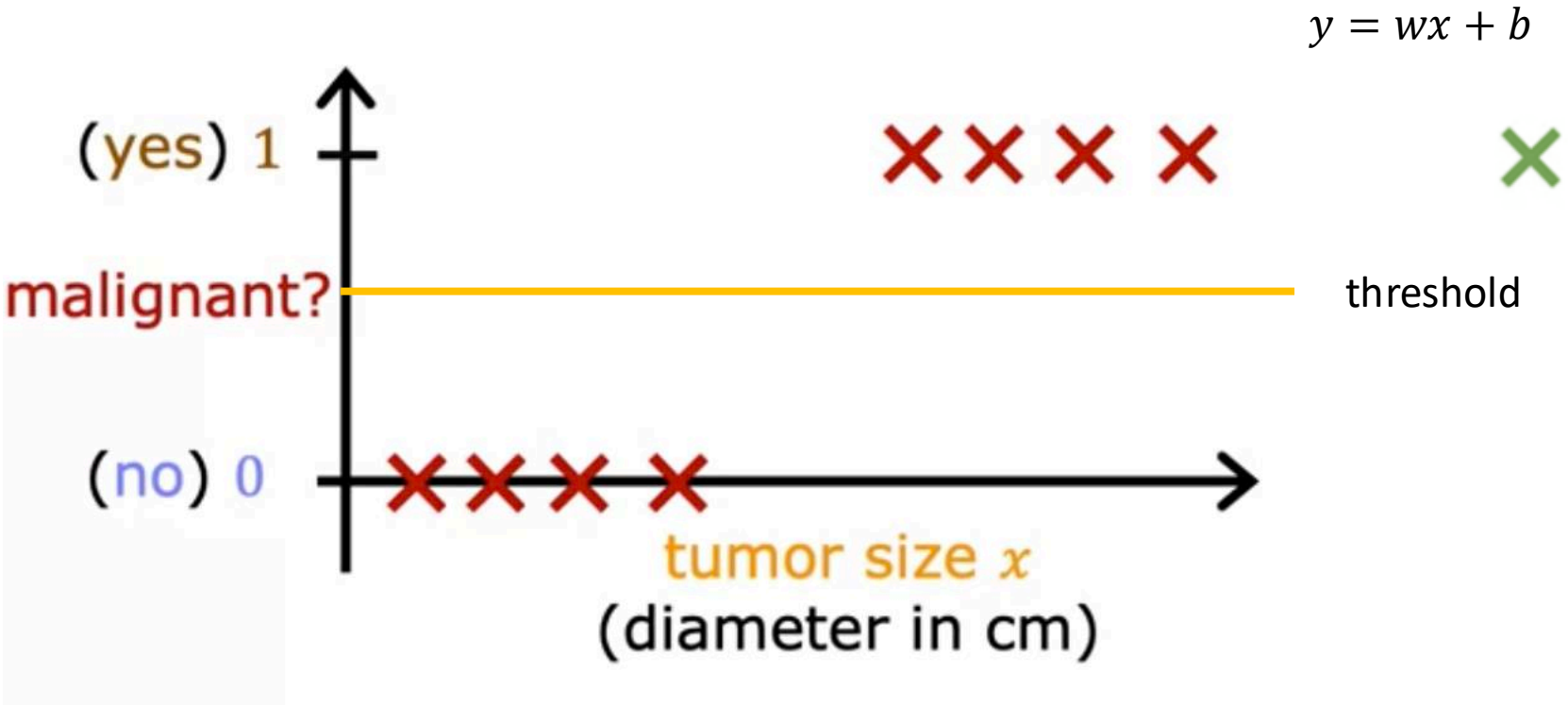
Machine Learning: *Classification*

Decision Boundary



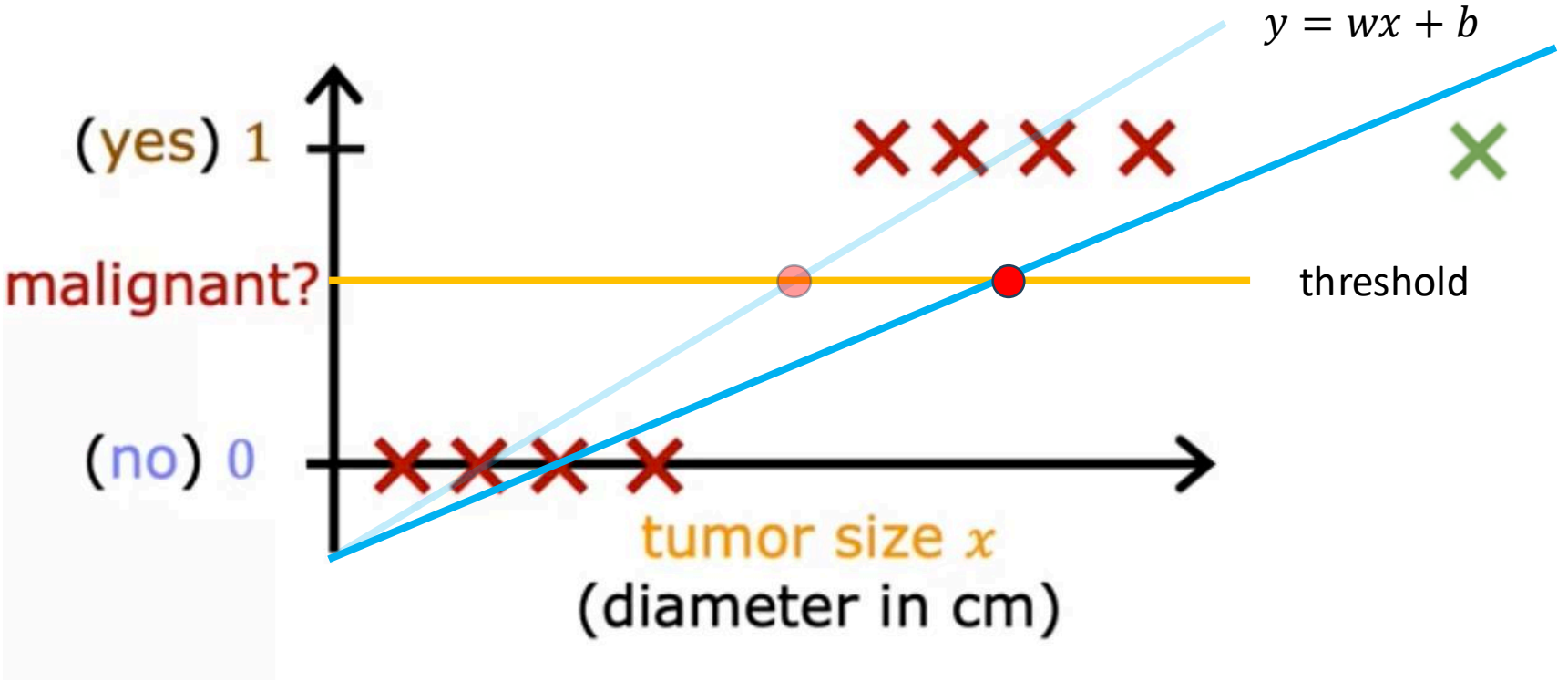
Machine Learning: *Classification*

Classification with Linear Regression



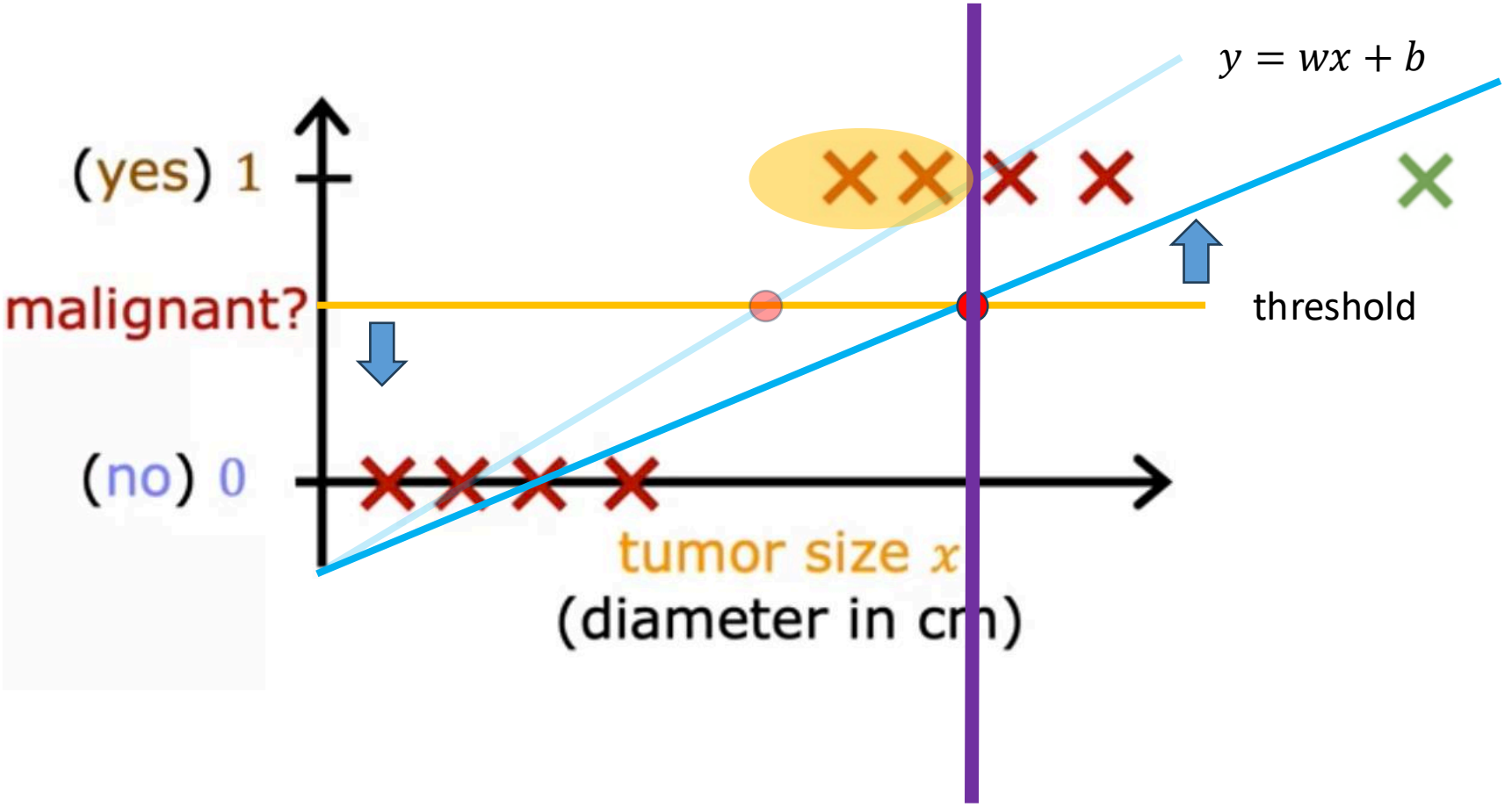
Machine Learning: *Classification*

Classification with Linear Regression



Machine Learning: *Classification*

Classification with Linear Regression

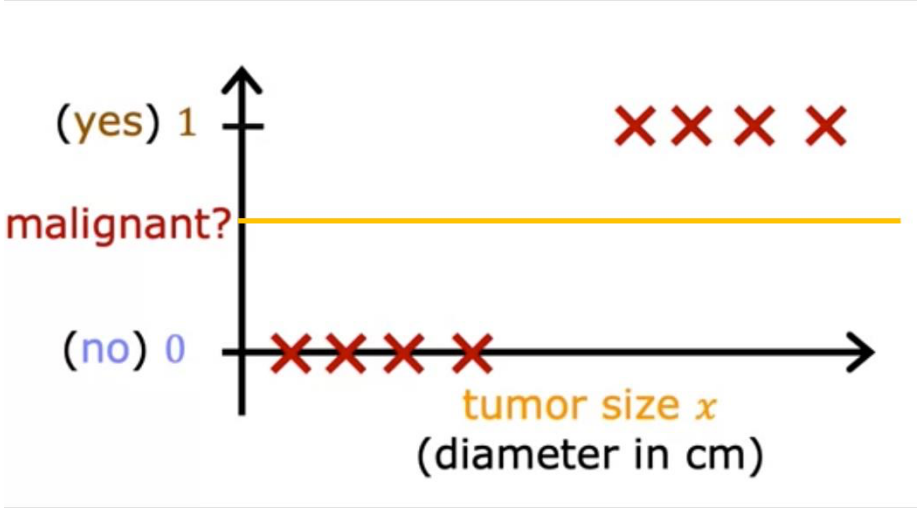


Machine Learning: *Classification*

Classification as a probability of event

We want to predict the probability that something is class 1: $P(y = 1)$.

Probability lives between 0 and 1, which is awkward to model directly with a linear function (which ranges from $-\infty$ to $+\infty$).



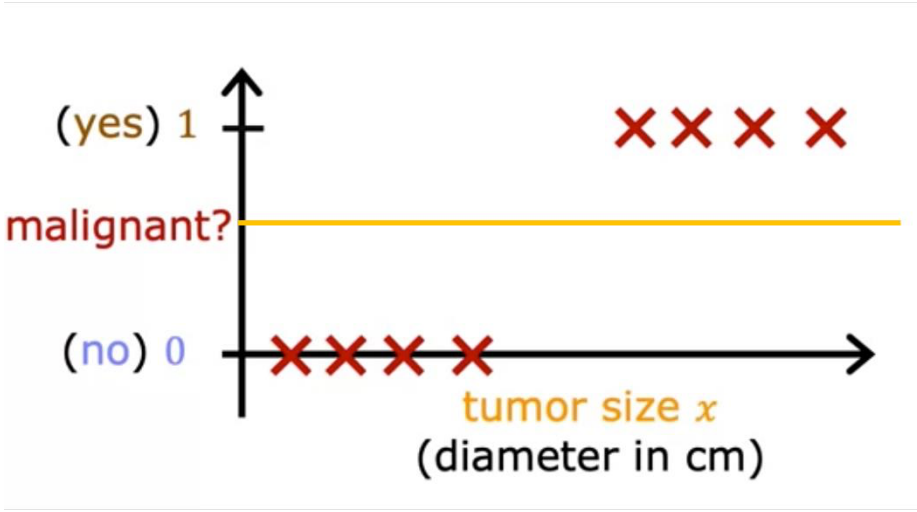
$$f_{\vec{w},b}(\vec{x}) = P(y = 1|\vec{x}; \vec{w}, b)$$

Machine Learning: *Classification*

Classification as a probability of event

To escape the 0–1 constraint, convert probability to **odds**:

$$\text{odds} = \frac{P(y=1)}{P(y=0)} = \frac{P(y=1)}{1-P(y=1)}$$



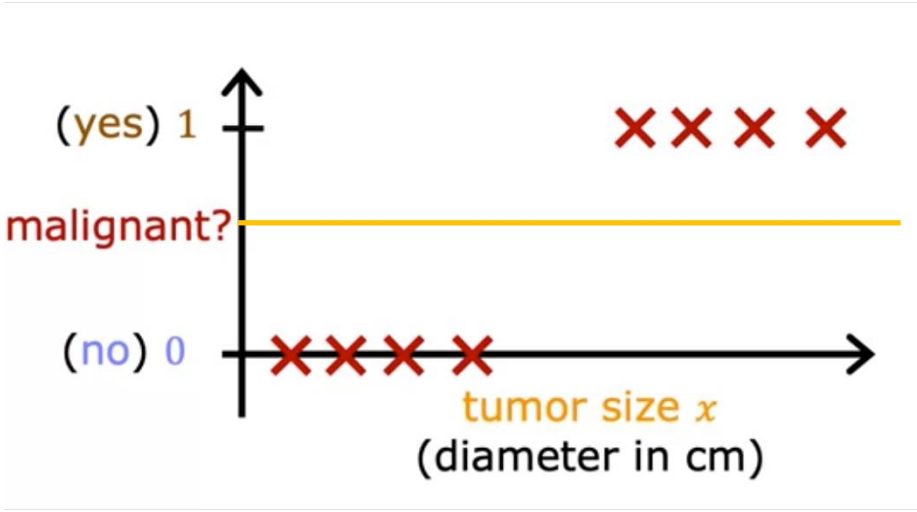
If P = 0.5 → odds = 1 (equally likely)
If P = 0.9 → odds = 9 (9x more likely to be class 1)

Machine Learning: *Classification*

Classification as a probability of event

To escape the 0–1 constraint, convert probability to **odds**:

$$\text{odds} = \frac{P(y=1)}{P(y=0)} = \frac{P(y=1)}{1-P(y=1)}$$



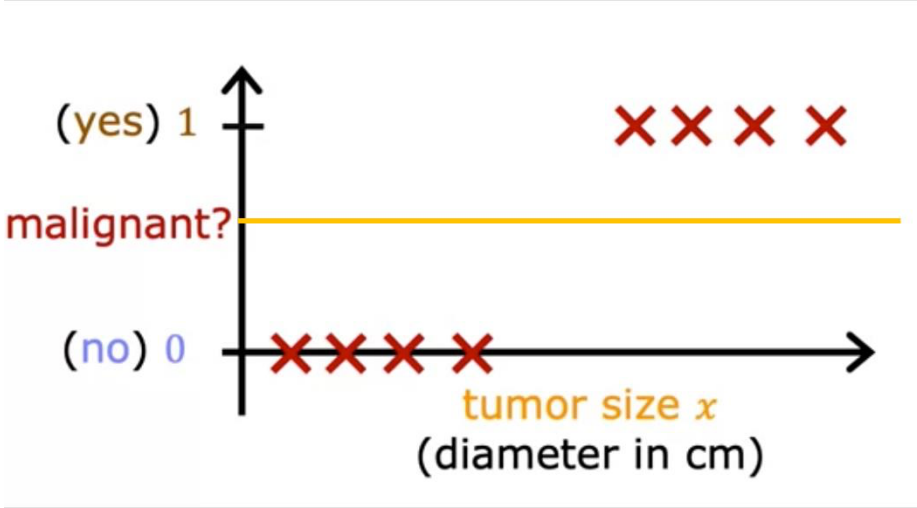
Odds live in $(0, +\infty)$…. still not symmetric around zero.

Machine Learning: *Classification*

Classification as a probability of event

Log-Odds (Logit)

$$\text{Log-odds} = \log\left(\frac{P(y=1)}{1-P(y=1)}\right)$$



Now it lives in $(-\infty, +\infty)$ — **same range as a linear function**. This is also called the **logit function**.

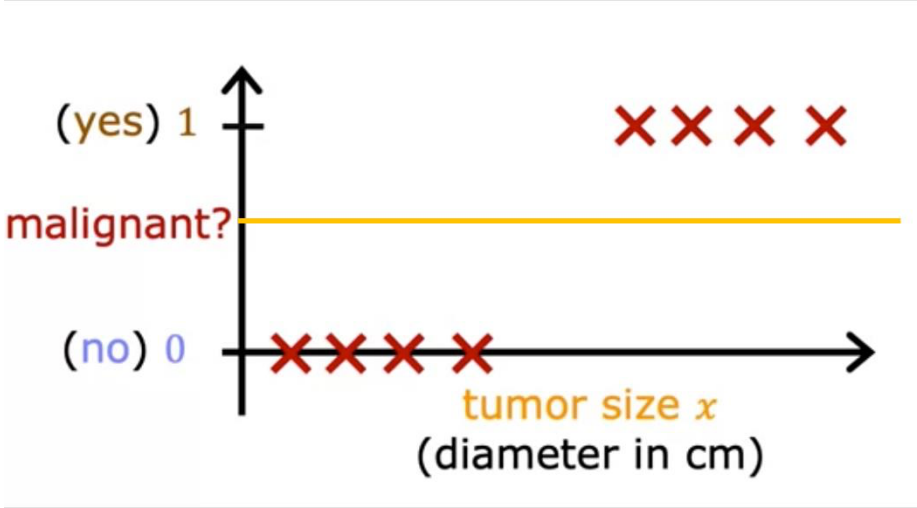
Machine Learning: *Classification*

Classification as a probability of event

Log-Odds (Logit)

$$\text{Log-odds} = \log\left(\frac{P(y=1)}{1-P(y=1)}\right)$$

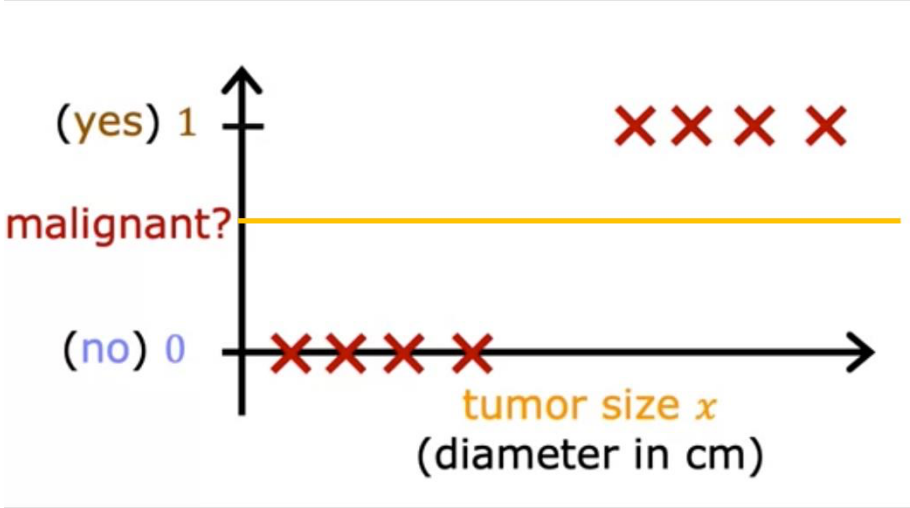
P = 0.5 → log-odds = 0
P = 0.9 → log-odds = 2.2
P = 0.1 → log-odds = −2.2



Machine Learning: *Classification*

Classification as a probability of event

We assumed the log-odds is **linear** in the features:



$$\mathrm{P} = \mathrm{P}(\mathrm{y}{=}1)$$

$$\log\left(\frac{P}{1-P}\right) = \vec{w}.\vec{x} + b$$

Solve for P

Machine Learning: *Classification*

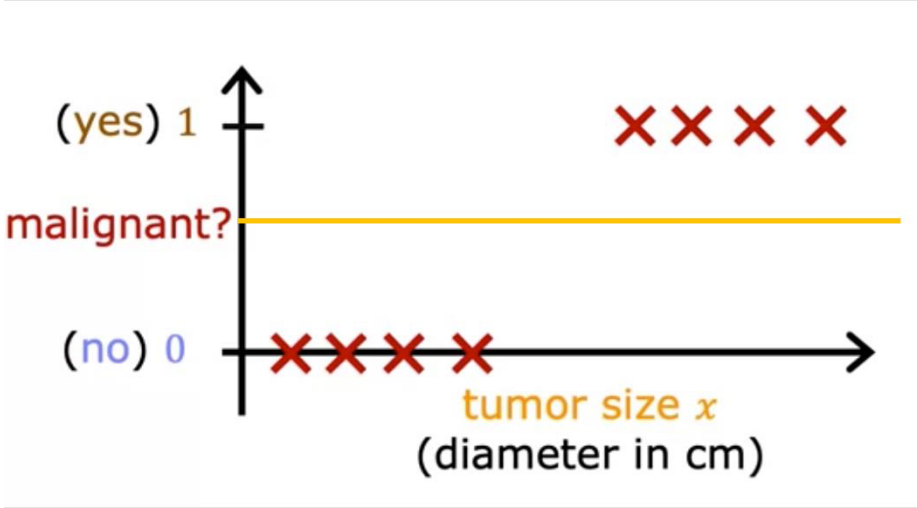
Classification as a probability of event

We assumed the log-odds is **linear** in the features:

$$\log\left(\frac{P}{1-P}\right) = \vec{w}.\vec{x} + b$$

$$\frac{P}{1-P} = e^{\vec{w}.\vec{x}+b}$$

$$P = \frac{1}{1 + e^{-(\vec{w}.\vec{x}+b)}}$$



Machine Learning: *Classification*

Classification as a probability of event

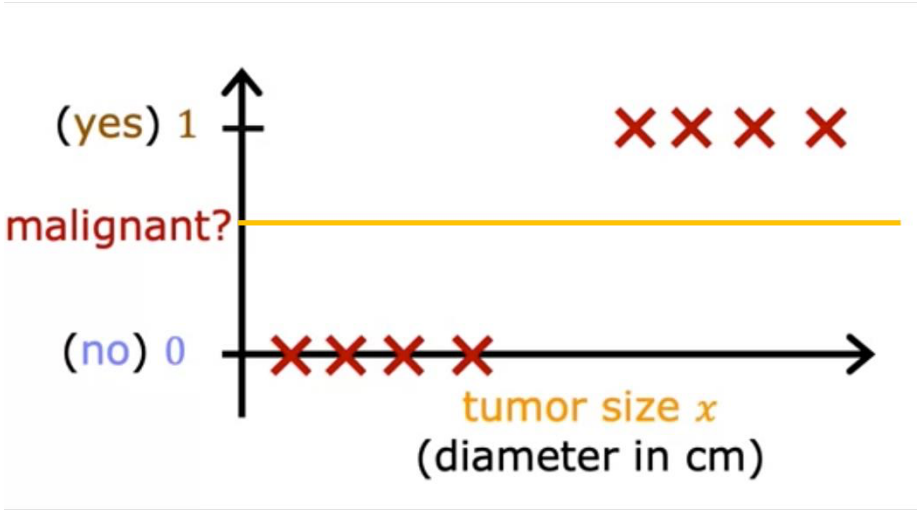
We assumed the log-odds is **linear** in the features:

$$\log\left(\frac{P}{1-P}\right) = \vec{w}.\vec{x} + b$$

$$\frac{P}{1-P} = e^{\vec{w}.\vec{x}+b}$$

$$P = \frac{1}{1 + e^{-(\vec{w}.\vec{x}+b)}}$$

Sigmoid Function

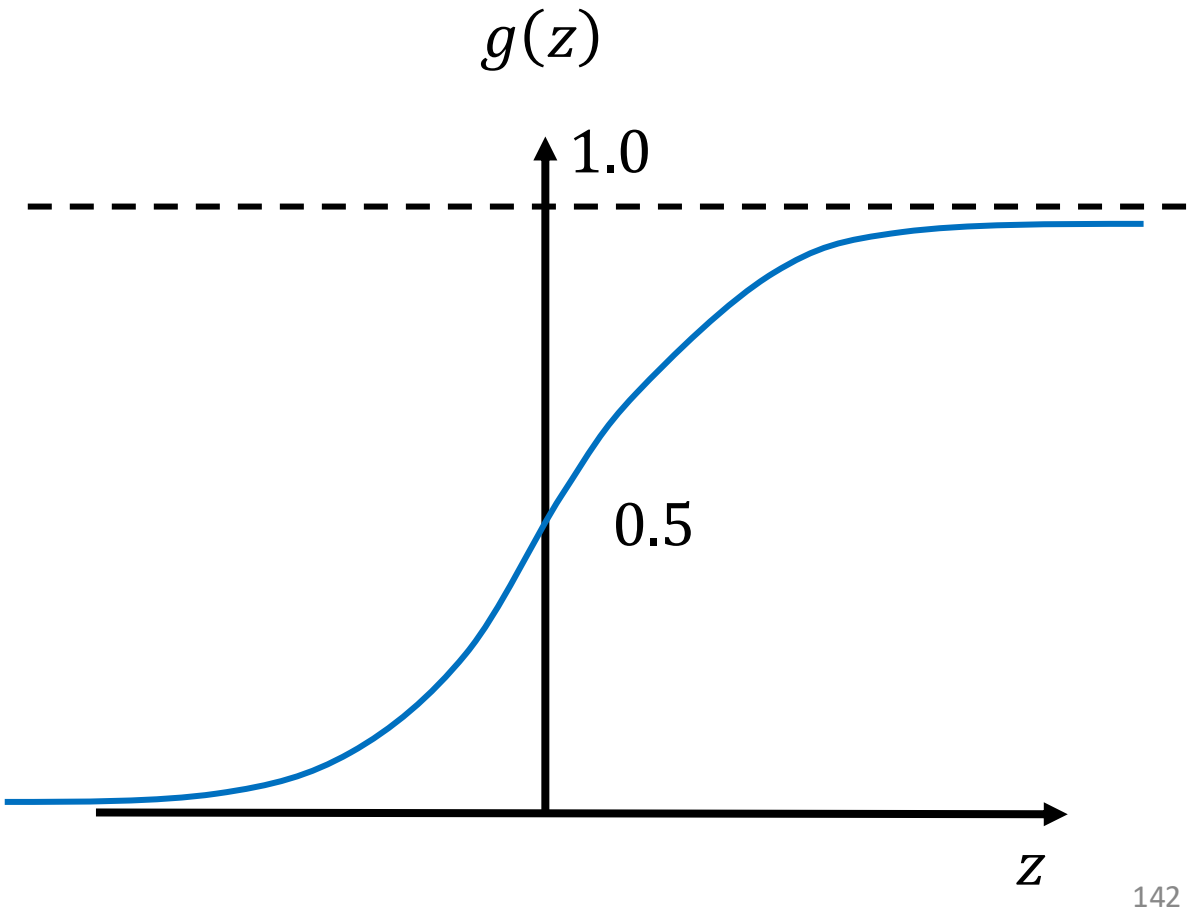


Machine Learning: *Logistic Regression*

Definition

$$g(z) = \frac{1}{1 + e^{-z}}, 0 < g(z) < 1$$

Sigmoid Function



Machine Learning: *Logistic Regression*

Formulation

$$f_{\vec{w},b}(\vec{x}) = g(z)$$

$$z = \vec{w}.\vec{x} + b$$

$$g(z) = \frac{1}{1+e^{-z}}, 0 < g(z) < 1$$

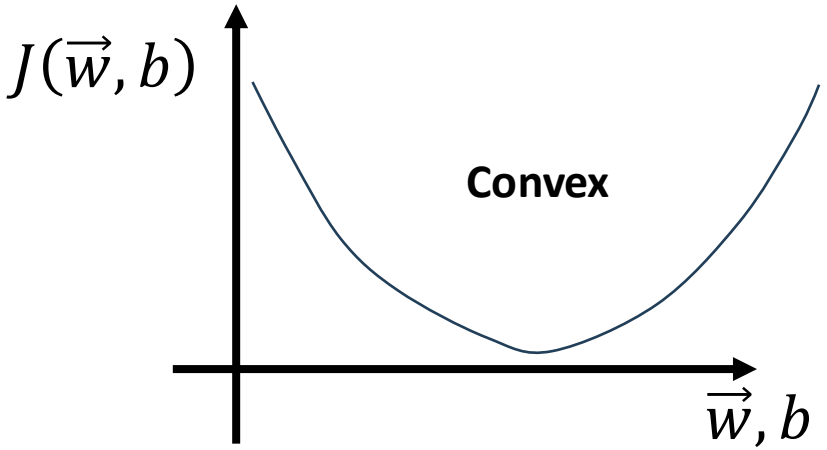
$$f_{\vec{w},b}(\vec{x}) = g(\overbrace{\vec{w}.\vec{x} + b}^{z}) = \frac{1}{1+e^{-(\vec{w}.\vec{x}+b)}}$$

Machine Learning: *Logistic Regression*

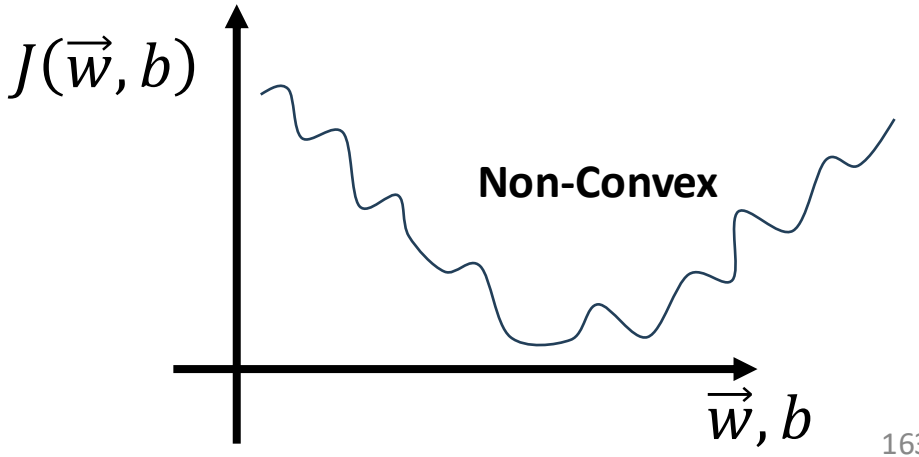
Cost Function

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^{M} \left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right) - y^{(i)} \right)^2$$

$$f_{\vec{w},b}(\vec{x}) = \vec{w}.\vec{x} + b$$



$$f_{\vec{w},b}(\vec{x}) = \frac{1}{1 + e^{-(\vec{w}.\vec{x}+b)}}$$



Machine Learning: *Logistic Regression*

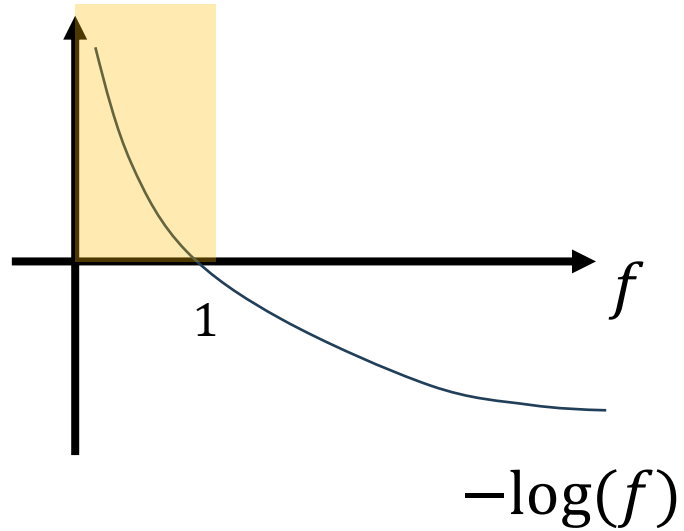
Cost Function

$$L\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right), y^{(i)}\right) = \begin{cases} -\log\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right) \text{ , if } y^{(i)} = 1 \\ -\log\left(1 - f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right), \text{ if } y^{(i)} = 0 \end{cases}$$

Machine Learning: *Logistic Regression*

Cost Function

$$L\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right), y^{(i)}\right) = \begin{cases} -\log\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right) \text{ , if } y^{(i)} = 1 \\ -\log\left(1 - f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right), \text{ if } y^{(i)} = 0 \end{cases}$$



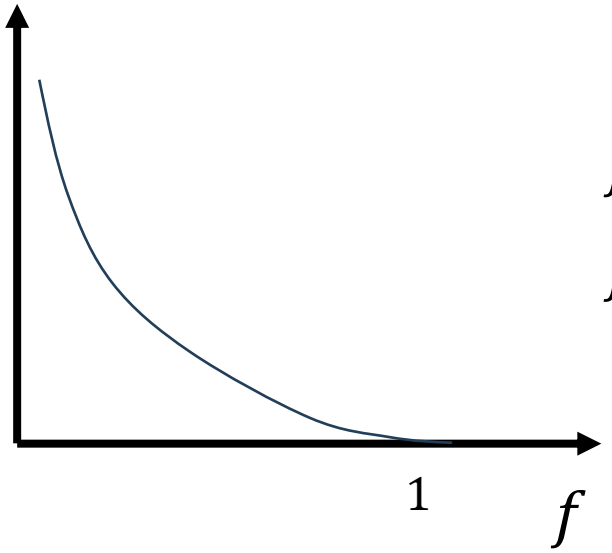
Machine Learning: *Logistic Regression*

Cost Function

$$L\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right), y^{(i)}\right) = \begin{cases} -\log\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right) \text{ , if } y^{(i)} = 1 \\ -\log\left(1 - f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right), \text{ if } y^{(i)} = 0 \end{cases}$$

$L\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right), y^{(i)}\right)$

if $y^{(i)} = 1$



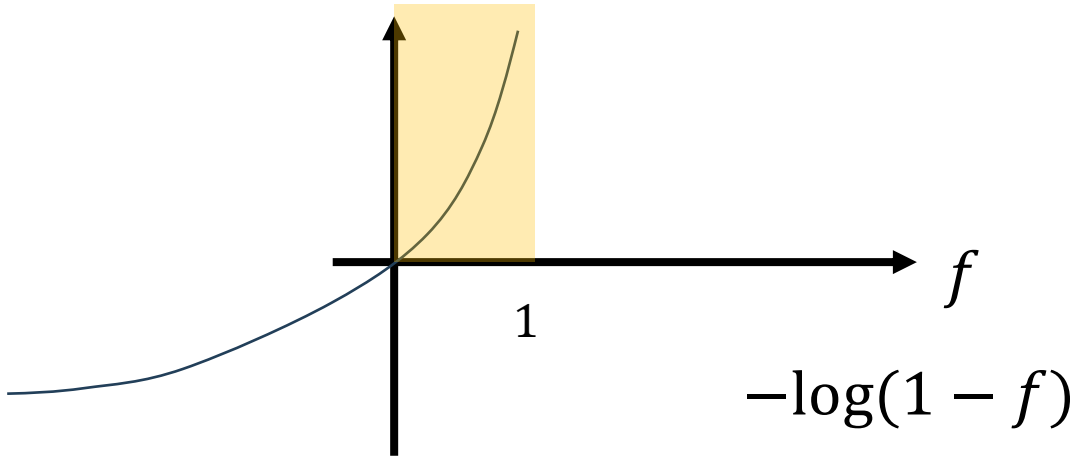
$f_{\vec{w},b}\left(\vec{x}^{(i)}\right) \to 1$ then $L\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right), y^{(i)}\right) \to 0$

$f_{\vec{w},b}\left(\vec{x}^{(i)}\right) \to 0$ then $L\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right), y^{(i)}\right) \to \infty$

Machine Learning: *Logistic Regression*

Cost Function

$$L\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right), y^{(i)}\right) = \begin{cases} -\log\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right), \text{ if } y^{(i)} = 1 \\ -\log\left(1 - f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right), \text{ if } y^{(i)} = 0 \end{cases}$$



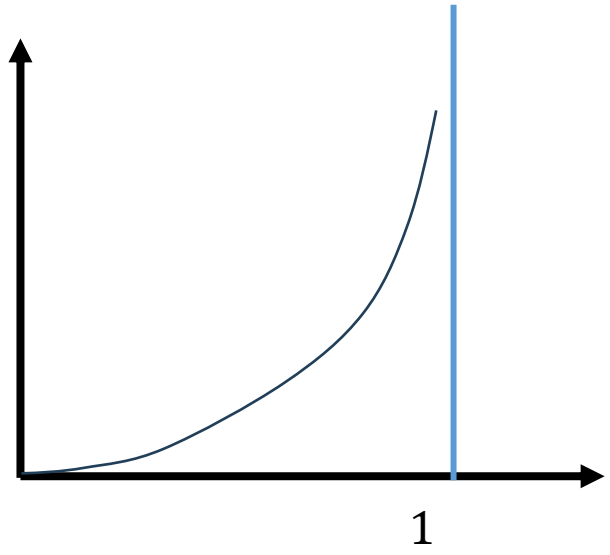
Machine Learning: *Logistic Regression*

Cost Function

$$L\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right), y^{(i)}\right) = \begin{cases} -\log\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right), \text{ if } y^{(i)} = 1 \\ -\log\left(1 - f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right), \text{ if } y^{(i)} = 0 \end{cases}$$

$L\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right), y^{(i)}\right)$

if $y^{(i)} = 0$



$f_{\vec{w},b}\left(\vec{x}^{(i)}\right) \to 1$ then $L\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right), y^{(i)}\right) \to \infty$

$f_{\vec{w},b}\left(\vec{x}^{(i)}\right) \to 0$ then $L\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right), y^{(i)}\right) \to 0$

Machine Learning: *Logistic Regression*

Cost Function

$$L\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right), y^{(i)}\right) = \begin{cases} -\log\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right) \text{ , if } y^{(i)} = 1 \\ -\log\left(1 - f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right), \text{ if } y^{(i)} = 0 \end{cases}$$

$$J(\vec{w}, b) = \frac{1}{m}\sum_{i=1}^{m} L\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right), y^{(i)}\right)$$

Machine Learning: *Logistic Regression*

Cost Function: *Simplified*

$$L\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right), y^{(i)}\right) = \begin{cases} -\log\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right) \text{ , if } y^{(i)} = 1 \\ -\log\left(1 - f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right), \text{ if } y^{(i)} = 0 \end{cases}$$

$$L\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right), y^{(i)}\right) = -y^{(i)} \log\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right) - \left(1 - y^{(i)}\right) \log\left(1 - f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right)$$

Machine Learning: *Logistic Regression*

Cost Function: *Simplified*

$$L\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right), y^{(i)}\right) = -y^{(i)} \log\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right) - (1 - y^{(i)}) \log\left(1 - f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right)$$

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right) + \left(1 - y^{(i)}\right) \log\left(1 - f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right)\right]$$

Machine Learning: *Logistic Regression*

Gradient Descent

$$f_{\vec{w},b}(\vec{x}) = \frac{1}{1 + e^{-(\vec{w}.\,\vec{x}+b)}}$$

Repeat Until Convergence:

{

$$w_j = w_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(x^{(i)}) - y^{(i)})$$

}

Simultaneously update $\vec{w}, b$

Machine Learning: *Logistic Regression*

Gradient Descent

Repeat Until Convergence:

{

$$w_j = w_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(x$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(x^{(i)}$$

}

Simultaneously update $\vec{w}$

1

Machine Learning: *General Form Linear Regression*

Gradient Descent

$n = 1$

Repeat Until Convergence:

{

$$w = w - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{w,b}(x^{(i)}) - y^{(i)})$$

}

Simultaneously update w, b

$n \geq 2$

Repeat Until Convergence:

{

$$w_j = w_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(x^{(i)}) - y^{(i)}) x_j^{(}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(x^{(i)}) - y^{(i)})$$

}

Simultaneously update $\vec{w}\ (w_1, , w_2, \ldots, w_n), b$

Machine Learning: *Logistic Regression*



Machine Learning: *Logistic Regression*

Logistic Regression

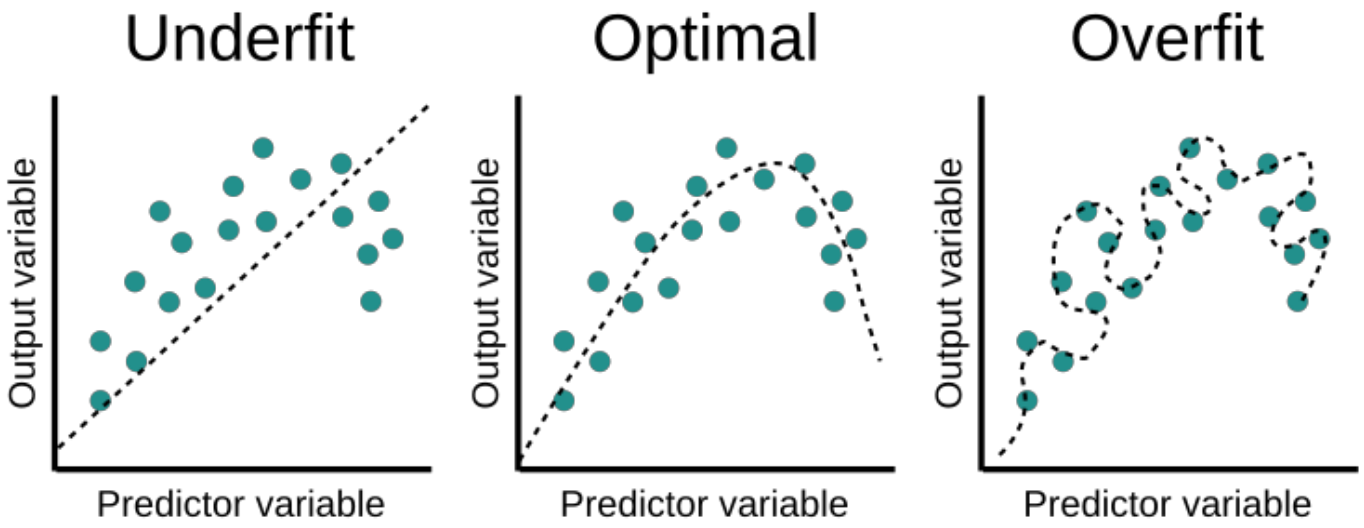
I put that on everything™



Machine Learning: *Regression*

Overfitting

Definition:
A model that is tailored too closely to the specificities of the training data, to the point where it performs poorly on new, unseen data.



https://www.fastaireference.com/overfitting

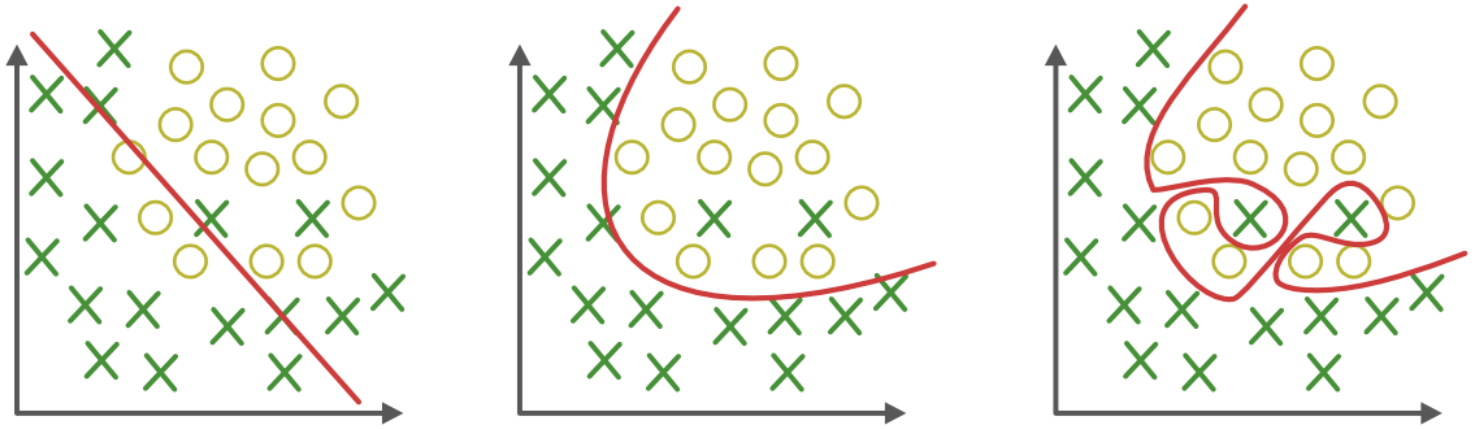
High-Bias

High-Variance

Machine Learning: *Classification*

Overfitting

Definition:
A model that is tailored too closely to the specificities of the training data, to the point where it performs poorly on new, unseen data.

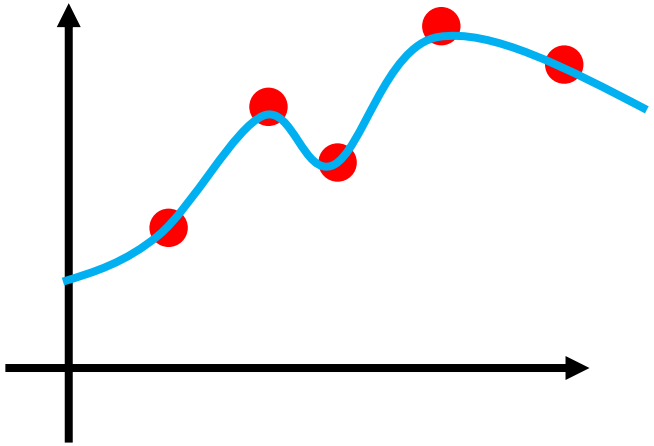


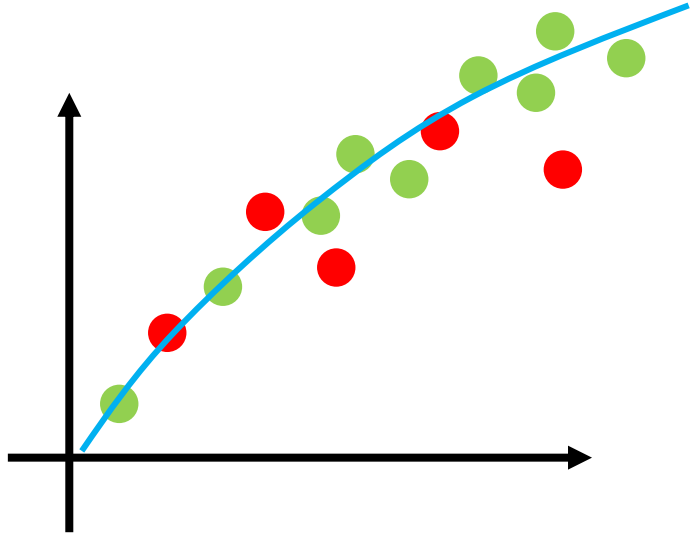
https://www.fastaireference.com/overfitting

Machine Learning: *Overfitting*

Addressing Underfitting/Overfitting

Collect more training samples





Machine Learning: *Overfitting*

Addressing Underfitting/Overfitting

Feature Selection (include/exclude)

ALL features + insufficient data  Selected features + some data

Machine Learning: *Overfitting*

Addressing Underfitting/Overfitting

Regularization

Adding a penalty to the complexity of the model to encourage simpler models that generalize better

$$J(\vec{w}, b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right) - y^{(i)}\right)^2 + Penalty(w, b)$$

Machine Learning: *Overfitting*

Addressing Underfitting/Overfitting

Regularization

Adding a penalty to the complexity of the model to encourage simpler models that generalize better

$$J(\vec{w}, b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right) - y^{(i)}\right)^2 + \frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2 + \frac{\lambda}{2m} b^2 \qquad \lambda > 0$$

Machine Learning: *Overfitting*

Addressing Underfitting/Overfitting

Regularization

Regularization term

$$\min_{w,b} J(\vec{w}, b) = \min_{w,b} [\frac{1}{2m} \sum_{i=1}^{m} \left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right) - y^{(i)}\right)^2 + \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2 + \frac{\lambda}{2m} b^2]$$

Mean square error

Machine Learning: *Overfitting*

Addressing Underfitting/Overfitting

Regularization



$$\min_{w,b} J(\vec{w}, b) = \min_{w,b} [\frac{1}{2m} \sum_{i=1}^{m} \left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right) - y^{(i)}\right)^2 + \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2 + \frac{\lambda}{2m} b^2]$$

Mean square error

Machine Learning: *Overfitting*

Addressing Underfitting/Overfitting

Regularization

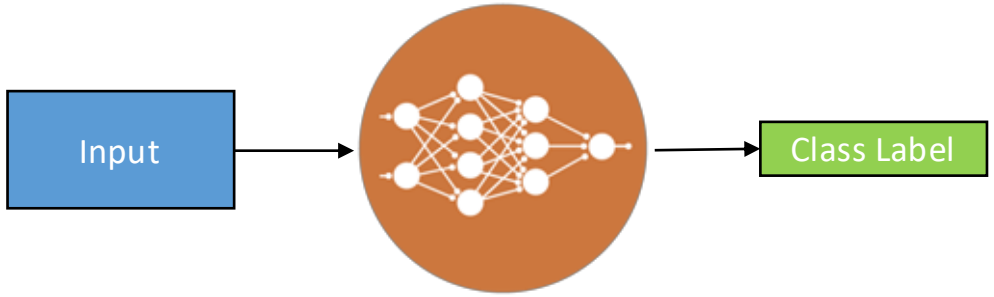
Types:
L1 Regularization (Lasso): Adds a penalty equal to the absolute value of the magnitude of coefficients. It can lead to some coefficients being zero, which implies feature selection.

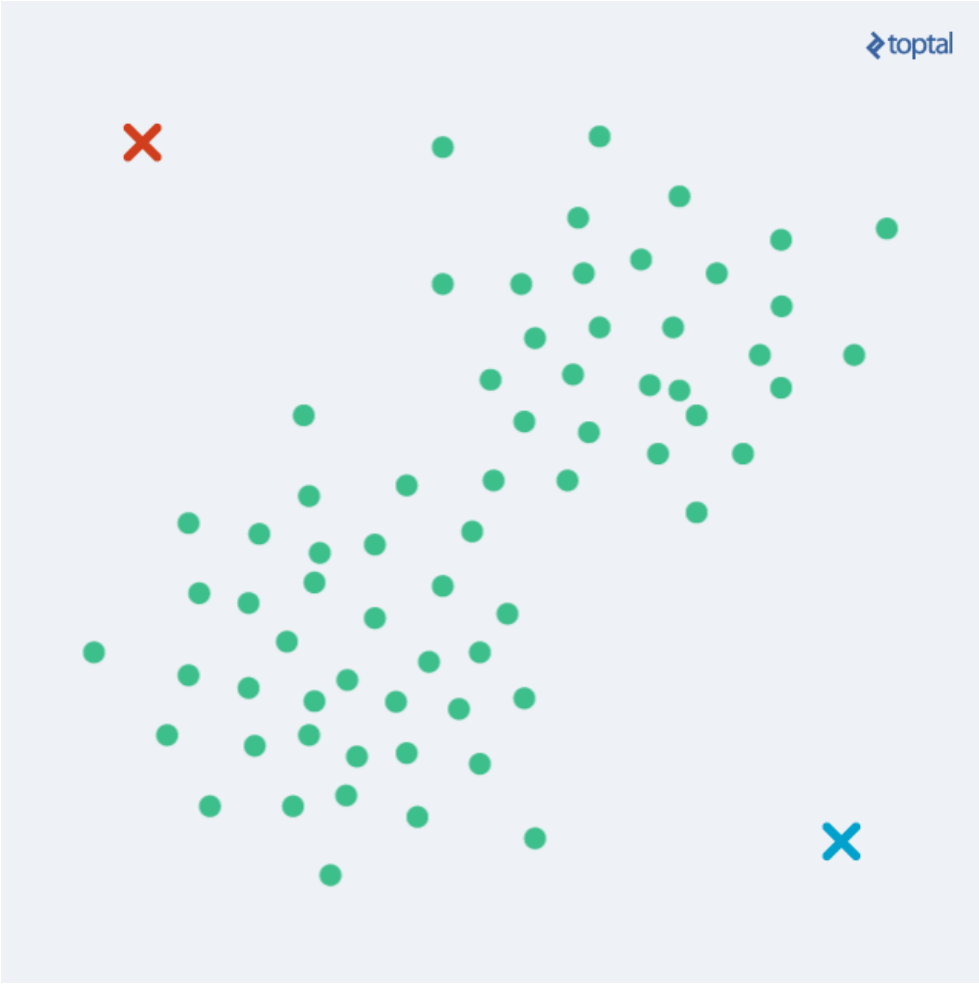
L2 Regularization (Ridge): Adds a penalty equal to the square of the magnitude of coefficients. This discourages large weights but typically does not result in zero coefficients.

Elastic Net: A combination of L1 and L2 regularization, it includes both penalties and is useful when there are multiple features that are correlated with each other.

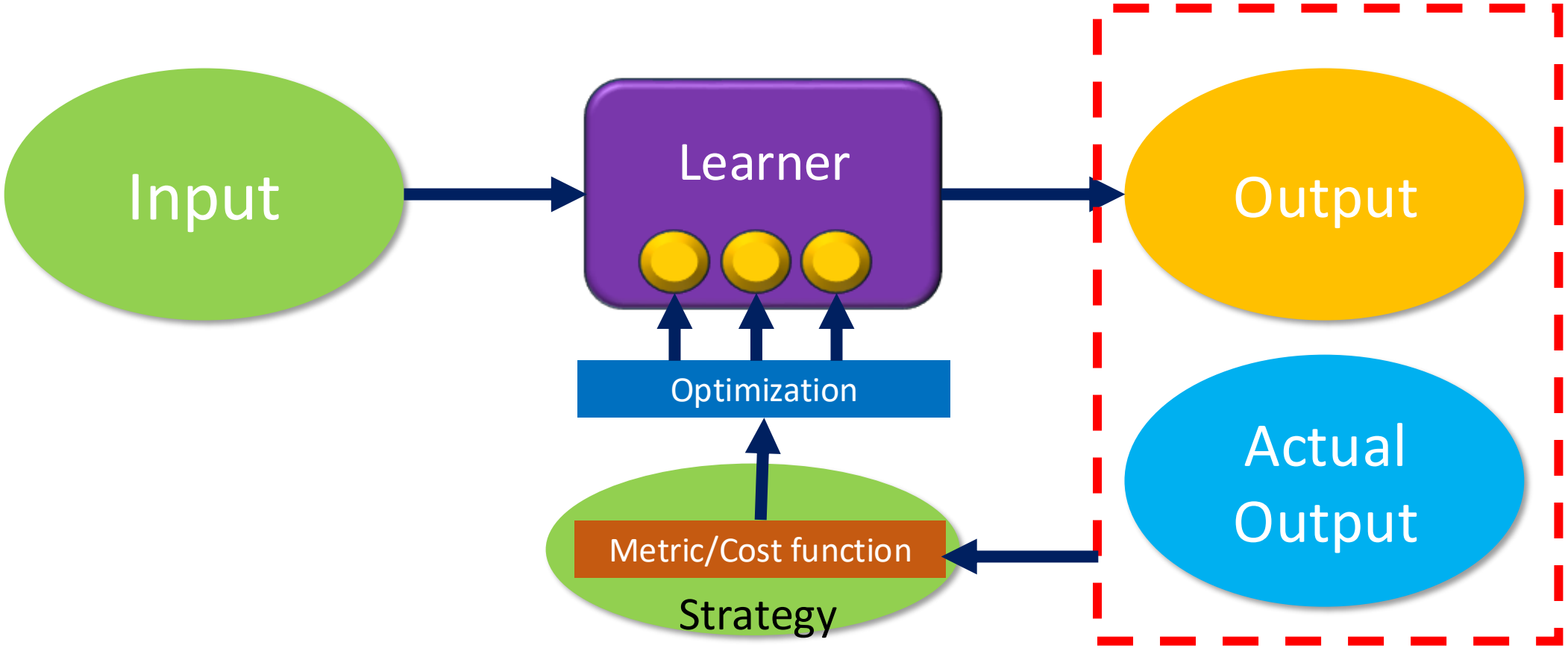
Machine Learning: *Categories (by Learning Paradigm)*

- Supervised Learning
- Unsupervised Learning
- Semi-supervised Learning
- Reinforcement Learning
- Self-supervised learning

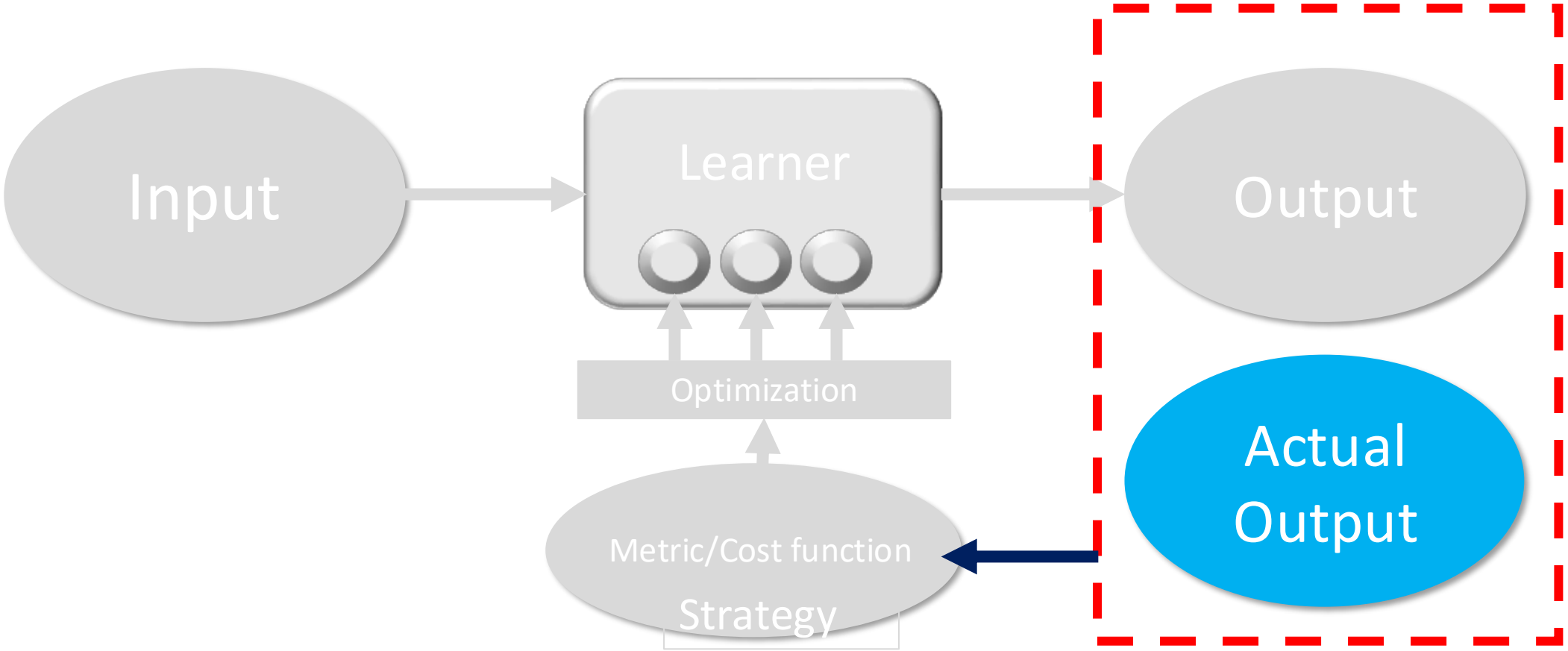




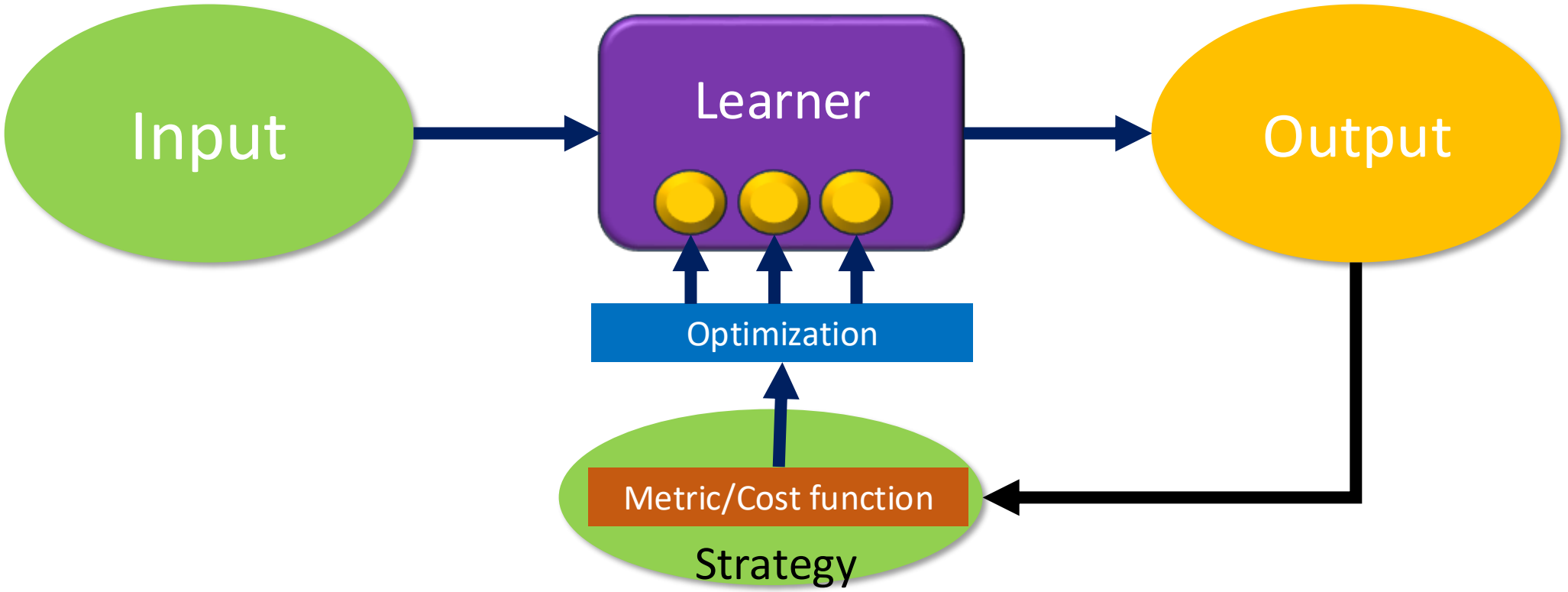
Machine Learning: *Supervised Learning*



Machine Learning: *Supervised Learning*



Machine Learning: *Unsupervised Learning*



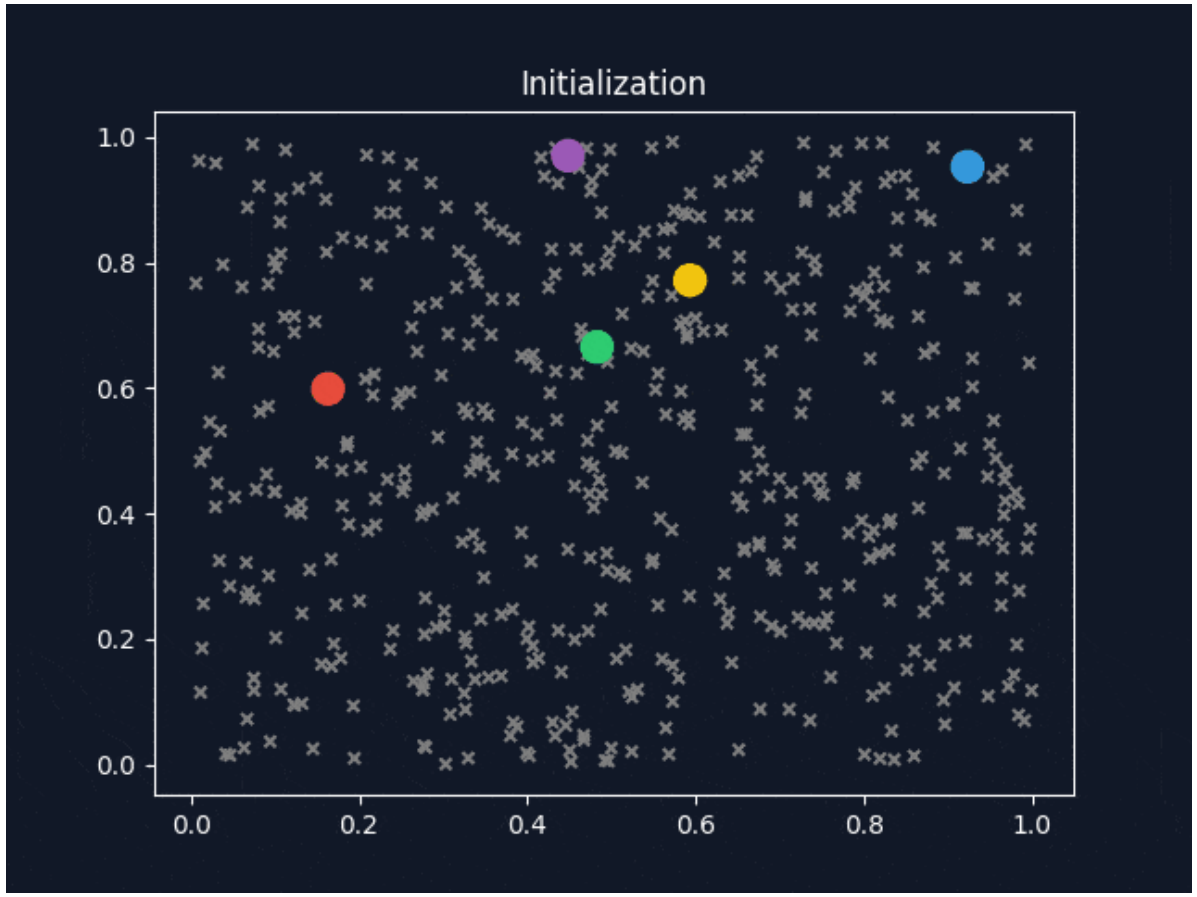
Machine Learning: *Unsupervised Learning*

Clustering

Definition: a type of unsupervised learning technique used to group data points into subsets or clusters.

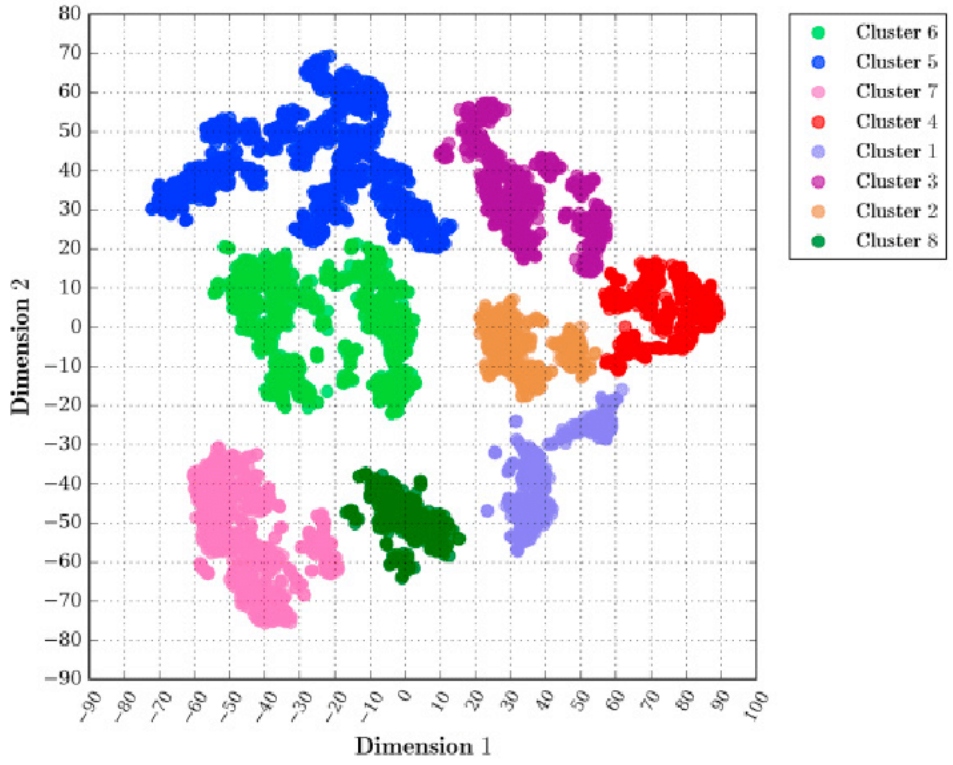
Goal: organize data in such a way that points in the same cluster are more similar to each other than to those in other clusters.

Metric: a similarity measure based on features or distance measures like Euclidean, Manhattan, or cosine similarity.



Machine Learning: *Unsupervised Learning*

Clustering: Example



"Data-Driven Identification of Hypertensive Patient Profiles for Patient Population Simulation"

First class (cluster #1 & #2): elderly females with high blood pressure and cardiavascular risk due to high prevalence of ischemic heart disease and dyslipidemia

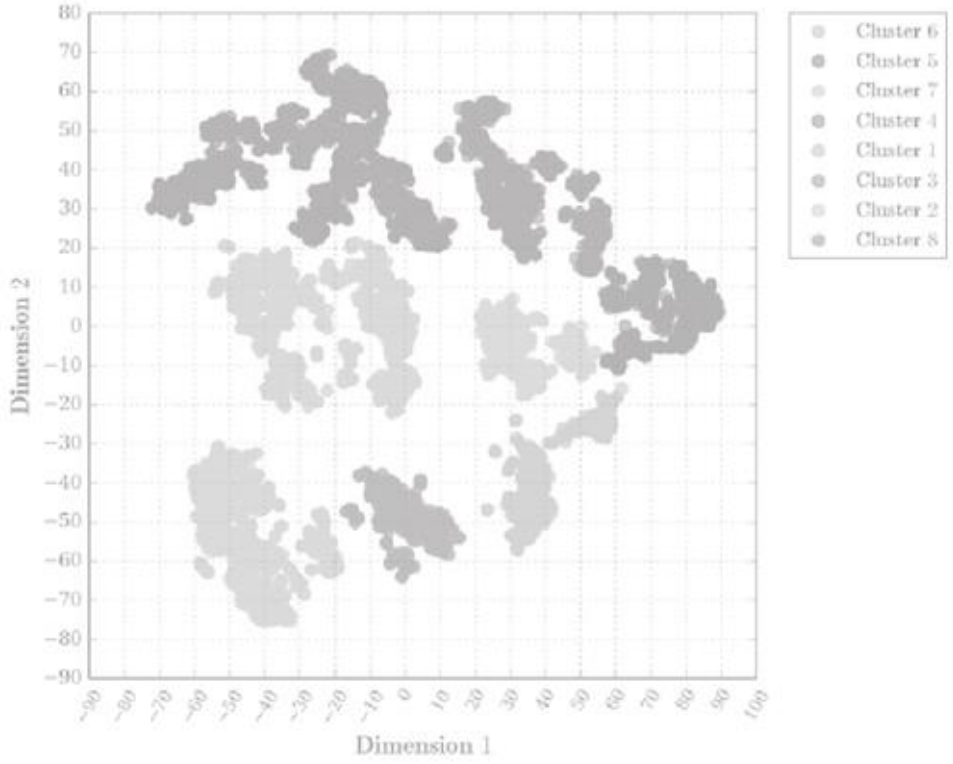
Second class (cluster #3 & #4): patients with severe AH (2 and 3 grades) and metabolic syndrome with a high
prevalence of carbohydrate metabolism disorder and lipid abnormalities

Third class (cluster #5): young males with family history, risk factors, and subclinical organ damage

Fourth class (cluster #6 & #7): is patients with grade 1 AH and advanced atherosclerosis

Machine Learning: *Unsupervised Learning*

Clustering: Example



"Data-Driven Identification of Hypertensive Patient Profiles for Patient Population Simulation"

"Clustering finds patterns in data—whether they are there or not."

First class (cluster #1 & #2): elderly females with high blood pressure and cardiavascular risk due to high prevalence of ischemic heart disease and dyslipidemia

Second class (cluster #3 & #4): patients with severe AH (2 and 3 grades) and metabolic syndrome with a high prevalence of carbohydrate metabolism disorder and lipid abnormalities

Third class (cluster #5): young males with family history, risk factors, and subclinical organ damage

Fourth class (cluster #6 & #7): is patients with grade 1 AH and advanced atherosclerosis

Machine Learning: *Clustering*

K-Mean

Definition: an unsupervised machine learning approach to partition a dataset into a set number of distinct, non-overlapping subgroups, or "clusters”.

Goal: group data points into clusters in such a way that similar items are together and dissimilar items are in different groups.



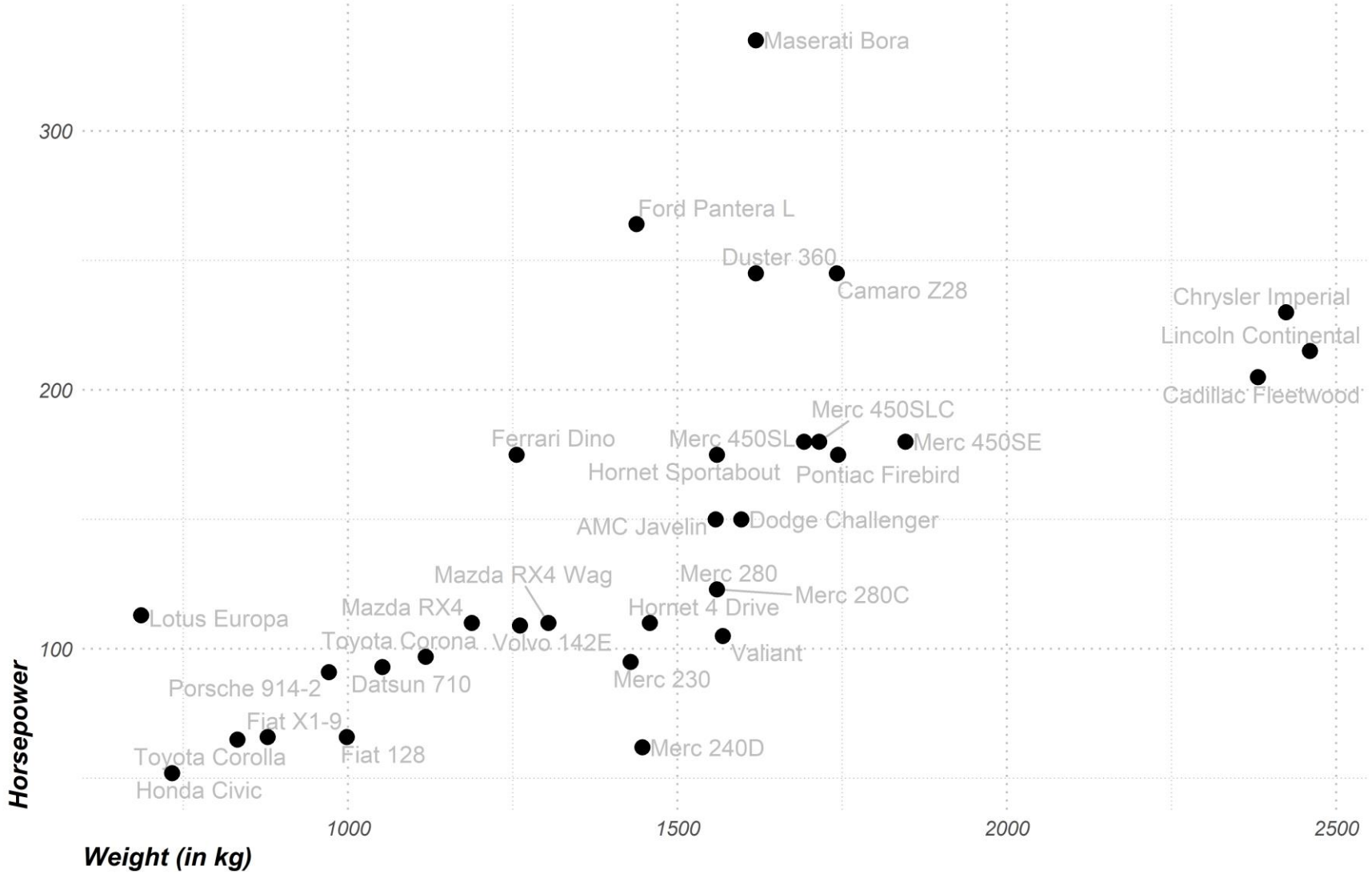
Machine Learning: *K-Mean*

Algorithm

1. Choose K
2. Randomly pick a point as cluster “Centroid”
3. Calculate Distance between every point and Centroids
4. Assign class
5. Calculate mean of each class
6. Move Centroid to mean location
7. Repeat 3-6 till centroid location doesn’t change

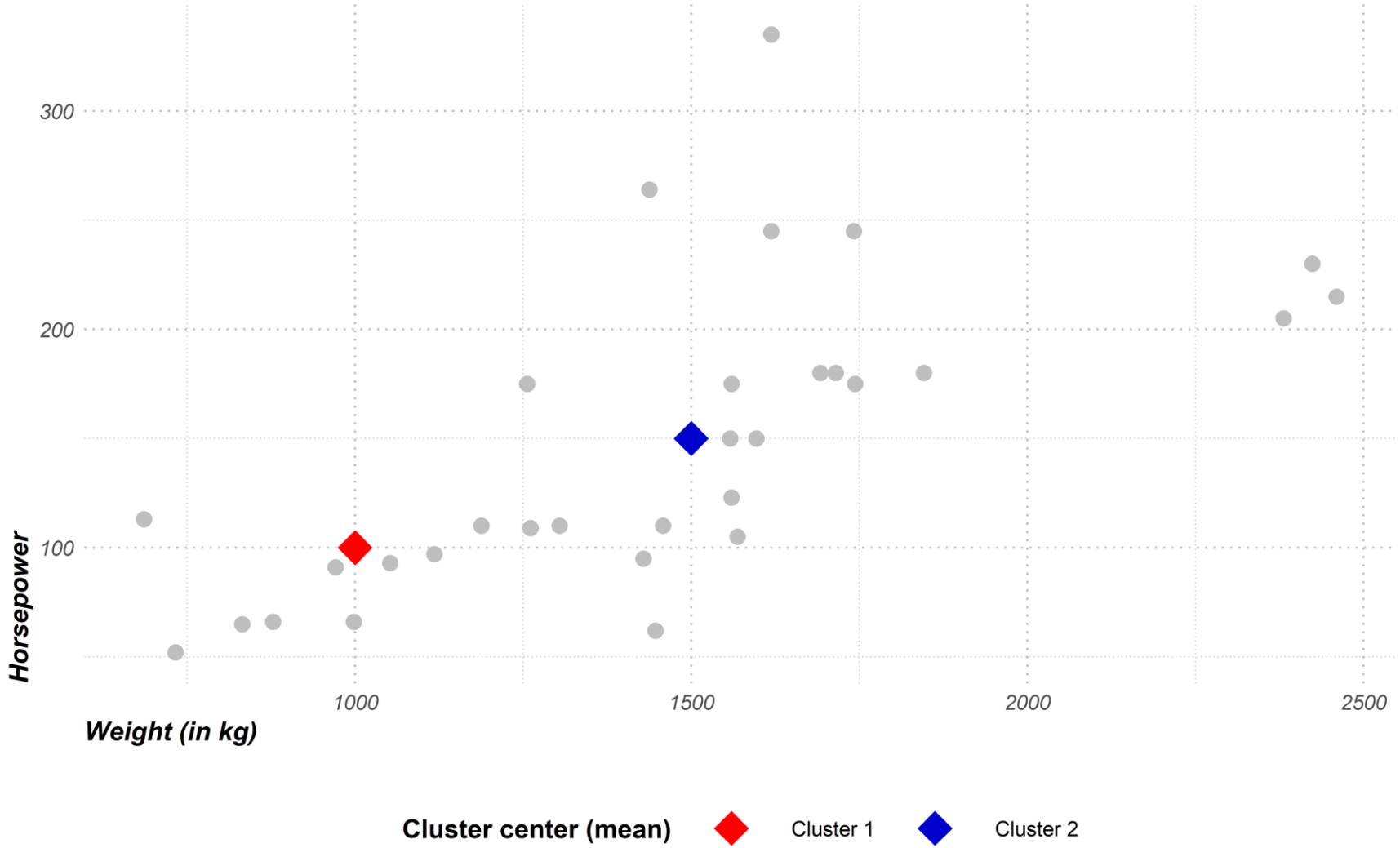
Machine Learning: *K-Mean*

Algorithm



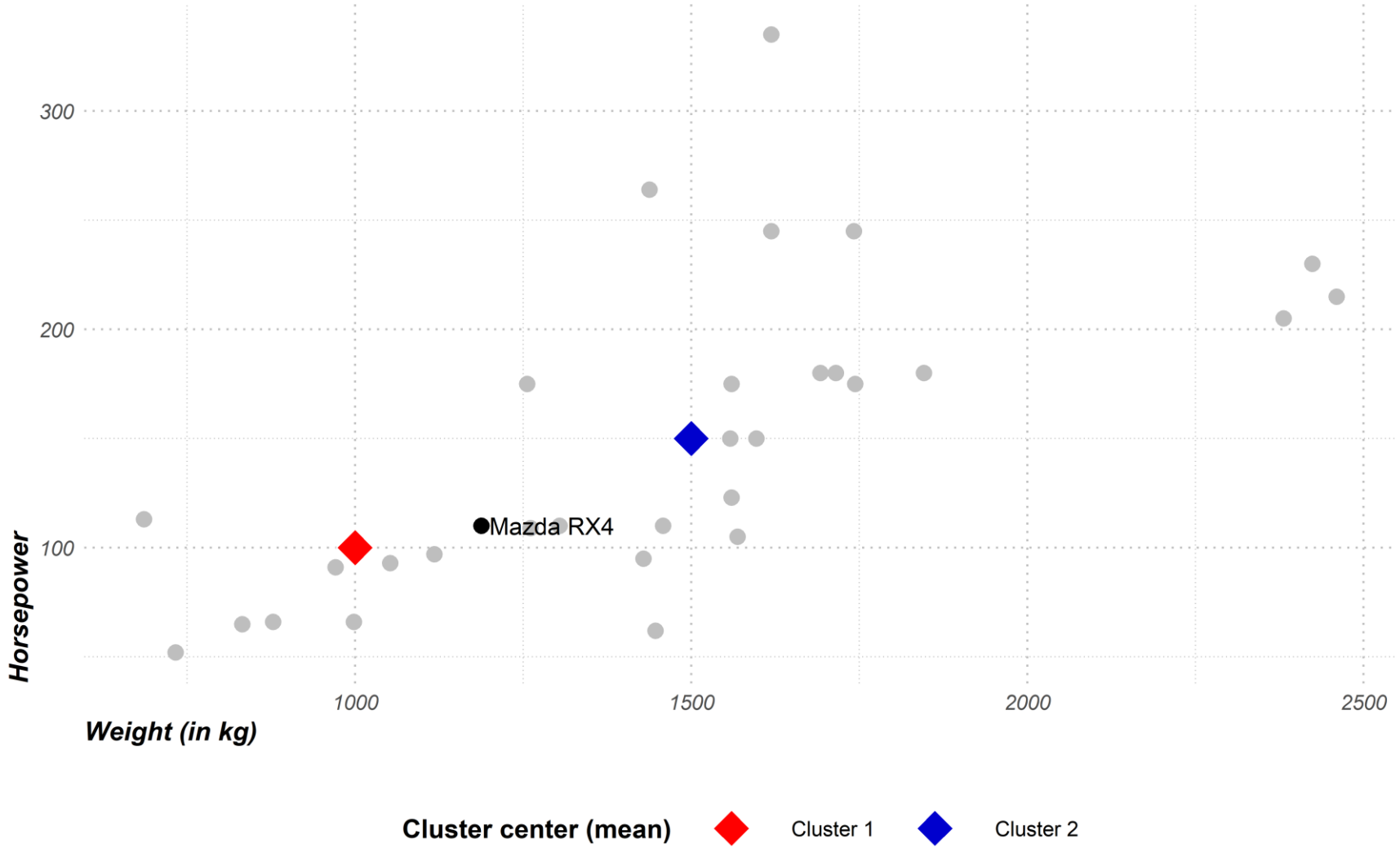
Machine Learning: *K-Mean*

Algorithm



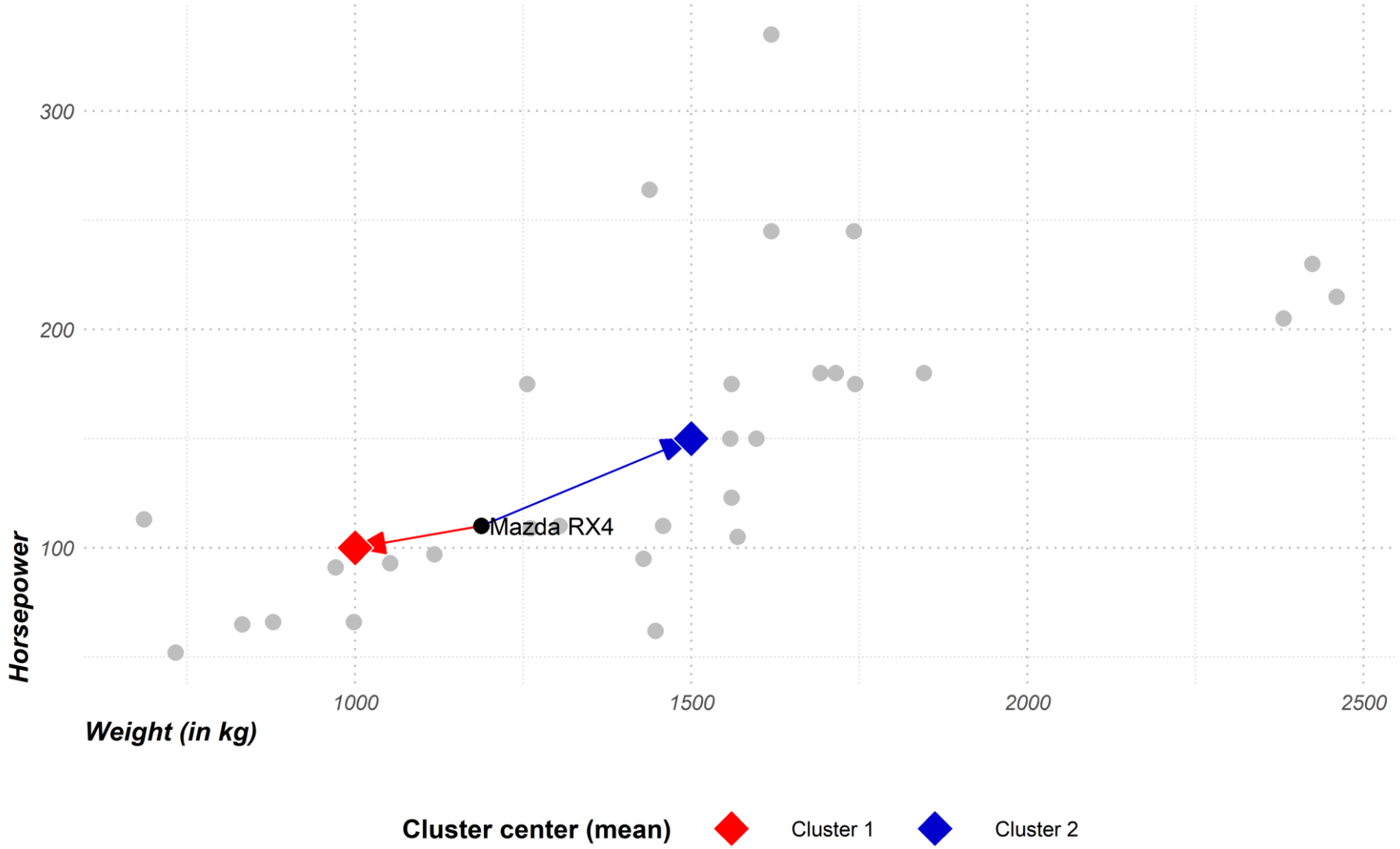
Machine Learning: *K-Mean*

Algorithm



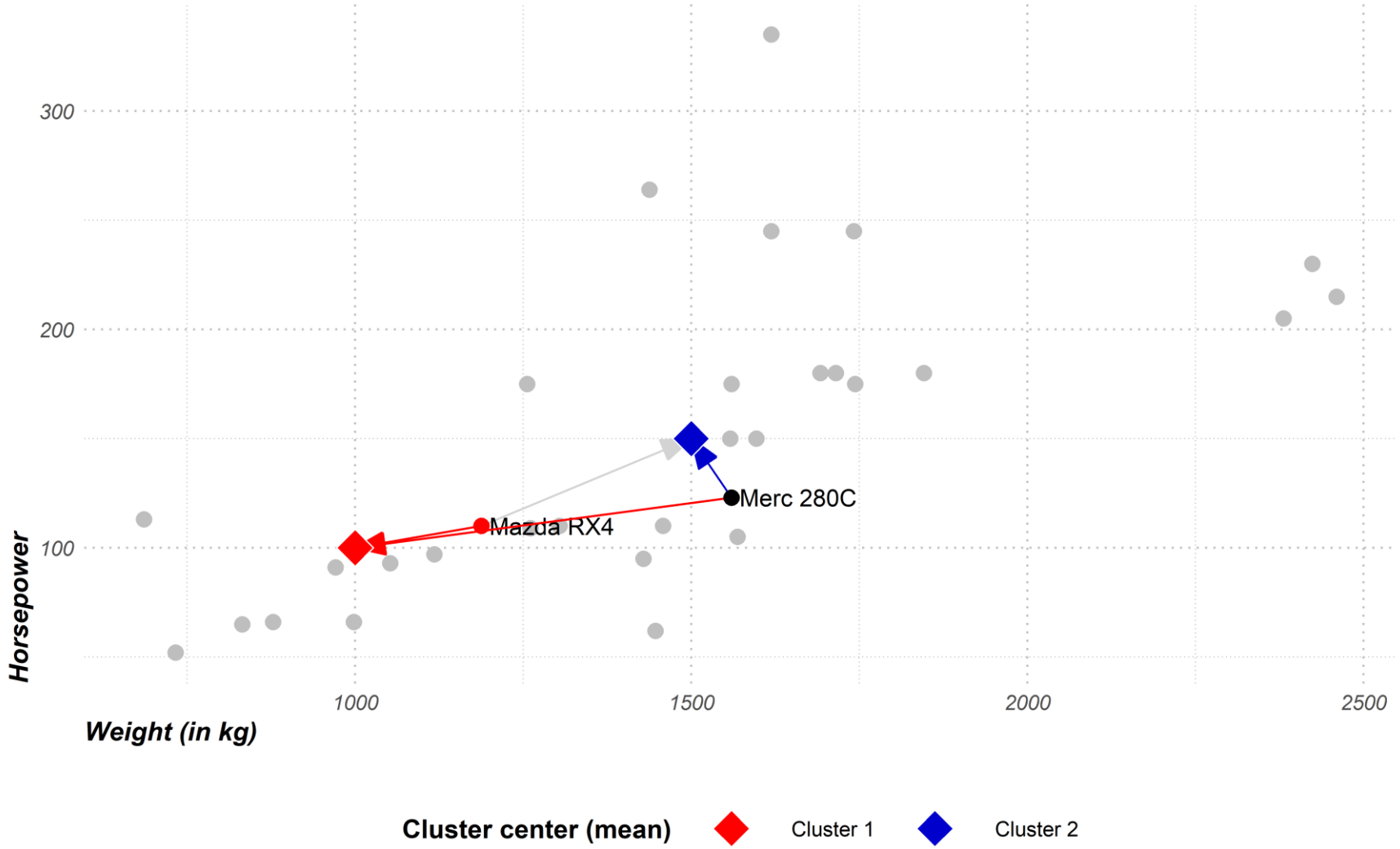
Machine Learning: *K-Mean*

Algorithm



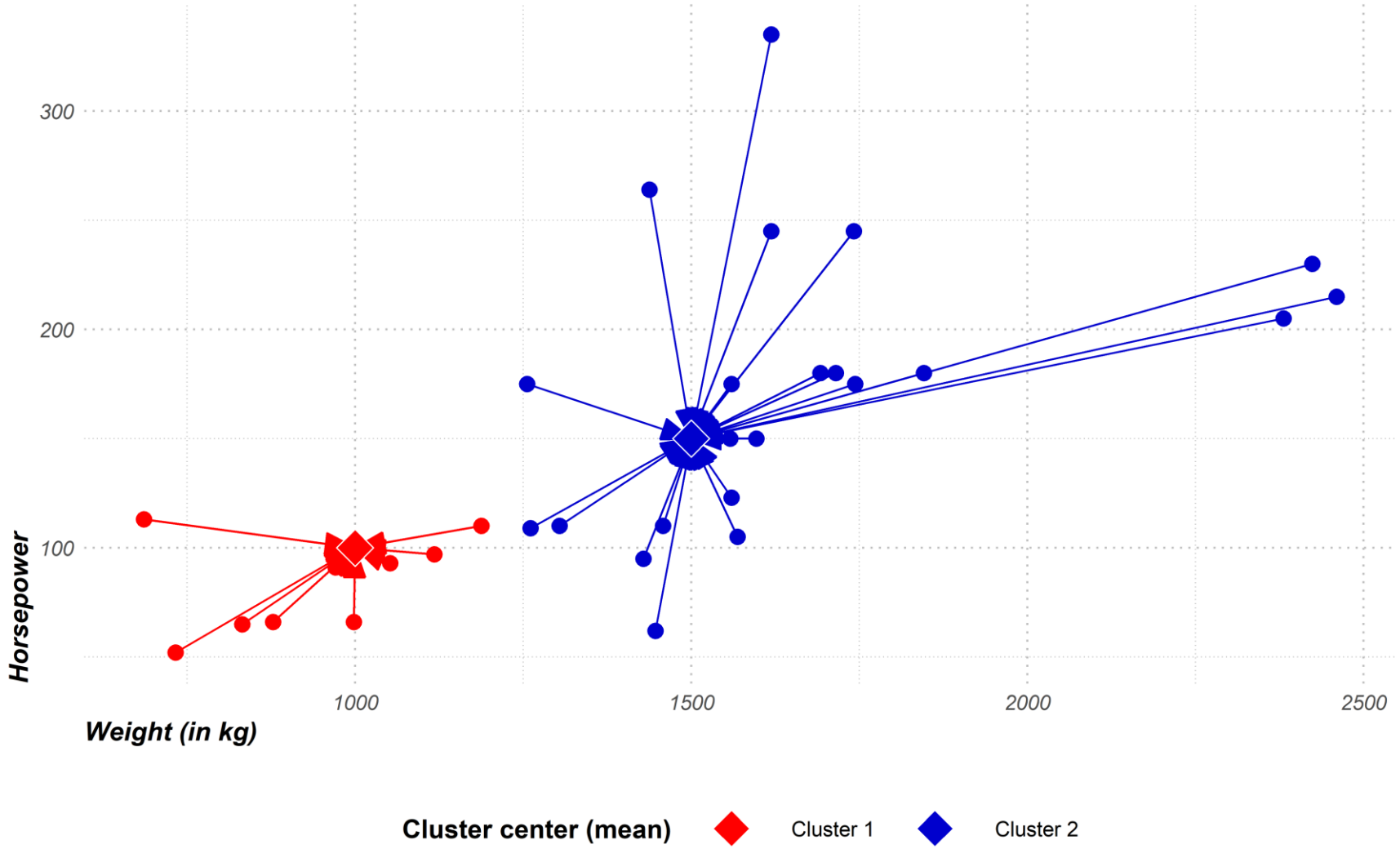
Machine Learning: *K-Mean*

Algorithm



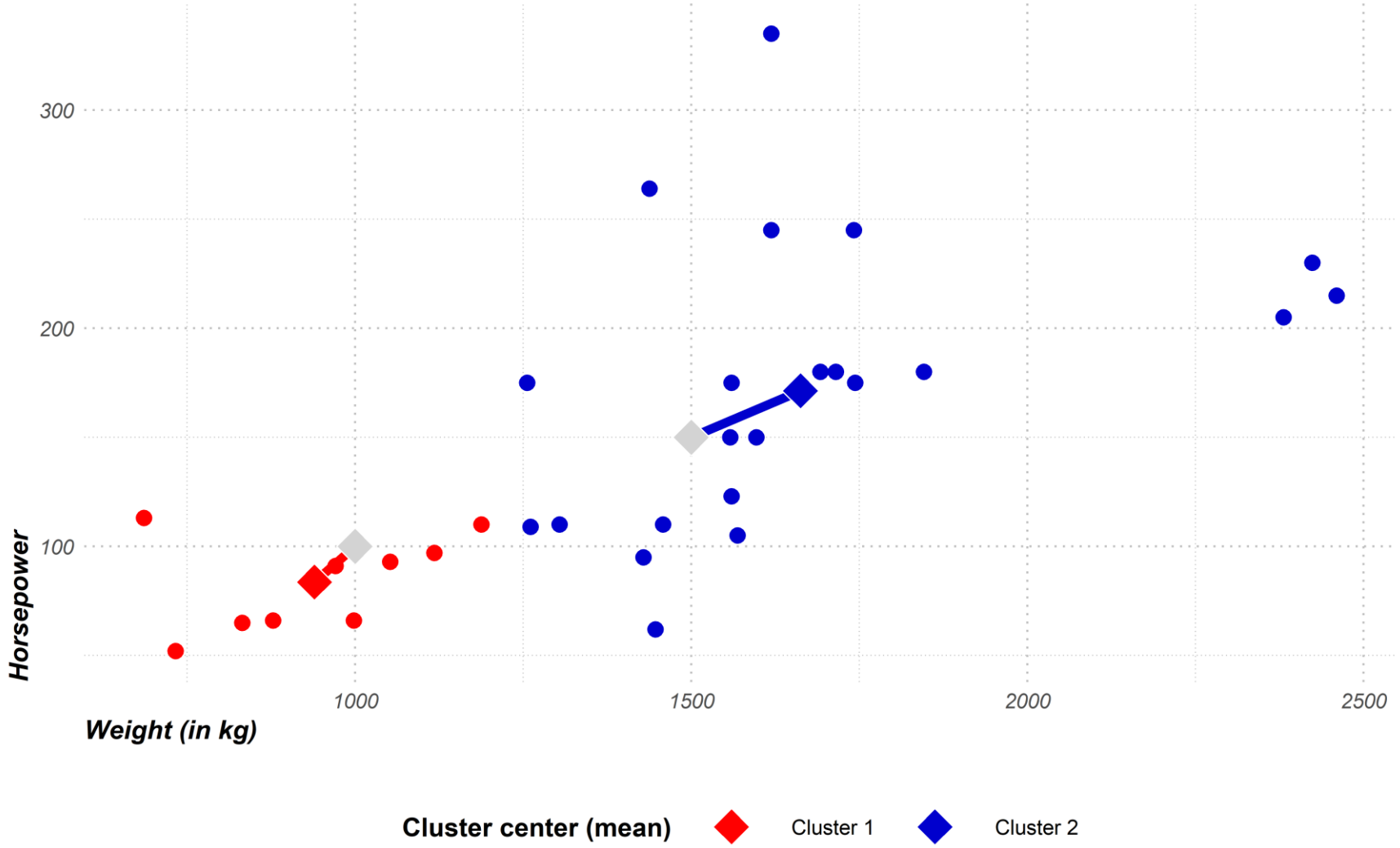
Machine Learning: *K-Mean*

Algorithm



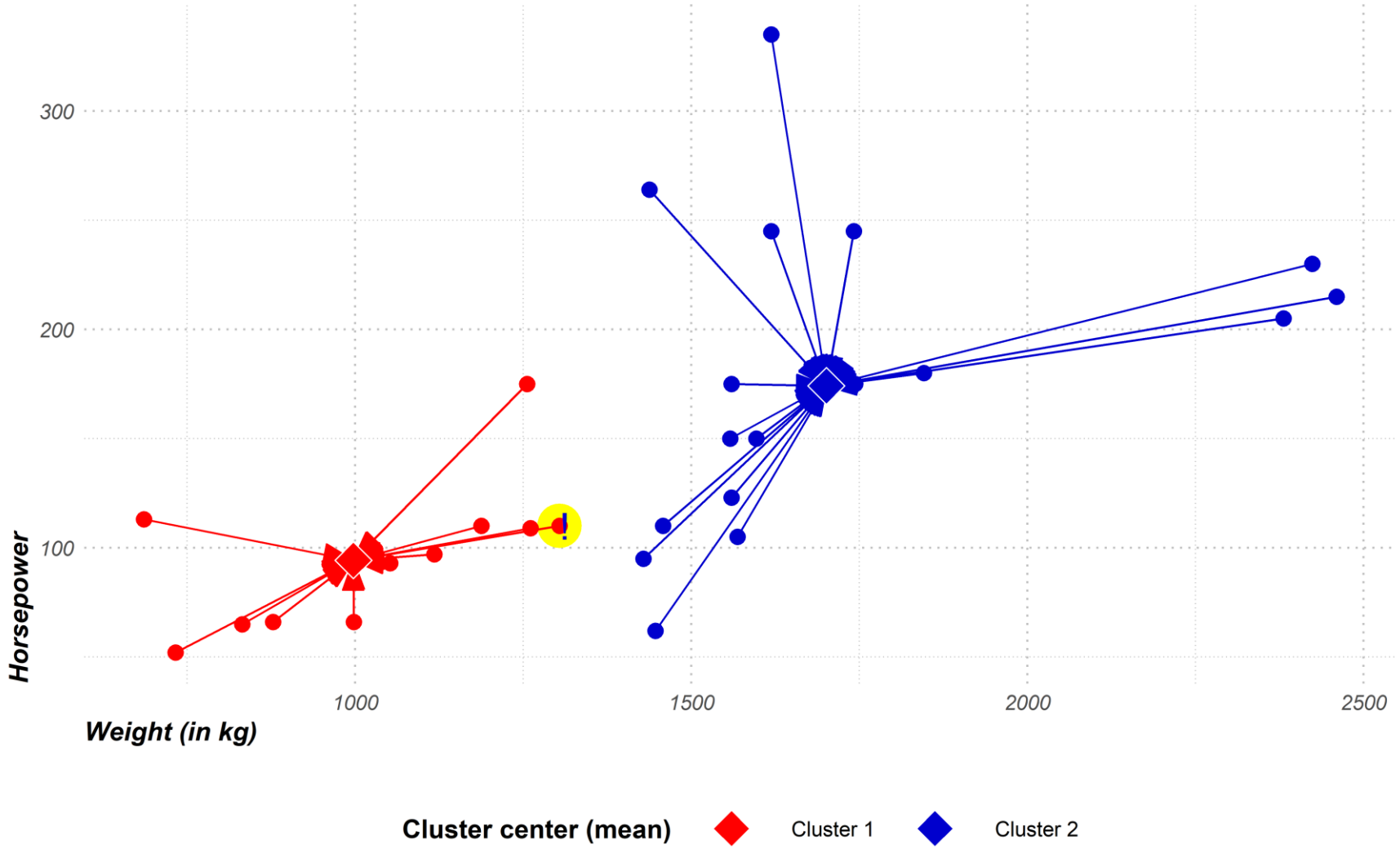
Machine Learning: *K-Mean*

Algorithm



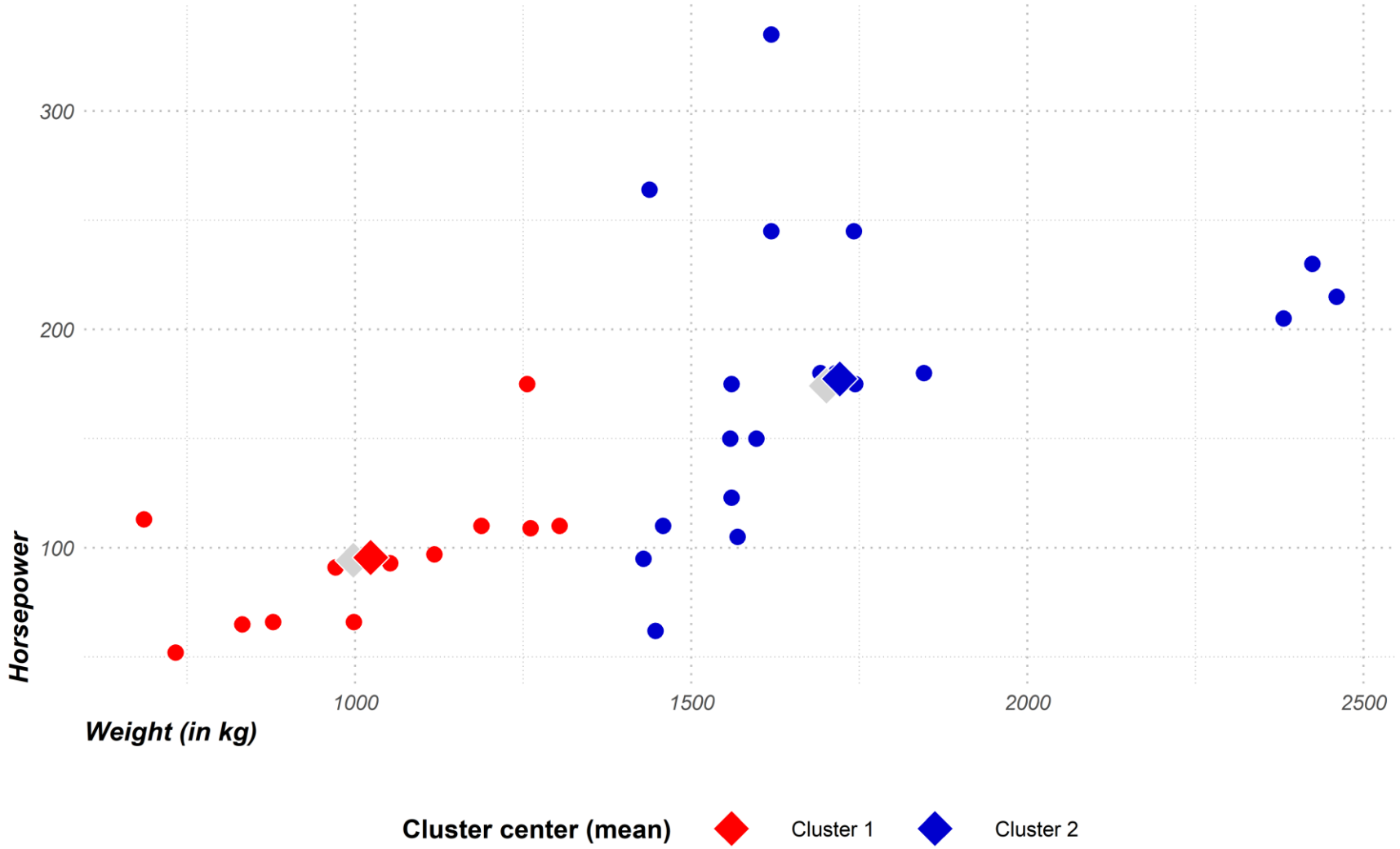
Machine Learning: *K-Mean*

Algorithm



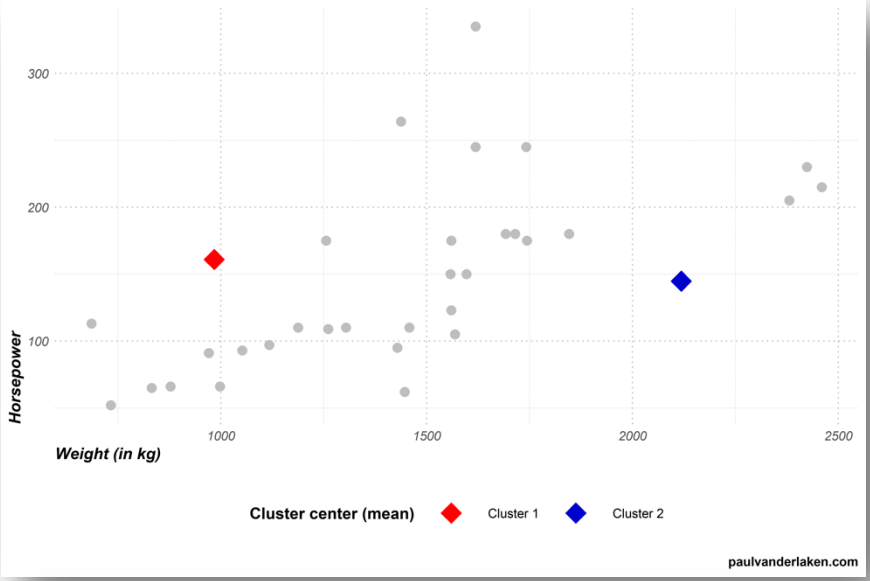
Machine Learning: *K-Mean*

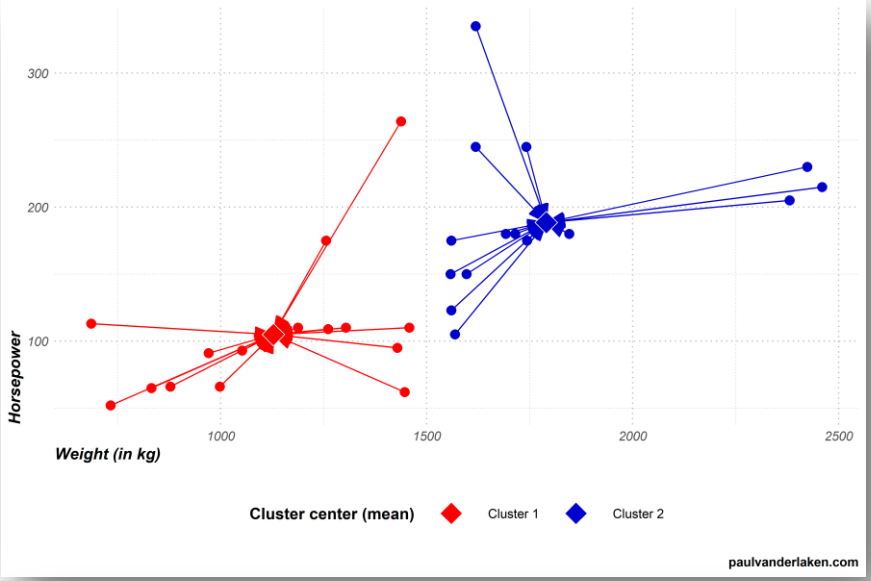
Algorithm

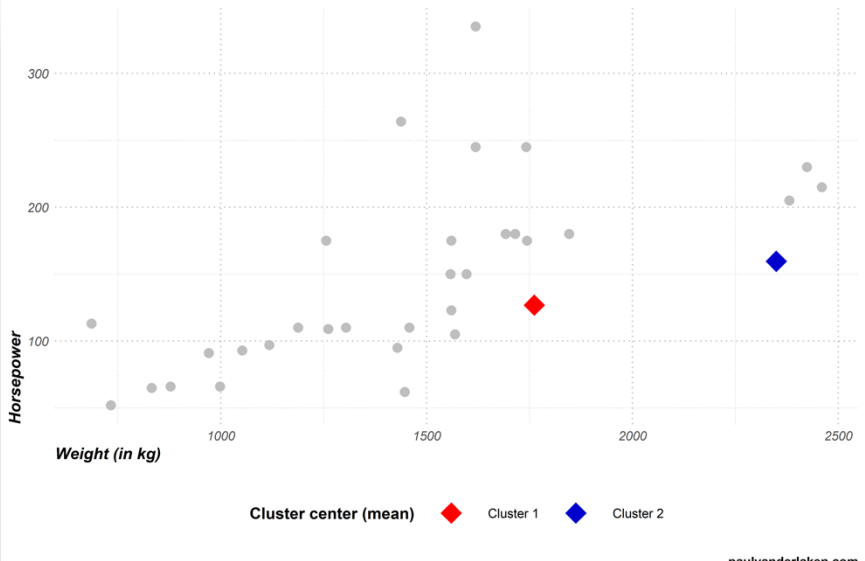


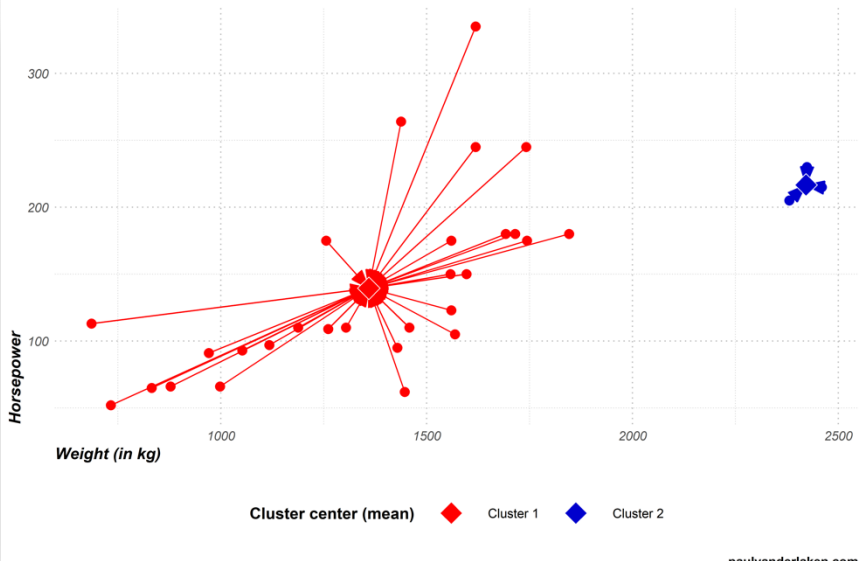
Machine Learning: *K-Mean*

Algorithm









Machine Learning: *K-Mean*

Algorithm

1. Randomly initialize K cluster centroids: $\mu_1, \mu_2, \cdots, \mu_K$
2. Repeat until no more change to cluster centroids

{

assign points to cluster centroid based on distance

for $i = 1{:}\,m$

$\min_k \left\| x^{(i)} - \mu_k \right\|^2 \rightarrow$ assign $c^{(i)}$ to μ_{k-min} (cluster centroid closest to $x^{(i)}$)

Update cluster centroids

for k$= 1{:}\,K$

$\mu_k =$ average of assigned points to cluster k

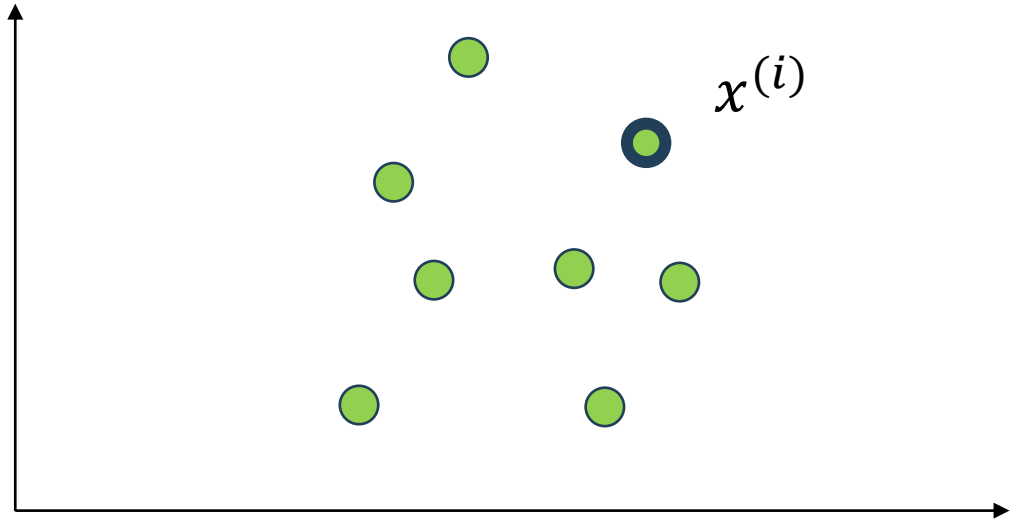
}

Machine Learning: *K-Mean*

Algorithm

Cost Function

$x^{(i)}$: data sample

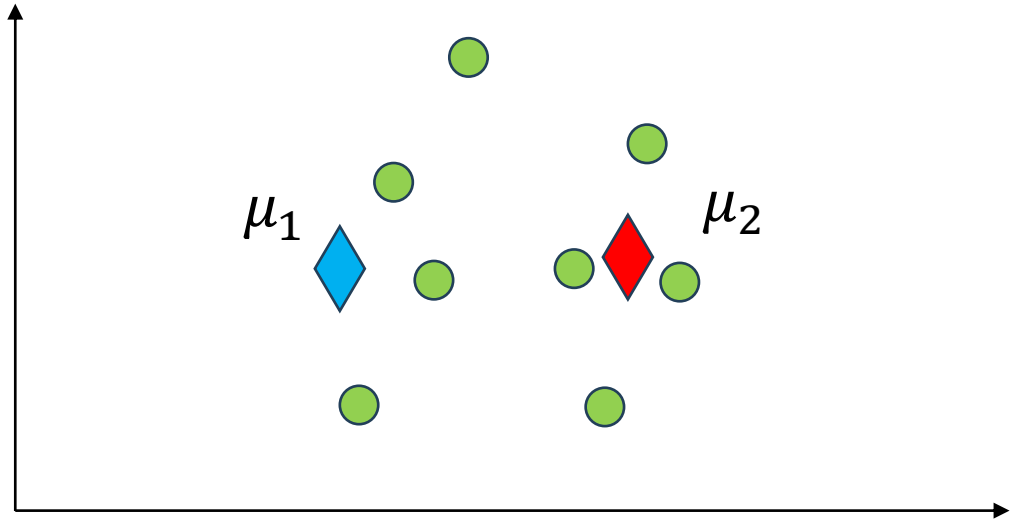


Machine Learning: *K-Mean*

Algorithm

Cost Function

μ_k: cluster centroid

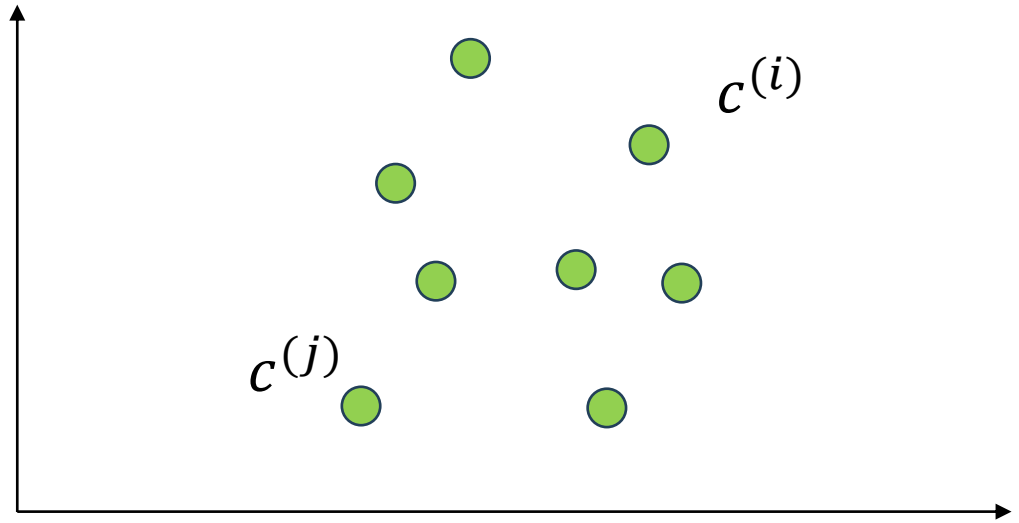


Machine Learning: *K-Mean*

Algorithm

Cost Function

$c^{(i)}$: index of cluster to which example $x^{(i)}$ is currently assigned to

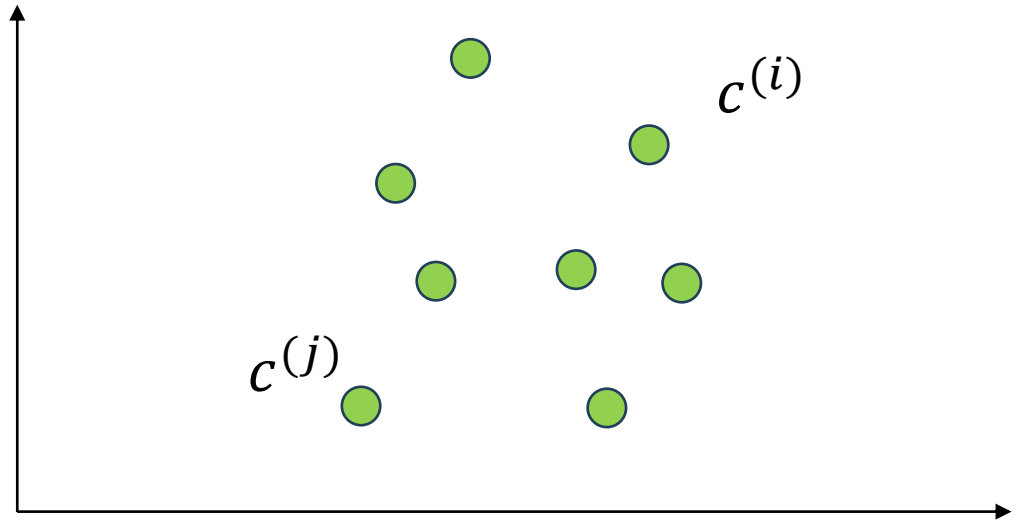


Machine Learning: *K-Mean*

Algorithm

Cost Function

$\mu_{c^{(i)}}$: cluster centroid of cluster to which example $x^{(i)}$ has been assigned to



$$\mu_{c^{(i)}} = \mu_1 \; or \; \mu_2$$

Machine Learning: *K-Mean*

Algorithm

Cost Function

$c^{(i)}$: index of cluster to which example $x^{(i)}$ is currently assigned to

μ_k: cluster centroid

$\mu_{c^{(i)}}$: cluster centroid of cluster to which example $x^{(i)}$ has been assigned to

$$J\left(c^{(1)}, \ldots, c^{(m)}, \mu_1, \ldots, \mu_K\right) = \sum_{i=1}^{m} \left\| x^{(i)} - \mu_{c^{(i)}} \right\|^2$$

$$\min_{\substack{c^{(1)},\ldots,c^{(m)} \\ \mu_1,\ldots,\mu_K}} J\left(c^{(1)}, \ldots, c^{(m)}, \mu_1, \ldots, \mu_K\right)$$

Machine Learning: *K-Mean*

Algorithm

$$J\left(c^{(1)}, \ldots, c^{(m)}, \mu_1, \ldots, \mu_K\right) = \sum_{i=1}^{m} \left\| x^{(i)} - \mu_{c^{(i)}} \right\|^2$$

1. Randomly initialize K cluster centroids: $\mu_1, \mu_2, \cdots, \mu_K$
2. Repeat until no more change to cluster centroids

{

assign points to cluster centroid based on distance

for $i = 1{:}\, m$

$\min_k \left\| x^{(i)} - \mu_k \right\|^2 \rightarrow$ assign $c^{(i)}$ to μ_{k-min}(cluster centroid closest to $x^{(i)}$)

Update cluster centroids

for k$= 1{:}\, K$

$\mu_k =$ average of assigned points to cluster k

}

Machine Learning: *K-Mean*

Algorithm

fixed

$$J\left(c^{(1)}, \ldots, c^{(m)}, \mu_1, \ldots, \mu_K\right) = \sum_{i=1}^{m} \left\| x^{(i)} - \mu_{c^{(i)}} \right\|^2$$

1. Randomly initialize K cluster centroids: $\mu_1, \mu_2, \cdots, \mu_K$
2. Repeat until no more change to cluster centroids

{

assign points to cluster centroid based on distance

for $i = 1{:}\,m$

$\min_k \left\| x^{(i)} - \mu_k \right\|^2 \rightarrow$ assign $c^{(i)}$ to μ_{k-min} (cluster centroid closest to $x^{(i)}$)

Update cluster centroids

for k$= 1{:}\,K$

$\mu_k =$ average of assigned points to cluster k

}

Machine Learning: *K-Mean*

Algorithm

fixed

$$J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K) = \sum_{i=1}^{m} \left\| x^{(i)} - \mu_{c^{(i)}} \right\|^2$$

1. Randomly initialize K cluster centroids: $\mu_1, \mu_2, \cdots, \mu_K$
2. Repeat until no more change to cluster centroids

{

assign points to cluster centroid based on distance

for $i = 1{:}\,m$

$\min_k \left\| x^{(i)} - \mu_k \right\|^2 \rightarrow$ assign $c^{(i)}$ to μ_{k-min} (cluster centroid closest to $x^{(i)}$)

Update cluster centroids

for k= $1{:}\,K$

μ_k = average of assigned points to cluster k

}